Università di Pisa

Dipartimento di Informatica
Corso di Laurea Magistrale in Informatica

Compressed Representations of Set Collections

Candidato
Luca Lombardo

Relatore
Roberto Grossi

Anno Accademico 2025/2026

Abstract

This thesis studies compressed representations of collections of sets drawn from a common ordered universe. The representation must support standard ordered-set queries on each set: membership, access by position, rank (which counts elements up to a value) predecessor, and successor. A natural baseline is to compress each set in isolation, treating it only as a subset of the universe. Such an encoding may exploit the internal structure of each set, but it cannot use the redundancy that appears only when the sets are viewed together. Containment, overlap, and small differences between sets are precisely the regularities lost by this isolated view. Existing relation-based representations exploit these regularities by storing one set through another and answering queries along labelled trees. Their references, however, are usually restricted to the input sets or to a graph on them, so they do not measure how much information the whole collection contains.

To measure the redundancy that is visible only at collection level, we change how we look at the collection. Instead of describing one set at a time, we look at each universe element and record which input sets contain it. Elements that appear in exactly the same input sets can be grouped together. If the groups that occur and their sizes are known, describing the collection reduces to deciding which labelled universe elements belong to each group. Counting these assignments gives a reference for the bits needed to describe how elements are distributed across the collection. Taken alone, however, this reference is only a count. It says how many bits are needed in principle, but not how to organize those bits so that membership, access, rank, predecessor, and successor on a single set can be answered without decoding the global assignment or adding separate indexes.

We introduce Set-Union Matching (SUM) to turn this counting view into a queryable hierarchy. Existing encodings can exploit a relation between two sets in two natural ways: if one contains the other, the smaller set can be recovered by deleting elements from the larger one, and if two sets are close, one can be described by recording the elements on which they differ. SUM keeps the first style by creating, for each chosen pair of current sets, their union as the smallest common parent, so that both children are recovered from the parent only by deleting the elements that do not belong to the corresponding child and every edge in the hierarchy has the same containment meaning. Repeating this step in rounds builds a hierarchy of such parents, using the saving exposed by each union to decide which pairs to merge and keeping the least costly hierarchy reached by the construction. The resulting count never exceeds the isolated-set baseline and improves on it when the hierarchy captures shared structure, while the global assignment count remains the lower benchmark for the hierarchy model considered in the thesis. Because the hierarchy uses only containment edges, each connected part becomes a deletion tree, where a set is recovered from its root by removing the labels found along one root-to-leaf path. A dictionary for each root and path-query indexes on the tree then support membership, access, rank, predecessor, and successor, with logarithmic terms governed by the root of that tree.

Preface

The interest in the topics behind this thesis stems from a talk at SEA 2025, held during the Festschrift for Roberto Grossi's sixtieth birthday [1], where Alanko, Bille, Gørtz, Navarro, and Puglisi presented [2]. The paper presents a compact representation that encodes each set as a subset of a larger one in the collection and supports membership, rank, predecessor, and successor. The idea of encoding each set inside a larger one and recovering it by traversing the containments was close to what we worked on in my bachelor thesis on succinct data structures over directed acyclic graphs [3].

Their framework exploits redundancy only when one set in the collection is a subset of another. Two sets that overlap heavily but neither contains the other lie outside this measure, even though their overlap is the same kind of redundancy a compressed representation should capture. The question we pursued was how to extend the compressibility regime to such pairs while keeping the queries efficient.

The algorithm we present in Chapter 5 keeps the containment style by adding synthetic union nodes as common parents for pairs of overlapping sets. A child is then recovered from its union parent by deleting the elements outside it, so the hierarchy has the same containment meaning on every edge. Independently and in parallel, Gagie, He, and Navarro pursued the same problem through symmetric differences: their representation describes one set from a nearby one by the elements on which they differ, at the price of a more involved access procedure. These two routes make the contrast between containment paths and difference-based descriptions explicit, and the comparison closes Chapter 5.

This thesis is one of several projects I have worked on during my bachelor's and master's years, all sitting along the line between combinatorial algorithm design, succinct data structures, algorithm engineering, and high-performance computing. The most rewarding ones have always been those that demanded all four at once.

Contents

Contents iv

1 Introduction 1
- 1.1 Why compress set collections? . . . 1
- 1.2 Problem setting . . . 2
- 1.3 Thesis roadmap . . . 4

2 Succinct Dictionaries 6
- 2.1 Rank and Select . . . 6
- 2.2 Sparse Bitvectors . . . 15
- 2.3 Wavelet trees . . . 19

3 Succinct Representations of Trees 25
- 3.1 Encoding problem . . . 25
- 3.2 Depth-first encodings . . . 29
- 3.3 Fully functional encodings . . . 34
- 3.4 Labeled trees and α-operations . . . 38
- 3.5 Tree extraction . . . 41
- 3.6 α-operations on labeled trees . . . 43
- 3.7 Path queries . . . 46

4 Hierarchical Encodings 54
- 4.1 Setup and baseline . . . 54
- 4.2 Insertion compressibility . . . 55
- 4.3 Symmetric-difference compressibility . . . 61

5 Set-Union Matching 71
- 5.1 Information theoretic bound . . . 71
- 5.2 Union matching . . . 76
- 5.3 Queries . . . 83

6 Conclusions 89

Appendix 91

A Lattice Models 92
- A.1 Lattice closure . . . 92
- A.2 Binary case . . . 99

Bibliography 104

List of Figures

2.1 A bitvector $B[1..20]$. The prefix $B[1..15]$ (shaded red) contains nine ones, so $\text{rank}_1(B, 15) = 9$. The seventh one (circled blue) lies at position 12, so $\text{select}_1(B, 7) = 12$. 7

2.2 The two-level rank directory of Jacobson's construction. Superblocks of size Z store absolute ranks r_i; blocks of size z inside each superblock store ranks $r_{i,j}$ relative to the enclosing superblock. The bottom row represents the four-Russians lookup table, indexed by the z-bit intra-block pattern and an in-block offset, returning the rank inside the pattern. 8

2.3 Elias-Fano encoding of the set $\{5, 7, 8, 15, 32\}$ with universe $n = 36$ and $m = 5$, so $w = 6$ and $z = \lfloor \lg 5 \rfloor = 2$. Each element is split into a high part of z bits and a low part of $w - z = 4$ bits. The array L is the concatenation of the low parts in order; the bitvector H unary-codes the successive increments of the high parts $(0, 0, 0, 0, 2)$. To answer $\text{select}_1(j)$, locate the j-th one in H at position $p = \text{select}_1(H, j)$, recover the high part $q_j = p - j$, and concatenate it with the low part $L[j]$. . . 16

2.4 Wavelet tree for the string $S =$ `wookies_wield_wicked_weapons_with_wisdom$` over an alphabet of 17 symbols. Each internal node carries a sub-alphabet interval. Its bitmap encodes, for every symbol of the implicit subsequence, which side of the midpoint split it falls on. The subsequences are drawn beside each node for illustration. The structure stores only the topology and the bitmaps. 20

2.5 Trace of $\text{rank}_{\texttt{e}}(S, 13) = 2$ on the wavelet tree of Figure 2.4. The descent follows the root-to-leaf path of the target symbol `e`, remapping the query prefix length through one binary rank at each of four levels. The intermediate values $13 \to 6 \to 5 \to 3 \to 2$ trace the query index from the root to the leaf `[e, e]`. Each remapping is a rank_0 or rank_1 on the local bitmap, selected by whether `e` falls in the left or right half of the current alphabet interval. 21

3.1 LOUDS encoding of an ordinal tree on seven nodes. The super-root $\perp$ is added so that every node, including the original root, has a unique parent and contributes a unary degree codeword. The augmented tree is traversed in BFS order, the BFS indices $1, \ldots, 8$ are shown next to each node, and the degree of each node in turn is written in unary as 1^d0, concatenated to form the bitmap B. Grouping braces above B mark the codewords contributed by the super-root and by nodes a, b, c, d, e, f, g in BFS order. 28

3.2 Balanced parenthesis encoding of the same seven-node ordinal tree used in Figure 3.1. A preorder DFS visit writes an open parenthesis on entry to a node and a close on exit. Each dashed arc joins the matching pair that represents one node, labelled by that node. The substring from a node's open parenthesis through its matching close is exactly the encoding of its subtree, of length $2 \cdot |T_v|$. 30

3.3 DFUDS encoding of the same seven-node ordinal tree of Figure 3.2. Each node v contributes a block of d_v open parentheses followed by one close, written in DFS preorder. A leading open parenthesis (block $\perp$) is prepended so that the resulting string of length $2n = 14$ is balanced. The colored blocks above the sequence highlight the per-node contributions of a, b, e, c, d, f, g in preorder. The position p_v of the first parenthesis of each block is the address by which the node is referenced. 33

3.4 A schematic two-level tree covering of an ordinal tree. Solid rectangles show minitree regions produced with parameter M. Dashed rectangles show microtree regions produced inside them with parameter $M' < M$, including the possible smaller piece that contains the root of a recursive cover. Different pieces may intersect only at a common root. The exact size bounds are those of Lemma 3.3.1. The drawing shows nesting and boundary sharing on a tree too small for the asymptotic parameters used in the representation. 35

3.5 X-extraction on a sixteen-node ordinal tree, reproduced from Gagie, He, and Navarro [9]. On the left, T has $X = \{A, B, D, E, F, H, I, J, K, O\}$ as the shaded nodes. On the right, the extracted tree T_X contains exactly those nodes. The deletion of each unshaded node promotes its children into the sibling list of its parent, and the natural bijection between X and $V(T_X)$ sends every shaded node to the position it occupies on the right. 41
3.6 From left to right, an ordinal tree T on five nodes with labels in $\{1, 2, 3\}$; the three extracted trees T_1, T_2, T_3 built on the X_α-extractions described in the text, with 0/1 markers drawn as empty and shaded circles; and the merged tree $\mathcal{T}$ obtained by appending the T_α as subtrees of a fresh root $\mathcal{R}$. The α-operations on T are answered by the 0/1 α-operations on the single tree $\mathcal{T}$, which is the role of the building block $\mathcal{D}_3$ of Lemma 3.6.1. 45
3.7 Hierarchical binary partition of a labeled ordinal tree for root-to-node path queries, reproduced from Gagie, He, and Navarro [9]. Starting from the root range, each intermediate tree $\mathcal{T}_{a,b}$ is 0/1-labeled by the midpoint of its range. Bold arrows lead to the child trees $\mathcal{T}_{a,m}$ on marker 0 and $\mathcal{T}_{m+1,b}$ on marker 1. The binary alphabet at every level makes each level transition cost $O(1)$ via a single rank or select on the marker sequence. 48

4.1 Reproduced from Gagie, He, and Navarro [9]. Left: an insertion graph for a collection $\mathcal{S} = \{S_1, \ldots, S_9\}$, with edge weights written as slanted values. The edge weights sum to $I(\mathcal{S}) = 20$. Right: the corresponding insertion tree obtained by expanding each edge $(v(p'(S)), v(S))$ of weight $w = |S \setminus p'(S)|$ into a chain of w labelled edges. The tree has $1 + I(\mathcal{S}) = 21$ nodes. Each set S_i labels the chain endpoint $v(S_i)$. 59
4.2 Reproduced from Gagie, He, and Navarro [9]. Left: a minimum-weight symdiff graph on $\mathcal{S} = \{S_1, \ldots, S_9\}$ with $\Delta(\mathcal{S}) = 13$. Dashed edges are available in the complete graph K but not used by the MST; they are the edges whose weight exceeds the largest MST edge weight $\ell = 2$. Right: the two indel trees rooted at U (drawn upside down) and at $\emptyset$. Chain labels $+x$ are insertions, $-x$ are deletions relative to the parent. 67
4.3 Hierarchical extraction of one indel-tree hierarchy, reproduced from Gagie, He, and Navarro [9]. The diagram shows GHN's worked example $\mathcal{T}^- = \mathcal{T}^-_{0,7}$ obtained from the indel tree rooted at $v(U)$ of Figure 4.2 (drawn upside down); the subscript $[0..7]$ reflects GHN's 0-indexed eight-element universe, while the thesis prose uses $[1..u]$. At each intermediate range $[a..b]$, the 0/1-labelled tree $\mathcal{T}^-_{a,b}$ extracts to $\mathcal{T}^-_{a,m}$ on label 0 and to $\mathcal{T}^-_{m+1,b}$ on label 1, following the bold rightward arrows. The same construction yields $\mathcal{T}^+$ on insertion marks; the coordinated descent of this subsection tracks images v^+ and v^- in two parallel instances of this structure. 69

5.1 The two bitvectors used to encode the three regions of M. The top row fixes the order of M. The first bitvector marks the k positions of $A \cap B$. The unmarked positions are compacted, and the second bitvector marks the l positions of $A \setminus B$. The remaining positions belong to $B \setminus A$. The number of choices is $\binom{|M|}{k}\binom{l+r}{l} = \binom{|M|}{k,l,r}$. 77
5.2 Trivial forest F_0 on $\mathcal{S} = \{\{1\}, \{2\}, \{3\}\}$ over $U = \{1, 2, 3\}$, and a level-1 candidate F_1 obtained by merging two singletons under the synthetic union $M_{12} = S_1 \cup S_2$. The costs are $\Phi(F_0) = 3 \lg 3$ and $\Phi(F_1) = 2 \lg 3 + 5$. Every level-1 candidate raises the objective, so SUM records F_0 and returns it. 80

1 Introduction

1.1 Why compress set collections? 1
1.2 Problem setting 2
1.3 Thesis roadmap 4

Compression starts from the idea that a representation should spend bits only on information needed to identify the object. If two descriptions lead to the same object, the shorter one has removed some redundancy. This viewpoint abstracts away from file formats and coding schemes, because it treats the object as the thing to be identified and the representation as the information needed to identify it.

Kolmogorov complexity gives this ideal a precise form. After fixing a universal machine, descriptions become programs for that machine, and the Kolmogorov complexity of a binary string is the length of the shortest program that prints it [4]. In this sense, a highly regular string is compressible because a short program can describe the rule that generates it, while an irregular string may have no description substantially shorter than itself.

[4]: Li et al. (2008), *An introduction to Kolmogorov complexity and its applications*

Kolmogorov complexity marks the limit obtained by removing every redundant bit from an individual object, but this limit is not computable and does not prescribe a data structure. A shortest program that prints a set may describe it succinctly while giving no efficient way to answer queries about its elements.

We therefore work with computable counting bounds. Once an object is known to belong to a finite family $\mathcal{F}$, any lossless representation that distinguishes all objects in $\mathcal{F}$ needs at least $\lg |\mathcal{F}|$ bits in the worst case. For a single set, the family can be counted directly. A subset of size n drawn from a universe of size u can be chosen in $\binom{u}{n}$ ways, so the counting cost of that set is $\lg \binom{u}{n}$ bits.

For a collection $\mathcal{S} = \{S_1, \ldots, S_m\}$, the first baseline describes each set separately and adds these costs. This baseline is valuable because it states the price of ignoring relations between sets. The subject of the thesis begins when those relations carry information. One set may contain another, and two sets may overlap so much that describing them independently repeats the same choices. The challenge is to turn such relations into a smaller description while preserving a structure that later queries can follow.

1.1 Why compress set collections?

A collection of sets appears when many objects are described through their associations with one shared universe. Each object contributes one subset, and the whole dataset becomes a family of subsets of the same ground set. The domain fixes what the universe means, but the compression problem begins once the subsets are known. At that point we can ask how much information the collection contains and which operations a compressed representation should support.

The universe is fixed first. Each stored object is represented by the subset of universe elements associated with it.

Compression becomes interesting when the subsets are dependent. Related objects often select overlapping parts of the universe, or differ

only by a small boundary around a common core. If we encode each set as a fresh subset of the whole universe, these relations disappear from the count. The compression problem is therefore to exploit dependence between sets without losing direct access to the set named by a query.

In web graphs, this dependence appears in the adjacency lists. A graph representation must store the successor list of each page, and a query may later ask for the neighbours of one selected page. Boldi and Vigna show that URL order exposes two forms of dependence. Successors of one page often have close identifiers, and pages close in URL order often share successors. Their WebGraph format stores gaps between consecutive successors, then represents a list by referring to a nearby list, marking copied portions, and storing the remaining extra nodes [5]. The compression gain comes from the relation between adjacency sets, while the data structure still has to answer graph queries.

[5]: Boldi et al. (2004), *The webgraph framework I: compression techniques*

Inverted indexes have the same shape with documents as the universe. For each distinct term, the index stores the sorted list of document identifiers containing that term. Query processing then scans or intersects the lists selected by the query, so compression cannot be separated from access to individual lists. Pibiri and Venturini survey this problem as inverted index compression, where the lists must occupy little space and still support fast query processing [6]. For the present model, the relevant abstraction is that the index is a large family of subsets of one document universe. Similar terms can induce similar lists, and the collection may contain information that a per-list encoding never sees.

[6]: Pibiri et al. (2020), *Techniques for Inverted Index Compression*

In coloured de Bruijn graphs, the universe is the set of genomes. The graph represents sequence fragments, and the colour set of a vertex records the genomes in which that fragment occurs [7]. Since neighbouring fragments often appear in nearly the same genomes, adjacent vertices often have identical or similar colour sets. Almodaresi et al. use this dependence by encoding colour classes through small differences from related classes [8]. Query access remains local. A compressed pangenomic index must answer which genomes contain a fragment without expanding all colour sets.

[7]: Alanko et al. (2023), *Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes*

[8]: Almodaresi et al. (2020), *An efficient, scalable, and exact representation of high-dimensional color information enabled using de Bruijn graph search*

In each case, the data contain many subsets of one universe, the subsets depend on one another, and the operations still ask about selected sets. A useful compression measure must say how much is saved by exploiting relations such as containment, overlap, or small difference. A useful representation must then realise that saving without turning the collection into an inaccessible archive. We now fix the common model and the independent baseline against which such savings will be measured.

1.2 Problem setting

We work with many finite sets over one ordered universe. The order lets the operations ask whether an element is present and where it lies among the elements of a set. The following definition fixes the notation used by the whole thesis.

Definition 1.2.1 (Set collection) *A set collection over a universe U is a family $\mathcal{S} = \{S_1, \ldots, S_m\}$ of m finite subsets of a finite, totally ordered set U. We call U the universe of the collection. We write $u = |U|$ for the size of the universe and $n = \sum_{i=1}^{m} |S_i|$ for the total size of the collection. We assume $U = [1..u] = \{1, 2, \ldots, u\}$ without loss of generality, since any totally ordered set of size u can be mapped to this interval by a rank function after preprocessing in $O(n \lg u)$ time.*

If the universe is not $[1..u]$, one collects the n elements, sorts them, and assigns integers in $[1..u]$ to the u distinct values. Queries and answers translate in $O(1)$ time via a lookup table.

Each set S_i inherits the total order from U, so its elements can be listed as $S_i[1] < S_i[2] < \cdots < S_i[|S_i|]$. The integer n counts elements with multiplicity across sets. If one universe element belongs to k sets, it contributes k to n. The parameter n can be much smaller than $m \cdot u$, but the representation must still distinguish the sets and support queries on each of them.

A representation of $\mathcal{S}$ must recover information about any chosen set S_i without decompressing the whole collection. Gagie, He, and Navarro [9] state the same five operations, and Alanko et al. [2] use the equivalent bitvector formulation obtained by identifying each set with its characteristic vector.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

[2]: Alanko et al. (2025), *Compact data structures for collections of sets*

Definition 1.2.2 (Operations on set collections) *Given a set collection $\mathcal{S}$ over $U = [1..u]$, we define five operations on each $S \in \mathcal{S}$:*

- member(S, x)*: returns true if $x \in S$ and false otherwise.*
- access(S, i)*: given $i \in [1..|S|]$, returns the element of rank i in S, that is, $S[i]$.*
- rank(S, x)*: returns $|\{y \in S : y \leq x\}|$, the number of elements of S at most x.*
- predecessor(S, x)*: returns $\max\{y \in S : y \leq x\}$, or $-\infty$ if no such element exists.*
- successor(S, x)*: returns $\min\{y \in S : y \geq x\}$, or $+\infty$ if no such element exists.*

These five operations are the primitives on which higher level tasks depend. Set intersection, for instance, can be computed by interleaving predecessor and successor queries on both operands, advancing through the sorted elements in tandem.

Predecessor and successor reduce to rank followed by access. A predecessor query performs one rank computation and one access computation, and a successor query does the same. The three operations member, rank, and access are therefore the independent primitives. Any representation that supports these three automatically supports all five, at the cost of one additional access per predecessor or successor query.

To compute predecessor(S, x), first compute $r = \text{rank}(S, x)$. If $r = 0$, no predecessor exists. Otherwise, return access(S, r). Successor works symmetrically.

For a finite class of objects $\mathcal{F}$, the worst-case entropy is $\lg |\mathcal{F}|$, the number of bits needed to distinguish one object of the class when no probability model is assumed [10]. To apply this count to a set collection, we first ignore all relations between different sets and fix only the cardinalities. A set $S_i \subseteq U$ of size $|S_i|$ is one of $\binom{u}{|S_i|}$ possible subsets of U of that size. Any encoding that distinguishes all such subsets uses at least $\lg \binom{u}{|S_i|}$ bits for S_i. If the sets are described independently, the admissible class has size $\prod_{i=1}^{m} \binom{u}{|S_i|}$, and its logarithm is the sum below.

[10]: Navarro (2016), *Compact data structures: A practical approach*

Definition 1.2.3 (Independent encoding cost) *The independent encoding cost of a set collection $\mathcal{S}$ over U is*

$$H_{wc}(\mathcal{S}) = \sum_{i=1}^{m} \lg \binom{u}{|S_i|}.$$

The notation $H_{wc}(\mathcal{S})$ records that this is the worst-case entropy of the independent class determined by the set sizes. Alanko, Bille, Gørtz, Navarro, and Puglisi use the same quantity for set collections, written $H(\mathcal{S})$ [2]. We keep the subscript to distinguish this reference point from the finer measures introduced later. The bound is achievable up to lower order terms. Encoding each set independently costs $H_{wc}(\mathcal{S})+O(n+m \lg n)$ bits, where the $O(m \lg n)$ term gives a directory for the m sets and the $O(n)$ term accounts for the overheads of the sparse bitvectors. The representation supports rank and constant time access to the elements of any set.

The independent cost is the reference point for representations that ignore relations between sets. If the representation may use a relation, the class to be counted can become smaller. If $S_i \subset S_j$, then describing S_i relative to S_j means choosing $|S_i|$ elements from a universe of size $|S_j|$, for a cost of $\lg \binom{|S_j|}{|S_i|}$ bits. Containment-based representations use exactly this smaller universe when a contained set is encoded through a containing set [2]. If two sets differ in few elements, the description can record the small change between them. We can recover one set from the other by storing the insertions and deletions that separate them, which is the set-difference viewpoint of Gagie, He, and Navarro [9]. Thus $H_{wc}(\mathcal{S})$ measures the cost before inter-set relations are used, and later measures drop below it only when such relations become part of the description.

After fixing the independent count, two questions remain. We first need a measure that tells which relations between sets shrink the admissible class below the one counted by $H_{wc}(\mathcal{S})$. We then need a representation that stores such a description while queries still run on the compressed data. Section 1.3 explains how the chapters follow these questions.

1.3 Thesis roadmap

The independent count of Definition 1.2.3 is meaningful only if a single ordered set can be stored near its own counting bound and still queried. We therefore start from the one-set problem. Chapter 2 develops bitvectors, sparse dictionaries, and wavelet trees as concrete representations for membership, access, and rank on ordered sets. These structures give the independent representation that reaches $H_{wc}(\mathcal{S})$ up to lower order terms when no relation between sets is used.

Relations between sets change the shape of the query problem. If a set is stored through another set, a query cannot stop at one dictionary. It must follow a reference path and account for the changes stored along that path. Chapter 3 develops the tree representations and path-query primitives needed for this step. Succinct ordinal trees represent the shape of a hierarchy, and labelled tree extraction turns path labels into the counting and selection operations needed by set queries.

Single set dictionaries and the path-query framework of Chapter 3 make relation-based compression queryable. Chapter 4 develops the recent state of the art in this direction, namely containment, insertion, and symmetric difference compressibility. Each choice changes the local description used on an edge. A containment edge records deletions from a larger set, an insertion edge records additions from a smaller set, and a symmetric difference edge records both additions and deletions. These regimes show the constraint that every later construction must satisfy. A smaller count matters only when it also produces paths on which the operations of Definition 1.2.2 can run.

Those constructions still choose their references among the sets already present in the collection. This restriction misses pairs of sets that overlap heavily without either set containing the other, since no existing containment parent is available. Chapter 5 gives the main contribution of the thesis by allowing the hierarchy to create such parents. It starts by counting the atoms of the membership table, which gives a benchmark for the information content of the whole collection. This atom count measures the membership table at once, but it does not specify a path for queries. The chapter then uses synthetic union nodes to create those paths. When two incomparable sets share many elements, their union becomes a common parent, and the two child edges become containment edges. The construction keeps the query structure of deletion hierarchies while allowing the hierarchy to use references that were absent from the input.

Appendix A examines the same principle in a larger reference space. Once the representation may add union nodes, it is natural to ask what changes if it may also add larger unions, intersections, or mixed references. Closing the collection under union and intersection exposes the lattice of possible references, while bounded arity measures how much is lost when the hierarchy insists on binary splits. The appendix places the binary-union construction inside this larger optimisation problem and separates the tractable subproblem restricted to unions from the broader lattice search.

2 Succinct Dictionaries

2.1 Rank and Select 6
2.2 Sparse Bitvectors 15
2.3 Wavelet trees 19

A single set $S \subseteq [1..u]$ is a binary-alphabet problem. Its characteristic bitvector turns the five operations of Definition 1.2.2 into questions about counts and positions of ones, answered by the primitives rank and select.

To represent many sets we first need to represent one, and representing one reduces to rank and select on a characteristic bitvector. Section 1.2 left two questions open, a combinatorial one about the right measure of compressibility and an algorithmic one about a representation that supports the five operations of Definition 1.2.2. This chapter takes up the algorithmic question for a single set $S \subseteq [1..u]$. The characteristic bitvector $B[1..u]$ records S position by position: $B[i] = 1$ if and only if $i \in S$, and the map $S \mapsto B$ is a bijection between subsets of $[1..u]$ and binary strings of length u. Each operation of Definition 1.2.2, evaluated on S, becomes a question about counts and positions of ones in B: membership asks whether $B[x] = 1$; access asks for the position of the i-th one; rank asks for the number of ones in a prefix; predecessor and successor combine these. The chapter measures both costs, how fast these queries run on a bitvector $B[1..n]$ and how many bits of auxiliary storage sit on top of the n bits of B itself; we write n for the bitvector length throughout, with $n = u$ in the single-set case.

2.1 Rank and Select

Throughout this chapter we work in the word RAM model with word size $\omega = \Omega(\lg n)$, so that a machine word holds a bitvector index and the bit-level primitives below admit constant-time arithmetic, bitwise, and shift operations. All space bounds are measured in bits, and lg denotes $\log_2$. Let $B[1..n]$ be a bitvector of length n. The two primitives we study are defined as follows.

Definition 2.1.1 (Rank) *For $b \in \{0,1\}$ and $1 \le i \le n$, the operation $\mathrm{rank}_b(B,i)$ returns the number of occurrences of b in the prefix $B[1..i]$,*

$$\mathrm{rank}_b(B,i) \;=\; |\{j : 1 \le j \le i,\ B[j] = b\}|.$$

By convention $\mathrm{rank}_b(B,0) = 0$.

Definition 2.1.2 (Select) *For $b \in \{0,1\}$ and $j \ge 0$, the operation $\mathrm{select}_b(B,j)$ returns the position in B of the j-th occurrence of b, that is, the unique index i such that $B[i] = b$ and $\mathrm{rank}_b(B,i) = j$. By convention $\mathrm{select}_b(B,0) = 0$, and $\mathrm{select}_b(B,j) = n+1$ whenever $j > \mathrm{rank}_b(B,n)$.*

The two operations are mutual inverses on their domains. For any i with $B[i] = b$ we have $\mathrm{select}_b(B, \mathrm{rank}_b(B,i)) = i$, and for every valid j we have $\mathrm{rank}_b(B, \mathrm{select}_b(B,j)) = j$. The identity $\mathrm{rank}_0(B,i) = i - \mathrm{rank}_1(B,i)$ reduces rank for the two values of b to a single subtraction, so only rank_1 needs direct support; select admits no analogous identity, and select_0 requires an auxiliary structure separate from select_1.

Figure 2.1 fixes a concrete bitvector of length $n = 20$ with population count 11. The red prefix gives $\text{rank}_1(B, 15) = 9$, and the blue-circled bit gives $\text{select}_1(B, 7) = 12$.

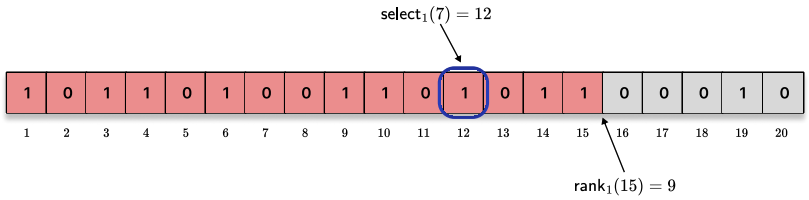

Figure 2.1: A bitvector $B[1..20]$. The prefix $B[1..15]$ (shaded red) contains nine ones, so $\text{rank}_1(B, 15) = 9$. The seventh one (circled blue) lies at position 12, so $\text{select}_1(B, 7) = 12$.

Without auxiliary structures, both queries reduce to a linear scan. Rank scans i bits and counts the ones; select scans from the left and returns the position once it has counted j ones. In the worst case, both queries take $\Theta(n)$ time, with no extra storage beyond the n bits of B itself and $O(\lg n)$ bits for a loop counter.

The linear-time scan is acceptable for short bitvectors, but the set collections of later chapters need fast repeated queries on much longer inputs. We therefore adopt $O(1)$ query time as the goal, and measure the price in extra space above B. Distinguishing the 2^n bitvectors of length n takes at least n bits, and B already attains this lower bound, so the question is how many bits of auxiliary storage in addition to B are needed to support $O(1)$-time rank and select. An auxiliary index of size $o(n)$, growing strictly slower than n, gives a total of $n + o(n)$ bits, the target Jacobson called *succinct* [11].

A succinct representation adds only $o(n)$ bits to the verbatim n-bit input and answers each query in $O(1)$ time, matching the information-theoretic minimum up to a sublinear term.

[11]: Jacobson (1988), *Succinct Static Data Structures*

The primitives of this section recur throughout the rest of the thesis as black-box building blocks, and every bound built on them inherits their query time: a linear-time rank would propagate linearly, a logarithmic rank logarithmically. The $O(1)$-time $o(n)$-bit target is therefore the operating assumption of every structure layered above bitvectors. Subsection 2.1.1 through Subsection 2.1.4 show how this target is met, why it is optimal in a precise sense, and what happens when the bitvector is sparse enough that even n bits are wasteful.

2.1.1 Constant-time rank

The first question to settle is whether the linear-time baseline can be improved to $O(1)$ without paying more than $o(n)$ extra bits. Jacobson showed this is possible [11], and the construction has remained the template for every subsequent rank structure. The idea is a two-level directory, combined with a universal lookup table on very short blocks.

Jacobson [11, 12] introduced *succinct* data structures and the construction below. He analyzed it as $O(\log n)$ bit-accesses; on the word RAM with $\omega = \Omega(\lg n)$, the same algorithm runs in $O(1)$ time, as recorded in [10, §4.7].

Theorem 2.1.1 (Constant-time rank [11]) *For any bitvector $B[1..n]$, there is a data structure of $n + O(n \lg\lg n / \lg n)$ bits that supports $\text{rank}_1(B, i)$ in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$. The bitvector B is stored verbatim and accessed read-only.*

The construction partitions B into two nested levels, illustrated in Figure 2.2. We choose a *superblock* size $Z = \lfloor \lg^2 n \rfloor$ and a *block* size $z = \lfloor (\lg n)/2 \rfloor$, so that each superblock contains exactly Z/z blocks. For simplicity we assume z divides Z and Z divides n; the general case differs only in the

treatment of the final partial superblock, which contributes $O(\lg n)$ bits of overhead.

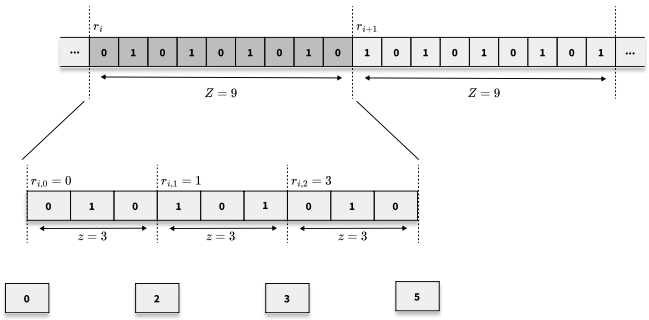

Figure 2.2: The two-level rank directory of Jacobson's construction. Superblocks of size Z store absolute ranks r_i; blocks of size z inside each superblock store ranks $r_{i,j}$ relative to the enclosing superblock. The bottom row represents the four-Russians lookup table, indexed by the z-bit intra-block pattern and an in-block offset, returning the rank inside the pattern.

Two auxiliary arrays store precomputed rank values. The *superblock directory* R_S holds absolute ranks at superblock boundaries. For $k = 0, 1, \ldots, n/Z - 1$ we set

$$R_S[k] \;=\; \mathrm{rank}_1(B, kZ).$$

The *block directory* R_B holds ranks relative to the enclosing superblock. For each block index $\ell = 0, 1, \ldots, n/z - 1$, belonging to superblock $k = \lfloor \ell z / Z \rfloor$, we set

$$R_B[\ell] \;=\; \mathrm{rank}_1(B, \ell z) - R_S[k].$$

A rank query at position i decomposes as the sum of three contributions: $R_S[\lfloor i/Z \rfloor]$, the block-relative rank $R_B[\lfloor i/z \rfloor]$, and the rank inside the final block from its start up to position i.

The third contribution, rank within a z-bit block, is handled by a single universal lookup table. Since $z \le \omega$, the block pattern $B[\ell z + 1..(\ell+1)z]$ fits in one machine word and is read directly from B. The table is indexed by the z-bit pattern p and the offset $j \in [1..z]$, returning the precomputed count $\mathrm{rank}_1(p, j)$; with $2^z \cdot z$ entries of $\lceil \lg(z+1) \rceil$ bits each, the total size is $O(2^z \cdot z \cdot \lg z)$ bits, which becomes $O(\sqrt{n}\ \lg n\ \lg\lg n) = o(n)$ once $z = \lfloor (\lg n)/2 \rfloor$ forces $2^z = \sqrt{n}$.

Sketch. The superblock directory R_S stores n/Z absolute ranks of $\lceil \lg n \rceil$ bits each, totalling $O(n/\lg n)$ bits. The block directory R_B stores n/z values in $[0, Z]$ at $O(\lg\lg n)$ bits each, totalling $O(n \lg\lg n / \lg n)$ bits. The lookup table has $2^z \cdot z = O(\sqrt{n} \lg n)$ entries of $O(\lg\lg n)$ bits, which is $o(n/\lg n)$. A query reads $R_S[\lfloor i/Z \rfloor]$, $R_B[\lfloor i/z \rfloor]$, the block pattern from B, and one table entry, performing $O(1)$ arithmetic. □

The four-Russians technique predates succinct data structures by almost two decades, named after a 1970 paper on Boolean matrix multiplication by Arlazarov, Dinic, Kronrod, and Faradzhev [13]. Jacobson's contribution was to apply the precomputed-table idea to rank.

The construction exposes where the $o(n)$ overhead comes from. The block directory R_B contributes the dominant term $\Theta(n \lg\lg n / \lg n)$, because each of $\Theta(n/\lg n)$ blocks carries a directory entry of $\Theta(\lg\lg n)$ bits. The superblock directory contributes only $O(n/\lg n)$ bits, smaller than R_B by a $\lg\lg n$ factor since superblock boundaries are sparser and each entry pays $\lg n$ bits while a block entry pays $\lg\lg n$. The lookup table is smaller still, contributing only $O(\sqrt{n}\,\mathrm{polylog}\, n)$ bits. The $\lg\lg n / \lg n$ overhead is a direct consequence of the block size $z = \Theta(\lg n)$. A larger z would shrink the block directory and blow up the lookup table, and a smaller z would do the opposite. Subsection 2.1.3 will establish that $\Theta(n \lg\lg n / \lg n)$ is essentially optimal within the class of indexing structures.

2.1.2 Constant-time select

Rank and select are mutual inverses, but the two-level directory of Theorem 2.1.1 does not support select directly. The source of the asymmetry is easy to see. The superblock and block directories record ranks at positions that are fixed multiples of Z and z. Given a rank query at position i, the superblock and block indices are $\lfloor i/Z \rfloor$ and $\lfloor i/z \rfloor$, computable from the query with constant-time arithmetic. Given a select query for the j-th one, the position is unknown, and the division that identified the right block for rank is unavailable. Jacobson's separate select structure stores explicit position markers at every $\lg^2 n$-th one and binary-searches the $\lg^2 n$-bit window between consecutive markers, giving $\Theta(\lg \lg n)$ query time [11]. Constant time by this route is unavailable.

Clark resolved this in his 1996 PhD thesis [14] by partitioning the bitvector according to the number of ones instead of the number of positions. The partition is no longer uniform in position; it is uniform in rank. A *chunk* contains a fixed number K of ones, so its boundaries lie at the positions $\text{select}_1(B, K)$, $\text{select}_1(B, 2K)$, and so on. Locating the j-th one at the top level reduces trivially to computing $\lceil j/K \rceil$. The cost is that chunks now have variable length, and a second idea is needed to handle queries within a chunk. Clark's solution is a three-level hierarchy with a sparse/dense dichotomy at each level.

Clark's 1996 PhD thesis at Waterloo [14] developed the three-level select hierarchy on top of Jacobson's rank machinery. The same construction remains standard in every practical library today.

[14]: Clark (1996), *Compact Pat Trees*

Theorem 2.1.2 (Constant-time select [14]) *For any bitvector $B[1..n]$, there is a data structure of $n + o(n)$ bits that supports* $\text{select}_1(B, j)$ *in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$. The bitvector B is stored verbatim and accessed read-only. A symmetric construction on the complemented bitvector $\overline{B}$ supports* select_0 *with the same asymptotic overhead, doubling the constant.*

The three-level partition is described here for select_1; select_0 is handled by replicating the construction on $\overline{B}$.Let $m = \text{rank}_1(B, n)$ be the number of ones in B, and set the top-level block size to $K = \lfloor \lg^2 n \rfloor$.

At the top level, we divide the positions of ones into groups of K. The chunk starts are recorded in an array P_1 of size $\lceil m/K \rceil$, with

$$P_1[c] \;=\; \text{select}_1(B, cK + 1) \qquad \text{for } c = 0, 1, \ldots, \lceil m/K \rceil - 1.$$

With $K = \lfloor \lg^2 n \rfloor$, a sparse chunk has length at least $K^2 = \lg^4 n$, while a dense chunk fits in a polylogarithmic window. The threshold balances explicit storage against the recursive layer applied within dense chunks.

Each entry of P_1 fits in $\lceil \lg n \rceil$ bits, so the total space for P_1 is $O\left(\frac{m}{K} \cdot \lg n\right) = O(n/\lg n) = o(n)$ bits, using $m \le n$. Given a query $\text{select}_1(B, j)$, the chunk index is $c = \lfloor (j-1)/K \rfloor$ and the local rank within the chunk is $j' = ((j-1) \bmod K) + 1$, both computed in $O(1)$ time.

A chunk covers positions $P_1[c]$ through $P_1[c+1] - 1$ of B, so its length $Z_c = P_1[c+1] - P_1[c]$ varies with c. The construction splits into two cases by density. A chunk is *sparse* when $Z_c \ge K^2 = \lg^4 n$, and *dense* otherwise. For a sparse chunk, the positions of its K ones are stored explicitly as an array of K offsets relative to $P_1[c]$, each fitting in $\lceil \lg Z_c \rceil = O(\lg n)$ bits. A sparse chunk has length at least K^2, so disjoint sparse chunks are substrings of B of total length at most n, giving at most n/K^2 of them. The total sparse-offset space is therefore

$$\frac{n}{K^2} \cdot K \cdot O(\lg n) \;=\; \frac{n \lg n}{K} \;=\; O\left(\frac{n}{\lg n}\right) \text{ bits},$$

which is $o(n)$. Within a dense chunk, explicit storage is too expensive, and we recurse.

Each dense chunk is sub-divided by number of ones, with sub-chunk size $k = \lceil (\lg \lg n)^2 \rceil$. Inside dense chunk c, we record the positions of the k-th, $2k$-th, ..., K-th ones, measured as offsets relative to $P_1[c]$; there are K/k such offsets. Each fits in $\lceil \lg Z_c \rceil = O(\lg \lg n)$ bits, since $Z_c < K^2 = \lg^4 n$ implies $\lg Z_c = O(\lg \lg n)$. With at most n/K dense chunks contributing $(K/k) \cdot O(\lg \lg n)$ bits each, the total dense-chunk recursive space is

$$\sum_{\text{dense } c} \frac{K}{k} \cdot O(\lg \lg n) \;=\; O\left(\frac{n}{K} \cdot \frac{K \lg \lg n}{k}\right) \;=\; O\left(\frac{n}{\lg \lg n}\right),$$

matching the bound of Clark [14] and Navarro [10, §4.3.3]. A sub-chunk spans k consecutive ones, so its length z_c is variable. The same sparse/-dense dichotomy applies at this level. A sub-chunk is sparse when $z_c \geq k^4 = (\lg \lg n)^8$ and dense otherwise. Sparse sub-chunks store the positions of their ones explicitly, giving $o(n)$ bits by the same analysis.

A dense sub-chunk has length $O((\lg \lg n)^8) = o(\lg n)$, fitting inside a $(\lg n)/2$-bit window; a precomputed table on all $(\lg n)/2$-bit patterns, of size $O(\sqrt{n}\, \lg n\, \lg \lg n) = o(n)$, resolves it.

Dense sub-chunks satisfy $z_c < k^4 = (\lg \lg n)^8 = o(\lg n)$, well inside a single $(\lg n)/2$-bit window of B. A precomputed table indexed by every $(\lg n)/2$-bit pattern and every local rank in $[1..(\lg n)/2]$ returns the position of the i''-th one within the pattern; with $2^{(\lg n)/2} = \sqrt{n}$ patterns of $(\lg n)/2$ entries at $O(\lg \lg n)$ bits each, the table occupies $O(\sqrt{n}\, \lg n\, \lg \lg n) = o(n)$ bits. This is the four-Russians technique of Jacobson [11] for rank, extended to select by Clark [14].

The three levels assemble into a constant-time query. First the chunk index c and local rank j' are computed, and $P_1[c]$ gives the chunk start. For a sparse chunk, the answer is $P_1[c]$ plus the j'-th stored offset. For a dense chunk, the sub-chunk index $\ell = \lfloor (j'-1)/k \rfloor$ and the local rank $j'' = ((j'-1) \bmod k) + 1$ give the sub-chunk start relative to $P_1[c]$ through the sub-chunk offset table. A sparse sub-chunk returns the precomputed offset of the j''-th one; a dense sub-chunk looks up the j''-th one in the dense-pattern table. Each level's branch choice is a single comparison, and the query reads $O(1)$ words.

Sketch. Each storage component is $o(n)$ bits. The top-level array P_1 costs $O(n/\lg n)$; the sparse-chunk offsets cost $O(n/\lg n)$; the dense-chunk recursive offsets and the bottom-level lookup table also contribute $o(n)$. The query performs $O(1)$ probes at each of the three levels. □

For very small dense sub-chunks, a direct broadword scan using hardware population count may outperform the lookup table on modern processors. The theoretical analysis does not rely on hardware popcount.

The same construction, applied separately to $\overline{B}$, handles select_0. Storing both structures doubles the auxiliary space while keeping it $o(n)$. Combined with the rank structure of Subsection 2.1.1, we obtain a bitvector representation that supports rank_b and select_b for $b \in \{0, 1\}$ in $O(1)$ time, with auxiliary space $o(n)$ bits on top of the verbatim bitvector.

2.1.3 Lower bounds

Jacobson and Clark achieve $n + o(n)$ bits for constant-time rank and select, with $\Theta(n \lg \lg n / \lg n)$ for rank. The remaining question is whether these bounds are tight: can the auxiliary index drop below $\Theta(n \lg \lg n / \lg n)$ while preserving $O(1)$ query time?

The *indexing* model, introduced by Miltersen [15], gives a sharp answer.

Miltersen's indexing model [15] stores the input verbatim and allows only an auxiliary index. The restriction is natural for bitvectors, because the n bits of B already meet the information-theoretic minimum for arbitrary bitvectors, and the only freedom is in how much additional structure the query algorithm may consult.

[15]: Miltersen (2005), *Lower Bounds on the Size of Selection and Rank Indexes*

Definition 2.1.3 (Indexing data structure) *An indexing data structure for a problem on inputs $x \in \{0,1\}^n$ consists of the input x stored verbatim in $\lceil n/\omega \rceil$ words, together with an r-bit index $\phi(x)$ stored in $\lceil r/\omega \rceil$ words.*

The query algorithm against such a storage scheme is an adaptive procedure that probes bits of x and $\phi(x)$ and returns an answer; it has query time t if it makes at most t bit-probes in the worst case.The indexing model constrains the representation to store the bitvector B untouched while allowing an extra index. This matches Theorem 2.1.1 and Theorem 2.1.2, where B is accessed read-only and the auxiliary structures R_S, R_B, P_1, and the lookup tables form the index. The redundancy r in this model is precisely the size of the index. Miltersen's 2005 paper gives the following lower bound.

Theorem 2.1.3 (Index size lower bound [15]) *For any indexing data structure supporting* select_1 *on a bitvector of length n in the cell-probe model with word size ω, with r bits of index and query time t, it holds that*

$$3(r+2)(t\omega+1) \;\geq\; n.$$

For rank_1*, it holds that*

$$2(2r+\lg(\omega+1))t\omega \;\geq\; n\lg(\omega+1).$$

In the regime $\omega = \Theta(\lg n)$ and $t = O(1)$, the select bound becomes $r = \Omega(n/\lg n)$ and the rank bound becomes $r = \Omega(n \lg\lg n/\lg n)$. The rank bound matches Theorem 2.1.1 up to a constant factor. The select bound falls short of Theorem 2.1.2 by a factor of $\lg\lg n$. Both are proved by counting arguments. We sketch the select argument, since the strategy is short and the technique recurs.

Sketch. Reduce to $\omega = 1$. Fix inputs $x \in \{0,1\}^n$ of Hamming weight $m = \lfloor (n/3-1)/t \rfloor$. Run the m select queries on x, collecting the mt probed bits into a string $\tau(x)$ after zeroing bits that belong to the index $\phi(x)$ or were already seen. Together, $\phi(x)$ and $\tau(x)$ suffice to reconstruct x by simulation. Counting pairs yields $2^r(mt+1)\binom{mt}{m} \geq \binom{n}{m}$. Since $m \leq n/3$, each ratio factor $(n-i)/(mt-i) \geq 2$, so $\binom{n}{m}/\binom{mt}{m} \geq 2^m$, forcing $r+1 \geq m$. The theorem statement follows from the $\omega = 1$ case by the standard bit-probe to cell-probe conversion. □

Golynski's 2006 argument [16] builds a *choices tree* whose leaves correspond to joint outcomes of probes across multiple queries, and bounds the number of bitvectors consistent with any leaf via a Stirling-based density estimate.

[16]: Golynski (2006), *Optimal Lower Bounds for Rank and Select Indexes*

Golynski [16] closed the $\lg\lg n$ gap between Miltersen's select bound and Theorem 2.1.2 with a matching lower bound that applies to both rank and select.

Theorem 2.1.4 (Tight select index bound [16]) *Any indexing data structure for* rank_1 *or* select_1 *on a bitvector of length n, with $O(\lg n)$ bit-probes per query to the bitvector and unlimited probes to the index, has index size $r = \Omega(n \lg\lg n/\lg n)$.*

Once the index is fixed, each query becomes a deterministic probe sequence, so different inputs trace different paths from root to leaf of the choices tree. Counting leaves and bounding the number of bitvectors compatible with each leaf yields the lower bound. The argument fixes

the index and considers all possible probe outcomes across p queries on blocks of size $k = t + \lg n$. The probes partition each block's bits into *probed* and *unprobed* bits, and the cardinality of each block is computable from the probe answers. Golynski classifies blocks as *determined* (more than half of the bits probed) or *undetermined*, shows that a constant fraction must be undetermined, and for rank bounds the number of bitvectors compatible with a leaf by

$$\frac{C(x)}{2^{U(x)}} \;\le\; \left(\frac{c}{\sqrt{\lg n}}\right)^{ap}$$

for some constants $a, c > 0$, where $U(x)$ is the number of unprobed bits. The same technique extends to select with the same bound. Summing over all 2^{n+r} leaves, and requiring that every bitvector of length n be compatible with at least one leaf, forces $r = \Omega(n \lg\lg n / \lg n)$. The Stirling calculation is in Golynski's technical report [16].

The $(m/t)\lg t$ bound is density-sensitive. For very sparse bitvectors, where $m \ll n$, the overhead can be reduced at the cost of increasing the query time. For dense bitvectors, the bound degenerates into the $\Omega(n \lg\lg n / \lg n)$ result.

Golynski's paper also gives a density-sensitive lower bound. If the bitvector has m ones and the query algorithm uses t bit-probes to the bitvector, the index must have size at least $(m/t)\lg t$ bits. For $m = \Theta(n)$ and $t = O(\lg n)$, this recovers $\Omega(n \lg\lg n / \lg n)$, and for sparser bitvectors it points toward compressed representations studied in Subsection 2.1.4.

[17]: Beame et al. (2002), *Optimal Bounds for the Predecessor Problem and Related Problems*

The select lower bounds above propagate to predecessor queries on bitvector-encoded sets, via Pătraşcu's observation that predecessor reduces to $\text{select}_1 \circ \text{rank}_1$. Beame and Fich [17] established tight cell-probe bounds for the predecessor problem itself, so the indexing-model bounds of Theorem 2.1.3 and Theorem 2.1.4 together with Beame-Fich constrain predecessor-supporting structures built from rank and select.

Pătraşcu's 2008 FOCS paper *Succincter* [18] broke the redundancy barrier by combining *spill-over representations*, which pass fractional bits of entropy between layers, with *aB-trees*, the augmented-tree abstraction introduced in the same paper. The construction yields redundancy $O(u/\text{polylog}\, u)$ at the cost of only a constant factor in query time.

[18]: Pătraşcu (2008), *Succincter*

[19]: Grossi et al. (2009), *More Haste, Less Waste: Lowering the Redundancy in Fully Indexable Dictionaries*

The lower bound of Theorem 2.1.4 applies to indexing data structures, where B itself is stored untouched. Pătraşcu [18] showed that this restriction is essential. If the bitvector is re-encoded, the redundancy can be pushed far below $\Theta(n \lg\lg n / \lg n)$, at the cost of an exponential trade-off with query time. A related line of work by Grossi, Orlandi, Raman, and Srinivasa Rao [19] lowers the redundancy of fully indexable dictionaries through a parametric family of representations refining Theorem 2.1.5.

Theorem 2.1.5 (Succincter [18]) *On a word RAM with cells of $\Omega(\lg u)$ bits, a bitvector of length u with n ones can be stored in*

$$\lg \binom{u}{n} \;+\; \frac{u}{((\lg u)/t)^t} \;+\; \widetilde{O}(u^{3/4}) \quad \text{bits,}$$

supporting rank_1 *and* select_1 *queries in $O(t)$ time, for any integer $t \ge 1$.*

Setting $t = 2$ in Theorem 2.1.5 gives redundancy $O(u/(\lg u)^2)$, strictly smaller than the Jacobson-Clark bound. Setting $t = \lg\lg u$ gives redundancy $O(u/(\lg u)^{\lg\lg u}) = u/2^{\Omega((\lg\lg u)^2)}$, superpolylogarithmically smaller. The lower bound of Theorem 2.1.4 is compatible, because Pătraşcu's construction departs from the indexing model and re-encodes the data through *spill-over representations*, which let fractional bits of entropy flow between levels.

The core idea is the *spill-over representation*. A value from a set X is stored as M memory bits together with an auxiliary *spill* value in $\{0, 1, \ldots, K-1\}$,

where $K \cdot 2^M \geq |X|$, in place of the $\lceil \lg |X| \rceil$ memory bits of a plain encoding. Pătraşcu [18] chooses parameters so that $r \leq K \leq 2r$ for a target r, and shows the redundancy of a single level is at most $2/r$ bits. Stacking these representations in t levels of an aB-tree yields redundancy $u/((\lg u)/t)^t$, exponential in t rather than linear. The query time is $O(t)$, because navigating the aB-tree touches t levels. Theorem 2.1.5 shows that the Jacobson-Clark bound is optimal only within the indexing model, and that dropping the verbatim-storage restriction yields exponential savings. The aB-tree template implicitly present in Jacobson's and Clark's constructions can be extracted, abstracted, and composed recursively; Pătraşcu's generic transformation from augmented search trees to succinct data structures applies to rank and select as one instance among several.

A spill-over representation stores M memory bits plus a spill value in $\{0, \ldots, K-1\}$. The spill carries fractional bits of entropy that would otherwise be wasted. Pătraşcu's key step is recursive composition of two spill-over representations, giving exponential redundancy reduction where the naive composition would give only a linear trade-off.

Practical implementations still overwhelmingly use the Jacobson-Clark scheme, because the indexing structure admits very fast word-level operations and simple code, while spill-over representations require more elaborate arithmetic.

2.1.4 Compressed bitvectors

Every structure so far pays n bits for the bitvector of length n, regardless of how many ones it contains. For bitvectors with few ones, this is wasteful. A set of n elements drawn from a universe of size u is one of $\binom{u}{n}$ possibilities, so the information-theoretic cost of encoding it is

$$\mathcal{B}(u, n) \;=\; \left\lceil \lg \binom{u}{n} \right\rceil \;=\; n \lg(u/n) + O(n)$$

bits for $n \leq u/2$,which is much smaller than u when $n \ll u$. Can we represent the bitvector in $\mathcal{B}(u, n) + o(u)$ bits while still supporting $O(1)$ rank and select? A sequence of papers starting with Raman and Rao [20] and culminating in Raman, Raman, and Rao [21] answers yes.

[20]: Raman et al. (1999), *Static Dictionaries Supporting Rank*

[21]: Raman et al. (2007), *Succinct indexable dictionaries with applications to encoding k-ary trees, prefix sums and multisets*

The 1999 ISAAC paper of Raman and Rao [20] established the template. Their structure combines the classical FKS perfect-hashing dictionary [22] with a *quotient* representation to achieve the following.

Raman and Rao's 1999 ISAAC paper [20] was the first to combine the classical FKS dictionary [22] with quotient information to break the $n\lceil \lg u \rceil$-bit barrier while keeping $O(1)$ rank. Its per-bucket rank trick foreshadows the full RRR construction.

[20]: Raman et al. (1999), *Static Dictionaries Supporting Rank*

[22]: Fredman et al. (1984), *Storing a Sparse Table with $O(1)$ Worst Case Access Time*

Theorem 2.1.6 (Quotient dictionary with $O(1)$ rank [20]) *A subset $S \subseteq [1..u]$ of size n can be stored in $n \lg u + O(\lg \lg u) - \Theta(n)$ bits so that membership and rank queries on elements of S are answered in $O(1)$ time.*

The bound in Theorem 2.1.6 consolidates the main theorem and a corollary of Raman and Rao [20]; the corollary refines the space by storing only the low-order bits of each rank within blocks of size $n/2^c$, with block-start pointers stored separately. The construction stores each element of S through a composite FKS hash, keeping only a *quotient* that distinguishes it from other elements mapped to the same bucket. Combined with k^{-1} and κ^{-1} inverses of the hash parameters, the quotient together with the bucket index and compressed tables recovers the original element in constant time. Rank on elements of S is supported in $O(1)$ time by storing each rank value alongside its element, and the corollary's block-wise refinement saves the $\Theta(n)$ term in the space bound. The space exceeds the information-theoretic minimum $\mathcal{B}(u, n) = n \lg(u/n) + O(n)$ by $\Theta(n \lg n)$ bits because FKS hashing keeps a copy of each element. The bound is the

first to break the $n\lceil \lg u \rceil$-bit barrier while keeping $O(1)$ rank on members; Raman and Rao leave open whether the $-\Theta(n)$ slop in their own bound can be tightened to $+o(n)$, a question settled by the RRR construction below.

[21]: Raman et al. (2007), *Succinct indexable dictionaries with applications to encoding k-ary trees, prefix sums and multisets*

The question was settled by the 2002 SODA paper and 2007 TALG journal version of Raman, Raman, and Rao [21]. Their construction, known as the RRR structure, achieves space within a lower-order term of the information-theoretic minimum while preserving $O(1)$ rank and select. We describe the class/offset encoding, state the main theorem, and trace the query algorithm.

The bitvector $B[1..n]$ is partitioned into $\lceil n/b \rceil$ blocks of length $b = \lfloor (\lg n)/2 \rfloor$, matching the block size used in Jacobson's rank structure of Subsection 2.1.1 to balance the block directory against the lookup table. Each block is replaced by a (class, offset) pair in place of its verbatim content.For each block $B_j = B[(j-1)b + 1..jb]$, define its *class* c_j as the population count,

$$c_j \;=\; \sum_{i=(j-1)b+1}^{jb} B[i] \;\in\; \{0, 1, \ldots, b\},$$

and its *offset* o_j as the rank of the block's pattern within the lexicographic enumeration of the $\binom{b}{c_j}$ b-bit patterns of population count c_j. The pair (c_j, o_j) uniquely determines B_j, and inversely B_j determines (c_j, o_j). The class fits in $\lceil \lg(b+1) \rceil$ bits; the offset fits in $\lceil \lg \binom{b}{c_j} \rceil$ bits, which depends on the class.

Theorem 2.1.7 (Indexable dictionary (RRR) [21]) *A bitvector $B[1..n]$ with m ones can be stored in*

$$\mathcal{B}(n, m) + o(n) + O(\lg \lg n) \quad \textit{bits},$$

where $\mathcal{B}(n, m) = \lceil \lg \binom{n}{m} \rceil$, so that $\mathrm{rank}_b(B, i)$ and $\mathrm{select}_b(B, i)$ for $b \in \{0, 1\}$ are answered in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$.

Sketch. Partition B into $p = \lceil n/b \rceil$ blocks of length $b = \lfloor (\lg n)/2 \rfloor$, encode each block by its (c_j, o_j) pair, and store the class array C plus the variable-length offset stream. For the $p-1$ full blocks of length b and the boundary block of length at most b, the Vandermonde-type identity controlled by $pb \geq n$ bounds the offset stream by $\mathcal{B}(n, m) + O(n/\lg n)$ bits.Variable-length offsets are accessed through the searchable prefix-sum structure of Theorem 2.1.6 applied to the bit-length array; this costs $o(n)$ bits.Lookup tables for rank and select within a block contribute $o(n)$ bits. Rank assembles a superblock prefix, the class-sequence prefix sum retrieved via the searchable prefix-sum structure, and the within-block lookup.Select uses a segment structure partitioning the universe into segments each containing $v = \lfloor (\lg p)^2 \rfloor$ ones, with a dense/sparse dichotomy at the segment level.An additional MSB-bucketing layer in [21] reduces the lower-order term, grouping keys by their most-significant bits into $\Theta(m\sqrt{\lg n})$ buckets and representing bucket boundaries compactly;the final bound is $\mathcal{B}(n, m) + o(n) + O(\lg \lg n)$ bits with $O(1)$ rank and select. □

The quantity $\mathcal{B}(n, m)$ connects directly to Definition 1.2.3.

2.2 Sparse Bitvectors

Every structure of Section 2.1 pays at least n bits for a bitvector of length n, regardless of how many ones it contains. When the number of ones m is much smaller than n, this is wasteful. A set of m elements drawn from a universe of size n is one of $\binom{n}{m}$ possibilities, so the information-theoretic minimum is

$$\mathcal{B}(n, m) \;=\; \lceil \lg \tbinom{n}{m} \rceil \;=\; m \lg(n/m) + O(m) \quad \text{bits},$$

a quantity much smaller than n when $m \ll n$. Theorem 2.1.7 already reaches $\mathcal{B}(n, m) + o(n) + O(\lg \lg n)$ bits with constant-time rank and select, through class/offset encoding over blocks of length $(\lg n)/2$. When the input is sparse, a structurally simpler representation stores the positions of the m ones directly, in the form of a monotone integer sequence, and exploits the monotonicity to split each position into a cheap high part and an explicit low part. The simpler mechanics trade a slower rank or access for a faster select, and for very sparse inputs yield smaller constants and simpler code than RRR.

Sparse representations and RRR meet at the same information-theoretic bound, but they distribute the cost differently across operations, and the regime $m \ll n$ determines which is preferable in practice.

Elias introduced a sparse integer-set representation [23], working in parallel with an independent unpublished MIT memorandum of Fano [24]. The construction now carries both names. Neither original formulation gives a word-RAM analysis; we present below the construction of Grossi and Vitter [25].

2.2.1 Elias-Fano

Let $B[1..n]$ be a bitvector with exactly m ones, and let $x_1 < x_2 < \cdots < x_m$ be the positions of its ones, so each $x_h \in [1..n]$. The sequence $(x_1, \ldots, x_m)$ is a monotone sequence of integers in a universe of size n, and the bitvector and the sequence are equivalent representations of the same subset. RRR compresses B at the block level, without acknowledging monotonicity. Elias-Fano exploits monotonicity directly. Split each x_h into a low part and a high part,

$$x_h \;=\; q_h \cdot 2^{w-z} + r_h,$$

where $z = \lfloor \lg m \rfloor$, $w = \lceil \lg n \rceil$, the high part q_h occupies the top z bits of x_h, and the low part r_h occupies the remaining $w - z$ bits. The low parts $r_1, r_2, \ldots, r_m$ are stored verbatim as a fixed-width array L, one $(w - z)$-bit field per element. The high parts are stored in a bitvector H obtained by unary-coding the differences of consecutive quotients,

$$H \;=\; 0^{q_1}\, 1\, 0^{q_2 - q_1}\, 1 \cdots 0^{q_m - q_{m-1}}\, 1.$$

The ones in H mark the m quotients, and the zeros interpolate the gaps between successive quotient values. Since $q_h \in [0, 2^z)$ and $2^z \le m$, there are at most m zeros and exactly m ones, so $|H| \le 2m$.

Figure 2.3 illustrates the encoding on a concrete example. The positions $5, 7, 8, 15, 32$ form a strictly increasing sequence in $[0, 36)$, so $n = 36$, $m = 5$, $w = 6$, and $z = \lfloor \lg 5 \rfloor = 2$. The low parts are the bottom $w - z = 4$

bits of each position and form the array L; the top $z = 2$ bits of each position form the quotient sequence $(0, 0, 0, 0, 2)$, unary-coded into H.

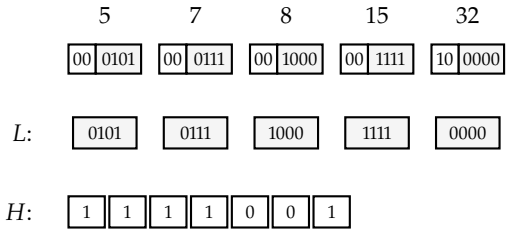

Figure 2.3: Elias-Fano encoding of the set $\{5, 7, 8, 15, 32\}$ with universe $n = 36$ and $m = 5$, so $w = 6$ and $z = \lfloor \lg 5 \rfloor = 2$. Each element is split into a high part of z bits and a low part of $w - z = 4$ bits. The array L is the concatenation of the low parts in order; the bitvector H unary-codes the successive increments of the high parts $(0, 0, 0, 0, 2)$. To answer $\text{select}_1(j)$, locate the j-th one in H at position $p = \text{select}_1(H, j)$, recover the high part $q_j = p - j$, and concatenate it with the low part $L[j]$.

[25]: Grossi et al. (2005), *Compressed Suffix Arrays and Suffix Trees with Applications to Text Indexing and String Matching*

Grossi and Vitter [25] give the first rigorous word-RAM analysis.

Theorem 2.2.1 (Elias-Fano encoding [25]) *Given m integers in sorted order, each containing w bits, with $m < 2^w$, there is a representation of*

$$m\left(2 + w - \lfloor \lg m \rfloor\right) + O\big(m/\lg \lg m\big) \quad \textit{bits}$$

that retrieves the h-th integer in $O(1)$ time on the word RAM with word size $\omega = \Omega(w)$. For the characteristic bitvector of a subset $S \subseteq [1..n]$ of size m, with $w = \lceil \lg n \rceil$, the bound is $m(\lceil \lg n \rceil - \lfloor \lg m \rfloor) + 2m + O(m/\lg \lg m) \leq m\lceil \lg(n/m) \rceil + 3m + O(m/\lg \lg m)$ bits, and the h-th retrieval answers $\text{select}_1(B, h)$.

[25]: Grossi et al. (2005), *Compressed Suffix Arrays and Suffix Trees with Applications to Text Indexing and String Matching*

Proof. We follow the construction of Grossi and Vitter [25]. Take $z = \lfloor \lg m \rfloor$ and split each x_h into the quotient $q_h \in [0, 2^z)$ and the remainder $r_h \in [0, 2^{w-z})$, as above. Encode L as the concatenation $r_1 r_2 \cdots r_m$ of fixed-width $(w - z)$-bit fields, totalling $|L| = m(w - z)$ bits. Encode H as the unary concatenation of the differences $q_1, q_2 - q_1, \ldots, q_m - q_{m-1}$, which has m ones (one per integer) and at most $2^z - 1 \leq m$ zeros (one per occupied cell of the quotient universe), so $|H| \leq 2m$ bits.

The two arrays together occupy $|L| + |H| \leq m(w - z) + 2m = m(2 + w - \lfloor \lg m \rfloor)$ bits. To support select_1 on H in constant time, we augment H with a select structure of $O(|H|/\lg \lg |H|) = O(m/\lg \lg m)$ bits, an overhead strictly smaller than the $O(|H| \lg \lg |H|/\lg |H|)$ budget of Theorem 2.1.2. The total space is $m(2 + w - \lfloor \lg m \rfloor) + O(m/\lg \lg m)$ bits.

To recover x_h we must compute its quotient q_h and its remainder r_h in constant time. The quotient is stored implicitly in H through the position of its h-th one. Indeed, the h-th one of H lies at position $q_h + h$, because the unary code for the differences places the h-th one after exactly q_h zeros cumulatively. Therefore

$$q_h = \text{select}_1(H, h) - h,$$

computable in $O(1)$ time by the auxiliary select structure. The remainder is read directly from L through the fixed-width access $r_h = L[h]$. The answer is $x_h = q_h \cdot 2^{w-z} + r_h$, a constant-time composition of two word-level arithmetic operations. □

The space bound of Theorem 2.2.1 consolidates the two intuitions behind Elias-Fano. The remainder array L costs $m(w - z)$ bits, within an additive

m-bit slack of $m\lceil \lg(n/m) \rceil$. The high-part bitvector H costs at most $2m$ bits, the classical constant-overhead observation of Elias and Fano; a monotone sequence of m integers in $[0, 2^z)$ with $2^z \leq m$ is cheaply encoded in unary, because the combined zeros-and-ones budget is bounded by the count of ones plus the span of the quotient range. The leading term $m \lg(n/m) + O(m)$ approaches $\mathcal{B}(n, m) = \lg \binom{n}{m}$ within a linear additive term, since $\lg \binom{n}{m} = m \lg(n/m) + m \lg e + O(m^2/n)$ for $m \ll n$.

The retrieval algorithm in the proof of Theorem 2.2.1 answers select_1, namely the position of the h-th one of B. A single word-RAM call to Theorem 2.1.2, applied to H, reduces the query on B to an $O(1)$ composition with a fixed-width lookup in L. Rank is not symmetric. Given a position $i \in [1..n]$, computing $\text{rank}_1(B, i)$ requires identifying the largest h such that $x_h \leq i$, a binary search over the sorted sequence. The standard implementation binary-searches $\text{select}_1(B, \cdot)$, trading $O(\lg m)$ time for $O(1)$ space overhead. Access reduces to rank by $\text{access}(B, i) = [\text{rank}_1(B, i) > \text{rank}_1(B, i-1)]$, inheriting the same $O(\lg m)$ cost. This is the operational asymmetry that distinguishes Elias-Fano from Theorem 2.1.7. The latter buys $O(1)$ rank and select simultaneously, at the cost of a two-level segment structure and variable-length offsets. Elias-Fano buys $O(1)$ select at the cost of logarithmic rank and access.

The argument that at most m bits flip in H between any consecutive pair of integers is the same counting principle behind gap encoding in inverted indexes [26]. Elias-Fano commits the low bits to a fixed-width format and the high bits to unary, which always costs $2m$ bits regardless of gap distribution, decoupling compression from statistical assumptions about gap skew.

2.2.2 Rank and access

Theorem 2.2.1 supports select_1 in $O(1)$ time but leaves rank_1 and access at $\Theta(\lg m)$, the cost of a binary search over $\text{select}_1(B, \cdot)$. Okanohara and Sadakane [27] lower the cost of rank and access at the same asymptotic space, by coupling Theorem 2.2.1 with a second-layer select structure on the high-part bitvector H. The bit profile of H, with m ones and at most m zeros (total length $\leq 2m$), admits efficient select_0.

[27]: Okanohara et al. (2007), *Practical entropy-compressed rank/select dictionary*

Theorem 2.2.2 (sdarray [27]) *A bitvector $B[1..n]$ with m ones can be represented in*

$$m\lceil \lg(n/m) \rceil + 2m + o(m) \quad \textit{bits},$$

supporting $\text{select}_1(B, j)$ *in* $O(\lg^4 m / \lg n)$ *time and* $\text{rank}_1(B, i)$ *and* $\text{access}(B, i)$ *in* $O(\lg(n/m) + \lg^4 m / \lg n)$ *time, on the word RAM with word size* $\omega = \Omega(\lg n)$.

The representation is a direct extension of Theorem 2.2.1. The remainder array L and the high-part bitvector H are stored as before, at a combined cost of $m\lceil \lg(n/m) \rceil + 2m$ bits. select_1 on B follows the same identity $\text{select}_1(B, j) = (\text{select}_1(H, j) - j) \cdot 2^{w-z} + L[j]$ as in Elias-Fano, reducing the query to one select on H and one fixed-width read on L. The select on H is now answered by darray, which contributes the $O(\lg^4 m / \lg n)$ overhead to the overall time. The novel component is select_0 on H, needed to support rank and access on B.

Vigna's 2013 paper [28] engineers Elias-Fano for large-scale inverted indexes, with the resulting format adopted by MG4J from version 5.0. Two additions to the base structure matter in practice. Forward pointers track the position reached after kq unary reads, giving average constant-time sequential access. Skip pointers installed on the negated unary code support *skipping*, the operation of finding the smallest $x_i \geq b$ for a given bound b, in average constant time under a linear lower bound on the number of zeros.

The sarray structure uses darray, the dense-set counterpart of Okanohara and Sadakane, as a subroutine on the high-part bitvector H. darray is stated in [27] for a bitvector of length N with parameters $L = O(\lg^2 N)$, $L_2 = O(\lg^4 N)$, $L_3 = O(\lg N)$; applied to H of length at most $2m$, the parameters become $L = O(\lg^2 m)$, $L_2 = O(\lg^4 m)$, $L_3 = O(\lg m)$. Partition the m zeros of H into blocks of L zeros each. A block whose length exceeds

L_2 stores the positions of its zeros explicitly; a shorter block records only every L_3-th position and resolves the remainder by sequential scan. The total auxiliary space is $o(m)$ bits, and select_0 on H runs in $O(\lg^4 m/\lg n)$ time in the worst case, the bound that Okanohara and Sadakane treat as constant in the practical word-RAM regime.

rank_1 and access on B then reduce to select_0 on H together with a local search on L. Given a query position $i \in [1..n]$, let $q = \lfloor i/2^{w-z} \rfloor$ and $r = i \bmod 2^{w-z}$. The number of ones of B with quotient strictly less than q equals the number of ones of H strictly before its q-th zero, retrievable as $\text{select}_0(H, q) - q$. The remaining ones, those with quotient exactly q and remainder in $[0, r]$, form a contiguous range of L of at most $2^{w-z} \le n/m$ elements, resolved by binary search on the $\lg(n/m)$-bit remainders in time $O(\lg(n/m))$. The total cost of rank is $O(\lg(n/m) + \lg^4 m/\lg n)$, and access follows the same route.

Theorem 2.2.2 and Theorem 2.1.7 meet at the same information-theoretic bound $\mathcal{B}(n, m) = \lg \binom{n}{m}$, and distribute the cost differently across operations. Theorem 2.1.7 achieves $\mathcal{B}(n, m) + o(n) + O(\lg \lg n)$ bits with $O(1)$ rank and select for both values, a uniform guarantee across all densities. Theorem 2.2.2 achieves $m\lceil \lg(n/m) \rceil + 2m + o(m)$ bits with select_1 in $O(\lg^4 m/\lg n)$ and rank_1 and access in $O(\lg(n/m) + \lg^4 m/\lg n)$, where the $\lg^4 m/\lg n$ term is the worst-case overhead from select_0 on the high-part bitvector, treated as constant in the practical word-RAM regime by the original analysis.

The difference lies in the layout. RRR carries the apparatus of class/offset encoding over every block of the bitvector regardless of density, with variable-length offsets and a searchable prefix-sum structure. sdarray avoids this apparatus by committing to a single global split of each position into high and low parts, and recovers the operations from two select structures on H. When $m \ll n$, the constants hidden in sdarray's $o(m)$ are small, the leading term $m\lceil \lg(n/m) \rceil + 2m$ is within $O(m)$ bits of the RRR bound, and the $\lg(n/m)$ factor in rank is small. Neither representation dominates the other. The density m/n and the operation mix determine which is preferable.

Each gap $g = s_i - s_{i-1}$ needs $1 + \lfloor \lg g \rfloor$ bits of binary representation. When the gap distribution concentrates on small values, $\text{gap}(\hat{S})$ falls well below the $2m$ bits spent by Elias-Fano on the unary H, and the Sadakane-Grossi bound beats Elias-Fano by more than a constant factor.

[29]: Sadakane et al. (2006), *Squeezing Succinct Data Structures into Entropy Bounds*

The Elias-Fano $2m$-bit overhead on H is tight against the gap distribution only up to a constant. When the gaps between consecutive ones are highly clustered, H contains large runs of zeros that encode redundant information, and the gap distribution itself admits coding well below $2m$ bits. Sadakane and Grossi [29] closed this gap with *entropy-bound indexable dictionaries*, a construction that achieves the following bound.

$$\min\{nH_k,\ \text{gap}(\hat{S})\} \ +\ O(n \lg \lg n/\lg n) \quad \text{bits},$$

Here H_k is the k-th order empirical entropy of B, and $\text{gap}(\hat{S}) = \sum_{i=1}^{m}(1 + \lfloor \lg(s_i - s_{i-1}) \rfloor)$ sums the minimum binary lengths of the gaps between consecutive ones, with $s_0 = 0$.

Like Theorem 2.1.5, the Sadakane-Grossi construction steps outside the indexing model of Subsection 2.1.3 and avoids the lower bounds of Theorem 2.1.3 and Theorem 2.1.4.

The construction is structurally different from sdarray. Sadakane and Grossi replace the verbatim bitvector B with a compressed representation from which any $O(\lg n)$-bit substring decodes in constant time, and build rank/select on top of the compressed form. The nH_k argument of the min is tight on dense inputs with symbol-level regularity; the $\text{gap}(\hat{S})$ argument is tight on sparse inputs with clustered gaps. The min adapts

to whichever term is smaller on the given input, without committing at encoding time.

The bound is never worse than the RRR or sdarray space, and on clustered inputs it is strictly smaller by a large factor; rank and select remain $O(1)$ under the word-RAM assumption. The construction carries a more involved encoding and decoding pipeline than sdarray, so RRR and sdarray still dominate in practice. The result confirms that neither $m\lceil \lg(n/m)\rceil$ nor $\mathcal{B}(n, m)$ is the last word on the sparse-bitvector problem.

The quantity $\mathcal{B}(n, m) = \lg \binom{n}{m}$, in the single-set notation of this chapter, is the independent-encoding cost H_{wc} of Definition 1.2.3 evaluated on a single set. Returning to the collection notation of Definition 1.2.1, with u for the universe size, n for the total size $\sum_i |S_i|$, and m for the number of sets, applying Theorem 2.2.2 (or Theorem 2.1.7) to each characteristic bitvector of $\mathcal{S} = \{S_1, \ldots, S_m\}$ yields a representation of total space $H_{wc}(\mathcal{S}) + O(n + m \lg n)$ bits, the set-by-set baseline. The leap from one set to many, where inter-set structure can drive the cost below H_{wc}, is the problem of Chapter 5.

2.3 Wavelet trees

Rank and select on a bitvector solve a problem with only two possible symbols. A sequence $S[1..n]$ over an alphabet $\Sigma = [1..\sigma]$ requires the same kind of positional navigation, but each position now carries one of σ possible values. Storing every symbol in a fixed $\lceil \lg \sigma \rceil$-bit field costs $n\lceil \lg \sigma \rceil$ bits, yet the fields do not by themselves count how many copies of a symbol occur in a prefix. The other direct reduction, one characteristic bitvector B_c for each symbol $c \in \Sigma$, gives rank and select immediately but repeats the length-n universe σ times, for $n\sigma$ bits. The binary viewpoint can still do the work without materializing one bitvector for every symbol. A single bit asks whether the current symbol lies in the left half or in the right half of the alphabet. Rank on that bitvector tells how many symbols of a prefix survive inside the chosen half, so it gives the prefix length for the next restricted sequence. Repeating the same question on the chosen half identifies the symbol after $\lceil \lg \sigma \rceil$ binary decisions, while preserving the position remapping needed by rank and select. Grossi, Gupta, and Vitter's wavelet tree [31] stores this hierarchy of binary decisions.

Chazelle's functional approach to multidimensional searching [30] is an older range-search ancestor. Grossi, Gupta, and Vitter [31] made the wavelet tree a sequence data structure over finite alphabets, with rank and select on the induced bitmaps.

[31]: Grossi et al. (2003), *High-order entropy-compressed text indexes*

Definition 2.3.1 (Sequence operations) *Let $S[1..n]$ be a sequence over the alphabet $[1..\sigma]$. For $c \in [1..\sigma]$, $1 \le i \le n$, and $j \ge 1$, we define three operations on S:*

- access(S, i)*: returns the symbol $S[i]$.*
- rank$_c(S, i)$*: returns $|\{k : 1 \le k \le i,\ S[k] = c\}|$, the number of occurrences of c in the prefix $S[1..i]$, with* rank$_c(S, 0) = 0$ *by convention.*
- select$_c(S, j)$*: returns the unique position k such that $S[k] = c$ and* rank$_c(S, k) = j$*, or $\perp$ if fewer than j occurrences of c appear in S.*

For $\sigma = 2$, these operations reduce to access, rank$_b$, and select$_b$ on the characteristic bitvector, recovering Definition 2.1.1 and Definition 2.1.2.

2.3.1 Structure and navigation

Begin with the whole alphabet interval $[1..\sigma]$ and the whole sequence S. At any stage, suppose the current interval is $[a_v, b_v]$ and the symbols

outside it have been filtered out on this branch. The remaining symbols form an implicit subsequence S_v, kept in their original order. If $[a_v, b_v]$ contains more than one symbol, the balanced version asks the next binary question by splitting the interval at its midpoint. Let $m_v = \lfloor (a_v + b_v)/2 \rfloor$. For each position of S_v, store whether the symbol lies in the left half $[a_v, m_v]$ or in the right half $[m_v + 1, b_v]$. This gives a bitmap $B_v[1..|S_v|]$ with

$$B_v[i] = \begin{cases} 0 & \text{if the } i\text{-th symbol of } S_v \text{ lies in } [a_v, m_v], \\ 1 & \text{if it lies in } [m_v + 1, b_v]. \end{cases}$$

The zeros of B_v form the subsequence $S_0 = S_v\langle B_v = 0\rangle$ on which the same construction continues over $[a_v, m_v]$. The ones form $S_1 = S_v\langle B_v = 1\rangle$ over $[m_v + 1, b_v]$. The process stops when the interval is a singleton $[c, c]$. At that point no bitmap is needed, because every remaining position contains the same symbol c, and the label is already determined by the previous binary choices. The recursive construction is a balanced binary tree of height $\lceil \lg \sigma \rceil$: the root has $S_{\text{root}} = S$ and covers $[1..\sigma]$, while each leaf corresponds to one symbol, as in Figure 2.4.

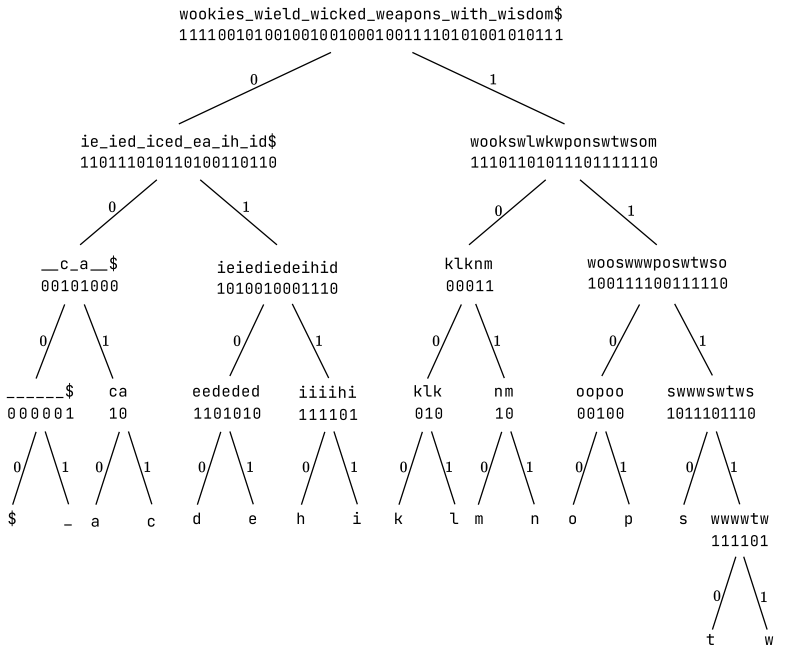

Figure 2.4: Wavelet tree for the string S = `wookies_wield_wicked_weapons_with_wisdom$` over an alphabet of 17 symbols. Each internal node carries a sub-alphabet interval. Its bitmap encodes, for every symbol of the implicit subsequence, which side of the midpoint split it falls on. The subsequences are drawn beside each node for illustration. The structure stores only the topology and the bitmaps.

At every level ℓ of the tree, the node subsequences $\{S_v\}_{v \text{ at level } \ell}$ partition the positions of S, because each symbol of S belongs to exactly one alphabet interval at every level. Therefore $\sum_{v \text{ at level } \ell} |B_v| \le n$, with equality as long as no node at level ℓ is a leaf. Deeper levels may store strictly fewer than n bits once some subtrees have already collapsed onto single-symbol leaves. Summing over the $\lceil \lg \sigma \rceil$ levels, the total length of the bitmaps is at most $n\lceil \lg \sigma \rceil$ bits. Attaching to each bitmap the constant-time rank and select auxiliaries of Theorem 2.1.1 and Theorem 2.1.2 costs $o(|B_v|)$ bits per bitmap. Summed level by level the overhead is $o(n) \lg \sigma$. The tree topology, encoded with explicit child pointers, takes $O(\sigma \lg n)$ additional bits, since the tree has σ leaves and $\sigma - 1$ internal nodes each storing pointers and bitmap offsets of $O(\lg n)$ bits. The total space of the pointered wavelet tree is therefore $n\lceil \lg \sigma \rceil + o(n) \lg \sigma + O(\sigma \lg n)$ bits. The tree is built in $O(n \lg \sigma)$ time by linear-time processing at each internal node, where the node reads its subsequence S_v, writes its bitmap

B_v, and partitions S_v into the two children's subsequences. Since the subsequences at any level partition the positions of S, the work at each level sums to $O(n)$, and summing over the $\lceil \lg \sigma \rceil$ levels yields the claimed bound.

Once every bitmap supports constant-time binary rank and select, the hierarchy turns the three sequence operations into root-to-leaf or leaf-to-root position remappings. Each level contributes one binary query and one child choice, so the cost is $O(\lg \sigma)$. Access, rank, and select differ only in where the traversal starts and in how the next edge is chosen.

Access reads the i-th symbol of S by a top-down traversal starting at the root. At the current node v, consult $B_v[i]$. If $B_v[i] = 0$, the i-th symbol of S_v lies in $[a_v, m_v]$ and appears in the left child's subsequence S_0. Its position within S_0 is the number of zeros of B_v up to position i, namely $\text{rank}_0(B_v, i)$. Descend to the left child with index updated to this value. If $B_v[i] = 1$, descend symmetrically to the right child with index $\text{rank}_1(B_v, i)$. The traversal terminates at a leaf $[c, c]$, which identifies the symbol $S[i] = c$. The cost is $\lceil \lg \sigma \rceil$ constant-time rank queries plus constant-time navigation, giving $O(\lg \sigma)$ time overall.

Symbol rank computes $\text{rank}_c(S, i)$, the number of occurrences of c in $S[1..i]$, by a top-down traversal along the fixed root-to-leaf path of c. At node v the direction of descent depends on c, not on the bitmap. When $c \le m_v$, the occurrences of c among $S[1..i]$ contribute to the zeros of $B_v[1..i]$. Update i to $\text{rank}_0(B_v, i)$ and descend to the left child. When $c > m_v$, update i to $\text{rank}_1(B_v, i)$ and descend to the right child. At the leaf $[c, c]$, the final index is $\text{rank}_c(S, i)$ for the original query. The path has length $\lceil \lg \sigma \rceil$ and each step performs one binary rank, yielding $O(\lg \sigma)$ time. Figure 2.5 traces a concrete instance of this descent.

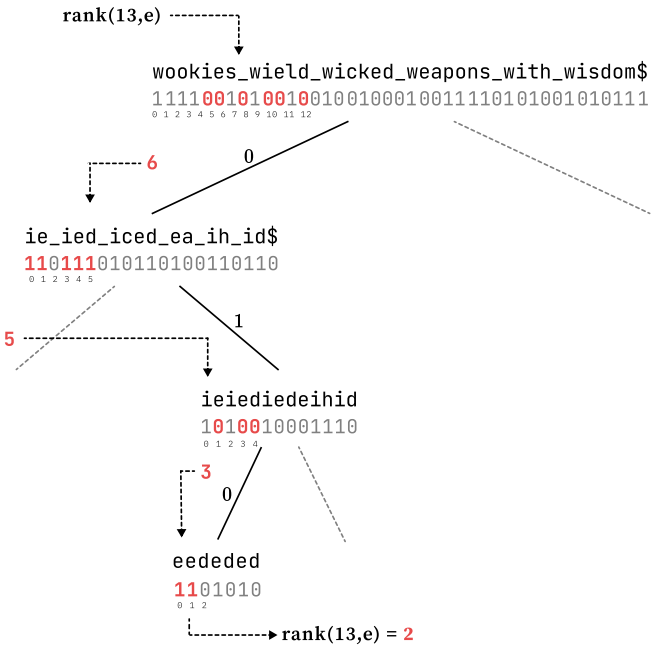

Figure 2.5: Trace of $\text{rank}_e(S, 13) = 2$ on the wavelet tree of Figure 2.4. The descent follows the root-to-leaf path of the target symbol e, remapping the query prefix length through one binary rank at each of four levels. The intermediate values $13 \to 6 \to 5 \to 3 \to 2$ trace the query index from the root to the leaf [e, e]. Each remapping is a rank_0 or rank_1 on the local bitmap, selected by whether e falls in the left or right half of the current alphabet interval.

Symbol select computes $\text{select}_c(S, j)$, the position of the j-th occurrence of c in S, by a bottom-up traversal starting at the leaf $[c, c]$ with $j_0 = j$. Moving from the current node v' to its parent v, the index is mapped

through a binary select on B_v. If v' is the left child of v, the j-th occurrence of c inside $S_{v'}$ corresponds to the j-th zero of B_v, so update j to $\text{select}_0(B_v, j)$. If v' is the right child, update j to $\text{select}_1(B_v, j)$. When the traversal reaches the root, the final index is the answer $\text{select}_c(S, j)$ in S. The total cost is $\lceil \lg \sigma \rceil$ constant-time selects, giving $O(\lg \sigma)$ time.

The construction and the three traversals give the following bound.

Theorem 2.3.1 (Wavelet tree [31]) *A sequence $S[1..n]$ over an alphabet $[1..\sigma]$ can be represented in*

$$n\lceil \lg \sigma \rceil \; + \; o(n) \lg \sigma \; + \; O(\sigma \lg n) \quad \textit{bits}$$

so that $\text{access}(S, i)$, $\text{rank}_c(S, i)$, *and* $\text{select}_c(S, j)$ *are supported in $O(\lg \sigma)$ time on the word RAM with word size $\omega = \Omega(\lg n)$.*

Sketch. The tree is the balanced binary partition of $[1..\sigma]$ with bitmaps $\{B_v\}$ as defined above. At every level, the bitmaps partition the n positions of S, so their combined length is n bits per level and $n\lceil \lg \sigma \rceil$ bits in total. Constant-time rank and select auxiliaries on each B_v from Theorem 2.1.1 and Theorem 2.1.2 contribute $o(|B_v|)$ bits per bitmap and $o(n) \lg \sigma$ bits summed over all levels. The tree topology, with σ leaves and child pointers of $O(\lg n)$ bits, contributes $O(\sigma \lg n)$ bits. The three operations are carried out by the top-down access and rank traversals and the bottom-up select traversal described above. Each performs one binary rank or select per level and visits $\lceil \lg \sigma \rceil$ levels, for $O(\lg \sigma)$ time. □

The $O(\sigma \lg n)$ overhead comes entirely from the explicit tree topology and the per-node offsets into the global bit storage. For alphabets of size $\sigma = \Theta(n)$ this term dominates and can overwhelm the $n\lceil \lg \sigma \rceil$ leading term, motivating the topology-free layouts surveyed by Navarro [32].

The topology term $O(\sigma \lg n)$ is asymptotically smaller than $n\lceil \lg \sigma \rceil$ only when $\sigma = o(n)$. For alphabets growing with the input, the pointer overhead can dominate the bitmaps themselves, and the representation is no longer succinct. Later pointerless layouts remove this explicit topology term at no asymptotic penalty in query time.

To remove the explicit topology, store the node bitmaps by level. At every level ℓ, concatenate the bitmaps of the 2^ℓ nodes into a single bitmap of length n, and concatenate the $\lceil \lg \sigma \rceil$ level bitmaps into one global string $B[1..n\lceil \lg \sigma \rceil]$. Level ℓ occupies positions $n(\ell - 1) + 1$ through $n\ell$ of B. The shape of the tree is altered so that all leaves are aligned to the left, which lets every level be filled to length exactly n without leaving gaps. A node v at level ℓ is identified by its bitmap's range $[l, r] \subseteq [n(\ell - 1) + 1, n\ell]$ within the level. The range of its left child at level $\ell + 1$ starts at $n + l$ and ends at $n + l + \text{rank}_0(B, l, r) - 1$, where $\text{rank}_0(B, l, r) = \text{rank}_0(B, r) - \text{rank}_0(B, l - 1)$ counts the zeros of B_v. The right child starts immediately after, extending for $\text{rank}_1(B, l, r)$ positions. Navigation from a node to its children requires one rank query per descent, so the downward traversal costs $O(\lg \sigma)$ rank queries, matching the pointered version. Upward traversal is handled by descending from the root to the target leaf to record every node's range, then unwinding the recursion. The asymptotic time is preserved. The topology is entirely encoded by B itself together with the level offsets, and the space drops to $n\lceil \lg \sigma \rceil + o(n) \lg \sigma$ bits, with no explicit $O(\sigma \lg n)$ term.

2.3.2 Entropy compression

The plain space bound counts one stored bit for every original position at every alphabet level. This is the correct worst-case accounting, but it treats a balanced bitmap and an almost constant bitmap as equally expensive. If the symbols of S are skewed, many splits produce skewed bitmaps, and those bitmaps are precisely the inputs on which the compressed dictionaries of Theorem 2.1.7 save space. Write $n_c = |\{k : S[k] = c\}|$ for the number of occurrences of symbol c in S. The zero-order empirical entropy of S is

$$H_0(S) \;=\; \sum_{c\in[1..\sigma]} \frac{n_c}{n} \lg \frac{n}{n_c} \;\le\; \lg \sigma,$$

with the inequality strict whenever the distribution of symbols is unbalanced. Thus the compression should be applied locally, to the bitmaps that record the binary choices. If each B_v is stored with an entropy-compressed indexable dictionary while preserving constant-time binary rank and select, the traversals above are unchanged. This local replacement gives a compressed sequence representation only if the bitmap costs, summed over all internal nodes, match the entropy of S. Grossi, Gupta, and Vitter [31] prove this entropy telescoping for the wavelet trees used in their index. In the zero-order sequence setting it gives the equality below.

[31]: Grossi et al. (2003), *High-order entropy-compressed text indexes*

For a binary wavelet tree T, write $n_v = |S_v|$ for the length of the subsequence at node v. If an internal node v has children v_0 and v_1, then

$$|B_v| H_0(B_v) = n_v \lg n_v - n_{v_0} \lg n_{v_0} - n_{v_1} \lg n_{v_1}.$$

The identity depends only on the bottom-up composition of conditional probabilities through the binary partition. The shape of the binary wavelet tree is immaterial. A skewed or Huffman-shaped layout yields the same $nH_0(S)$ as the balanced one.

Empty children contribute zero under the usual entropy convention. Summing this expression over all internal nodes cancels every non-root internal term. It appears once with positive sign in its own bitmap and once with negative sign in its parent's bitmap. The root contributes $n \lg n$, while each leaf $[c, c]$ contributes $-n_c \lg n_c$. Therefore

$$\sum_{v \text{ internal}} |B_v| \cdot H_0(B_v) \;=\; nH_0(S).$$

The per-node zero-order costs therefore reproduce the zero-order cost of the whole sequence.

Theorem 2.3.2 (Entropy-compressed wavelet tree [31]) *A sequence $S[1..n]$ over an alphabet $[1..\sigma]$ can be represented in*

$$nH_0(S) \;+\; o(n \lg \sigma) \quad \textit{bits}$$

so that access(S, i), rank$_c(S, i)$, *and* select$_c(S, j)$ *are supported in $O(\lg \sigma)$ time on the word RAM with word size $\omega = \Omega(\lg n)$.*

Sketch. Take the pointerless wavelet tree of Theorem 2.3.1. For each nonempty internal node v, store B_v with the entropy-compressed fully indexable dictionary of Theorem 2.1.7. It stores the bitmap within its zero-order entropy plus lower-order redundancy and supports rank$_b$ and select$_b$ in $O(1)$ time.The entropy terms of these dictionaries sum to $nH_0(S)$ by the identity above. The compressed binary wavelet-tree

analysis bounds the summed lower-order redundancies by $o(n \lg \sigma)$ bits, so the total space is $nH_0(S) + o(n \lg \sigma)$. The traversals of Theorem 2.3.1 do not change. Each step now performs the constant-time binary rank or select operation supplied by Theorem 2.1.7 on the local dictionary for the current bitmap. A traversal visits $\lceil \lg \sigma \rceil$ levels, so access, rank, and select take $O(\lg \sigma)$ time. □

[33]: Ferragina et al. (2007), *Compressed Representations of Sequences and Full-Text Indexes*

Ferragina, Mäkinen, Manzini, and Navarro [33] refine the sequence representation problem in a different direction. They treat the binary wavelet tree of Grossi, Gupta, and Vitter as the baseline that already gives $nH_0(S) + o(n \lg \sigma)$ bits and $O(\lg \sigma)$ time. Their generalized wavelet tree uses an r-ary alphabet partition, so each internal node stores a sequence over child indices $[1..r]$ instead of a binary bitmap. For polylogarithmic alphabets this yields $nH_0(S) + o(n)$ bits and constant-time access, rank, and select. For larger alphabets, choosing $r = O(\lg n/(\lg \lg n)^2)$ gives $nH_0(S) + O(\sigma \lg n) + o(n \lg \sigma)$ bits and time $O(\lg \sigma / \lg \lg n)$. When $\sigma = o(n)$, the space simplifies to $nH_0(S) + o(n \lg \sigma)$ bits.

The same structure supports two uses. As a sequence representation, it answers access, rank_c, and select_c in $O(\lg \sigma)$ time within $nH_0(S) + o(n \lg \sigma)$ bits. As a reordering device, the leaves, traversed left to right, present the symbols of S in stable sorted order. The j-th position inside the leaf $[c, c]$ corresponds to the j-th occurrence of c in S, and the bottom-up select traversal of Theorem 2.3.1 converts this sorted position into the original position in S via one select_0 or select_1 per level. The sequence S and its stable sorting are both directly readable from the same bitmaps, related to each other by a $\lceil \lg \sigma \rceil$-level chain of rank/select operations.

Both uses rely on a hierarchical binary partition of the alphabet, instantiated level by level as a bitmap on which rank and select answer the primitive queries. The same template, a hierarchical binary partition supported by rank on the induced bitmaps, generalizes from sequences to labeled ordinal trees in a later chapter of this thesis. There the partition acts on the alphabet of node labels, and rank on bitmaps becomes the primitive for queries on labeled trees. The wavelet tree is the prototypical instance of this template on strings.

Succinct Representations of Trees | 3

3.1 Encoding problem 25
3.2 Depth-first encodings . . . 29
3.3 Fully functional encodings 34
3.4 Labeled trees and α-operations 38
3.5 Tree extraction 41
3.6 α-operations on labeled trees 43
3.7 Path queries 46

A single set can be stored as a bitvector: once the universe positions are marked, the rank and select structures of Chapter 2 make those marks searchable within a low-order term of the relevant information-theoretic bound. A collection adds structure that no single bitvector contains. Sets can be organized so that one is decoded relative to another, and a query then follows the dependencies chosen by the representation. When those dependencies form a tree, the query structure has two compressed layers to manage, the data stored at the nodes and the tree that tells the query where to move. This chapter isolates the second layer. It first represents the shape of an ordinal tree in succinct space while preserving the navigation expected from pointers. It then adds node labels, so later constructions can attach set information to a node rather than treating the node as a position only.

3.1 Encoding problem

A pointer representation makes navigation direct: a node stores references to the neighbors a query may ask for. On an n-node tree this costs $O(n \lg n)$ bits, because each reference has width $\Theta(\lg n)$. A succinct representation has to replace most of those references by a bit string that still determines the shape. This section separates the problem into three steps. We first fix the class of trees, then list the operations that a replacement for pointers must support, and finally count how many shapes an encoding has to distinguish before adding the first concrete representation.

3.1.1 Catalan lower bound

Before comparing encodings, we need a common model. We encode rooted trees whose children have a left-to-right order. A replacement for pointers must still answer questions about parents, children, siblings, ancestors, and traversal positions. With the object and operations fixed, we can count how many shapes an encoding has to distinguish.

An *ordinal tree* is a rooted tree whose children at every node are linearly ordered, so the children of a node v form a sequence $c_1, c_2, \ldots, c_{\deg(v)}$ rather than an unordered set. Munro and Raman call the same object a *rooted ordered tree* [34]. We use ordinal tree throughout. The unique node with no parent is the root. A node with no children is a leaf, and a node with at least one child is internal. The degree of v is the number of children of v. The depth of v is its distance from the root, with the root at depth 0. The subtree rooted at v, written T_v, is the induced subtree on v and all its descendants, and its size is $|T_v|$. A preorder traversal visits a node before its children, recursively from left to right. A postorder traversal visits the children before the node. Both traversals order all

[34]: Munro et al. (2001), *Succinct representation of balanced parentheses and static trees*

nodes, and preorder will be the default numbering when nodes are addressed by integers in $[1..n]$.

Since the representation replaces pointers, it must still answer the elementary moves that pointers expose. Jacobson's binary-tree model used the three operations $\{\texttt{left-child}, \texttt{right-child}, \texttt{null}\}$, while later ordinal-tree representations support more operations [12, 35]. The definition below records the operations used in this chapter.

[12]: Jacobson (1989), *Space-efficient static trees and graphs*
[35]: Navarro et al. (2014), *Fully Functional Static and Dynamic Succinct Trees*

Definition 3.1.1 (Operations on ordinal trees) *Given an ordinal tree T on n nodes addressed by integers in $[1..n]$, a representation should support, for every node $x \in T$, the following operations.*

- $\texttt{parent}(x)$, *the parent of x (undefined at the root).*
- $\texttt{first\_child}(x)$, *the leftmost child of x (undefined at a leaf).*
- $\texttt{next\_sibling}(x)$, *the sibling of x immediately to its right (undefined at the rightmost child).*
- $\texttt{child}(x, i)$, *the i-th child of x in left-to-right order.*
- $\texttt{degree}(x)$, *the number of children of x.*
- $\texttt{subtree\_size}(x) = |T_x|$, *the size of the subtree rooted at x.*
- $\texttt{depth}(x)$, *the depth of x measured from the root at 0.*
- $\texttt{level\_ancestor}(x, d)$, *the ancestor of x at d levels above x.*
- $\texttt{ancestor}(x, d)$, *the ancestor of x at depth d.*
- $\texttt{LCA}(x, y)$, *the lowest common ancestor of x and y.*
- $\texttt{preorder\_rank}(x)$ *and* $\texttt{preorder\_select}(i)$, *mapping nodes to their preorder positions and back.*

The shape count is easiest through parentheses. During a preorder traversal, write an open parenthesis when entering a node and a close parenthesis when leaving its subtree. An n-node tree gives a balanced string with n pairs whose first parenthesis matches the last one [34]. Removing the outer pair gives the inner balanced string, and adding the outer pair back gives the traversal string of one ordinal tree. This is the count used by Navarro and Sadakane for n-node ordinal trees [35]:

[34]: Munro et al. (2001), *Succinct representation of balanced parentheses and static trees*
[35]: Navarro et al. (2014), *Fully Functional Static and Dynamic Succinct Trees*

$$N_n = \frac{1}{2n-1}\binom{2n-1}{n-1} = \frac{2^{2n}}{\Theta(n^{3/2})}.$$

Distinguishing all these shapes requires one memory state per shape, so any encoding needs at least $\lg N_n$ bits in the worst case. In the lower-bound form used for succinct trees, this is

$$\lg N_n = 2n - \Theta(\lg n),$$

so the information-theoretic minimum is $2n$ bits up to a lower-order correction. A pointer representation uses $O(n \lg n)$ bits, since it stores $O(n)$ references of $\Theta(\lg n)$ bits each. Compared with the lower bound, this is a factor $\Theta(\lg n)$ away from optimal.

Theorem 3.1.1 (Catalan lower bound [35]) *Every representation that distinguishes all ordinal trees on $n \geq 2$ nodes has worst-case space at least $\lg N_n = 2n - \Theta(\lg n)$ bits, where $N_n = \frac{1}{2n-1}\binom{2n-1}{n-1}$ is the number of such trees.*

Sketch. The count above gives N_n possible inputs. If the worst-case space is b bits, the representation has at most 2^b memory states. Distinguishing

all inputs requires $2^b \geq N_n$, hence $b \geq \lg N_n$. The displayed bound gives $\lg N_n = 2n - \Theta(\lg n)$. □

Since Theorem 3.1.1 gives $\lg N_n = 2n - \Theta(\lg n)$, using $2n + o(n)$ bits is the same as using $(1 + o(1)) \lg N_n$ bits. This is the succinct-space criterion used by Navarro and Sadakane [35] and matches Jacobson's ratio-to-lower-bound formulation [12]. A tree representation must also answer the operations of Definition 3.1.1 in constant time on the word RAM.

The preorder parenthesis string above has the right order of space and determines the tree, yet finding the matching close, the enclosing pair, or a subtree boundary may still require a scan. The encodings that follow keep a near-minimum description of the shape and add sublinear indexing so that tree operations reduce to a constant number of queries on bitvectors or balanced-parenthesis sequences.

3.1.2 Level-order degrees as a first encoding

To obtain a first concrete encoding, ignore depth-first structure and record only how many children each node has. If the nodes are read in level order, each degree tells how many children must appear later in the same order. Jacobson [12] writes these degrees in unary and calls the result the *level-order unary degree sequence*. Later work abbreviates this name as LOUDS [35, 36].

[12]: Jacobson (1989), *Space-efficient static trees and graphs*

[35]: Navarro et al. (2014), *Fully Functional Static and Dynamic Succinct Trees*

[36]: Benoit et al. (2005), *Representing Trees of Higher Degree*

We build B from the augmented tree. First, we add a synthetic *super-root*, a new node of degree one whose only child is the original root. The super-root is not part of T, and its only role is to make the original root, like every other original node, appear as a child of some node and therefore have one associated 1-bit. Second, we list the degrees of all nodes of the augmented tree in level order, starting from the super-root. Third, we encode each degree d in unary as 1^d0, namely d ones followed by one zero, and concatenate the codewords in the same order. The resulting bitvector B is the unary-encoded BFS list of child counts. Within each codeword, the 1-bits announce the children of the current node, and the final 0 terminates that node's block.

Figure 3.1 shows the construction on a tree of seven nodes, augmented to eight by the super-root.

Counting bits is straightforward. Each non-root node contributes exactly one 1-bit, since it is the child of some other node, so the 1-count equals the number of children summed over all parents, which equals the number of non-root nodes. Each node contributes exactly one 0-bit, the terminator of its own degree codeword. With the super-root added, the augmented tree has $n + 1$ nodes, of which n are non-root, so B contains n ones and $n + 1$ zeros, for a total length of $2n + 1$.

Theorem 3.1.2 (LOUDS [12, 36]) *An ordinal tree of n nodes can be represented in $2n + 1 + o(n)$ bits so that, when a node is addressed by the position of its associated 1-bit in B, the operations* `parent`, `first_child`, `next_sibling`, `child`, *and* `degree` *run in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$. The bitvector B is stored verbatim with the rank/select structures of Theorem 2.1.1 and Theorem 2.1.2 on top.*

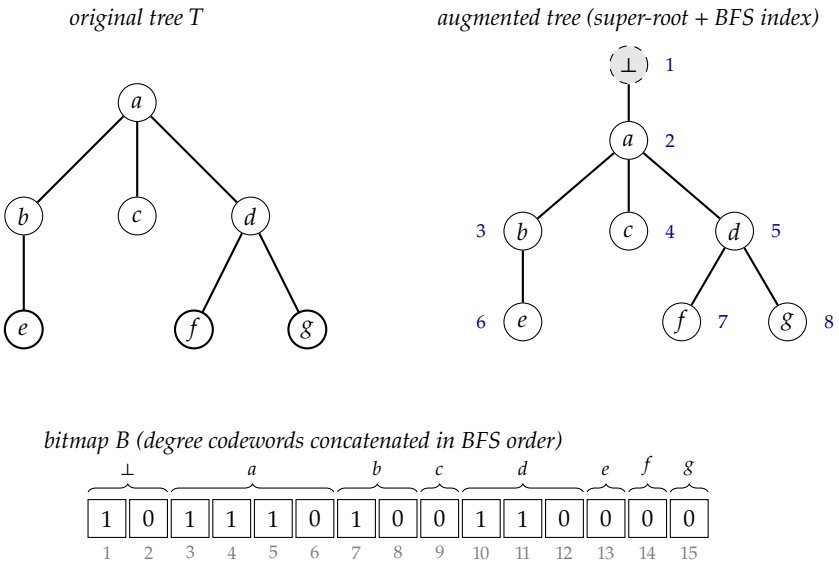

Figure 3.1: LOUDS encoding of an ordinal tree on seven nodes. The super-root $\perp$ is added so that every node, including the original root, has a unique parent and contributes a unary degree codeword. The augmented tree is traversed in BFS order, the BFS indices $1, \ldots, 8$ are shown next to each node, and the degree of each node in turn is written in unary as 1^d0, concatenated to form the bitmap B. Grouping braces above B mark the codewords contributed by the super-root and by nodes a, b, c, d, e, f, g in BFS order.

Proof. We have already counted the bits. The $o(n)$ overhead is the rank-and-select index of Theorem 2.1.1 and Theorem 2.1.2 stored with B, which gives $O(1)$ time for rank_b and select_b for $b \in \{0, 1\}$. Jacobson states the formulas for `parent`, `first_child`, and `next_sibling` in a bit-access model, where each rank or select query costs $O(\lg n)$ bit inspections. We use the same formulas with the word-RAM rank/select structures already introduced in this thesis, so each formula costs $O(1)$ time.

We use the local node address of this encoding, not the preorder address fixed in Definition 3.1.1. A node x of the original tree is represented by the position p_x of the 1-bit that announces x as a child in the augmented tree. Let $\beta_x = \text{rank}_1(B, p_x)$ be the BFS index of x among original nodes. The degree codeword of x is the block between the β_x-th and the $(\beta_x + 1)$-st 0-bit, namely the range $\text{select}_0(B, \beta_x) + 1, \ldots, \text{select}_0(B, \beta_x + 1)$.

For the parent query, we count how many degree codewords have ended before p_x is reached. The value $\text{rank}_0(B, p_x)$ is exactly the BFS index, among original nodes, of the parent that emitted this 1-bit. Hence, for every nonroot node x, the address of the parent is $p_{\texttt{parent}(x)} = \text{select}_1(B, \text{rank}_0(B, p_x))$. The original root is the special case whose parent is undefined in T.

For the first-child query, the child block of x begins at $s_x = \text{select}_0(B, \beta_x) + 1$. If $B[s_x] = 1$, then $p_{\texttt{first\_child}(x)} = s_x$. If $B[s_x] = 0$, the degree block is empty and x is a leaf. For the next-sibling query, we use the fact that siblings are emitted as consecutive 1-bits inside the same degree block. Therefore $p_{\texttt{next\_sibling}(x)} = p_x + 1$ exactly when $B[p_x + 1] = 1$.

We derive `child` and `degree` from the same block. If i is at most the number of 1-bits before the terminator of the block, then $p_{\texttt{child}(x,i)} = s_x + i - 1$. The number of such children is the length of the run of 1-bits before the next 0, so $\texttt{degree}(x) = \text{select}_0(B, \beta_x + 1) - \text{select}_0(B, \beta_x) - 1$. Every expression uses a constant number of bit accesses, rank/select calls, and arithmetic operations, so the claimed operations take $O(1)$ word-RAM time. □

LOUDS gives a succinct baseline: it stores the tree in $2n + 1 + o(n)$ bits

and answers the operations of Theorem 3.1.2 with a constant number of rank/select calls. This is succinct because Theorem 3.1.1 gives $2n - \Theta(\lg n)$ bits as the information-theoretic lower bound. The explicit bitmap is only $\Theta(\lg n)$ bits above that bound, apart from the $o(n)$ auxiliary index. What LOUDS does not give directly is the depth-first locality needed by operations such as `subtree_size`, `depth`, `ancestor`, `LCA`, `level_ancestor`, and `preorder_rank`. In BFS order, the descendants of a node may span many later levels, and their codewords are interleaved with codewords of nodes outside the subtree.

This limitation explains the move to depth-first encodings. Instead of adding more local formulas to the same level-order bitmap, the next representations write the tree in an order where every subtree occupies a contiguous range. Balanced parentheses obtains this locality by writing one pair of parentheses per node in DFS order, while DFUDS keeps the unary degree blocks but writes them in DFS order.

3.2 Depth-first encodings

The limitation of LOUDS is not its space but its order. A level-order sequence keeps the children of a node close to the node's degree block, which is why the formulas of Theorem 3.1.2 are simple, but it spreads a subtree across several BFS levels and interleaves it with nodes outside the subtree. The operations still missing from the LOUDS baseline, including `subtree_size`, `depth`, `ancestor`, `LCA`, `level_ancestor`, and `preorder_rank`, are easier to express when descendants form a contiguous range or when ancestry becomes parenthesis nesting. We now replace the level-order layout with depth-first layouts. First, we use balanced parentheses, which writes one open and one close parenthesis per node in preorder. Then we keep the unary degree blocks of LOUDS but write them in preorder instead of BFS order, obtaining DFUDS.

3.2.1 Balanced parentheses

We begin with the simplest depth-first layout. A preorder traversal of T writes an open parenthesis when a node is first visited and the matching close parenthesis when the traversal leaves its subtree. The result is a string $B[1..2n]$ over $\{\,(\,,\,)\,\}$ in which each node contributes exactly one matched pair. We fix this preorder convention and identify each node v with the position i_v of its open parenthesis in B. The preorder rank of v is then $\mathrm{rank}_{(}(B, i_v)$. This address makes subtree intervals explicit. If $j_v = \texttt{findclose}(i_v)$, then the substring $B[i_v..j_v]$ is exactly the balanced-parenthesis sequence of the subtree rooted at v, and its length is $2 \cdot |T_v|$. Figure 3.2 shows the matched pairs and subtree intervals on the running tree.

The basic numerical quantity on B is the *excess* at a position. For $i \in [1..2n]$ define

$$e(B, i) \;=\; \mathrm{rank}_{(}(B, i) \;-\; \mathrm{rank}_{)}(B, i),$$

the number of open parentheses minus the number of close parentheses among the first i symbols. With this inclusive convention, the excess at an open position i_v is $\texttt{depth}(v) + 1$, because the unmatched opens are exactly

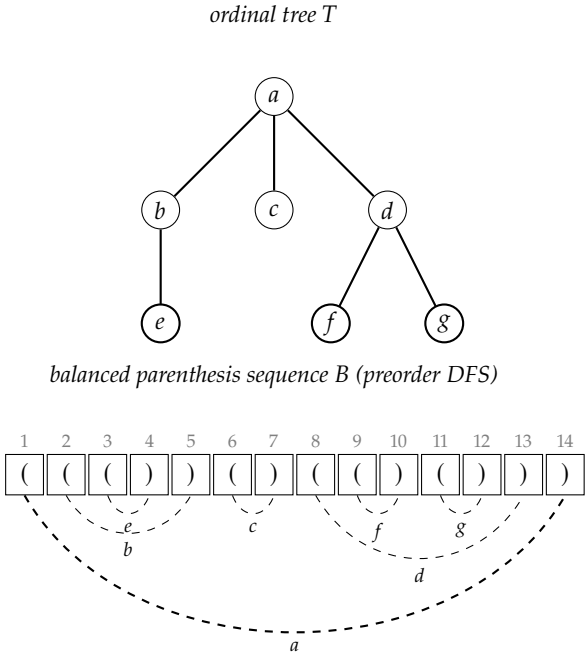

Figure 3.2: Balanced parenthesis encoding of the same seven-node ordinal tree used in Figure 3.1. A preorder DFS visit writes an open parenthesis on entry to a node and a close on exit. Each dashed arc joins the matching pair that represents one node, labelled by that node. The substring from a node's open parenthesis through its matching close is exactly the encoding of its subtree, of length $2 \cdot |T_v|$.

the ancestors of v together with v itself. The excess is non-negative for every prefix of B and equals zero only at the end of B, since the sequence is balanced. If i is an open parenthesis, then its matching close is the smallest $j > i$ such that $e(B, j) = e(B, i) - 1$. If v is not the root, then the open parenthesis of its parent is the largest open position $i' < i_v$ such that $e(B, i') = e(B, i_v) - 1$.

[34]: Munro et al. (2001), *Succinct representation of balanced parentheses and static trees*

Munro and Raman [34] index the balanced sequence through five primitives. The operation `findclose`(i) returns the close parenthesis matching the open at i. The operation `findopen`(i) returns the open parenthesis matching the close at i. The operation `excess`(i) returns the excess value at i. The operation `enclose`(i) returns the open parenthesis of the closest matched pair that strictly contains the pair beginning at i. The operation `double_enclose`(i, j) returns the closest matched pair that contains two non-overlapping matched pairs beginning at i and j, when such a pair exists. The rank identity above lets us compute `excess`(i) from binary rank, so the explicit tree-navigation primitives we keep are `findclose`, `findopen`, `enclose`, and `double_enclose`. Together with binary rank, these primitives are enough for the parent, subtree-size, depth, sibling, preorder-rank, and LCA operations below, while the i-th child still costs $O(i)$ in this encoding.

Theorem 3.2.1 (Balanced parentheses [34]) *Given a balanced parenthesis sequence B of length $2n$, there is a data structure of $o(n)$ bits attached to B such that on the word RAM with word size $\omega = \Omega(\lg n)$ the operations `findclose`, `findopen`, `excess`, `enclose`, and `double_enclose` on B all run in $O(1)$ time. Consequently, an ordinal tree on n nodes admits a representation in $2n + o(n)$ bits, with each node represented by its opening parenthesis, supporting `parent`(v), `subtree_size`(v), `depth`(v), `first_child`(v), `next_sibling`(v), `preorder_rank`(v), and `LCA`(u, v) in $O(1)$ time, with `child`(v, i) supported in $O(i)$ time.*

Sketch. The auxiliary structure is a three-level decomposition of B that mirrors the hierarchical layout used by Jacobson [11, 12] for rank and select, but indexes the excess sequence rather than raw bit counts. Writing $b = \lg^2 n$, the sequence B is partitioned into blocks of length b. Each such block is partitioned into subblocks of length $\lg^2 b = 4(\lg \lg n)^2$, and each subblock is partitioned into bottom-level chunks of length $2 \lg \lg n$. For cross-block matches, the structure stores pointers only for selected representatives: whenever consecutive cross-block parentheses stop matching into the same block, the first parenthesis of the new run is selected. At each top-level boundary it stores the current excess and the immediately preceding selected representative. Subblock headers carry the analogous information one level finer. If a string is divided into k blocks, then it has at most $2k - 3$ such representatives [12]. Since the number of top-level blocks is $O(n/\lg^2 n)$, the cumulative space for these pointers is $O(n/\lg n)$ bits. A precomputed lookup table [37], indexed by a low-level block pattern and an in-block position, resolves local `findclose` and `findopen` queries in $O(1)$ word-RAM operations. Its size is polynomial in $\lg n$ and therefore $o(n)$. A query $\texttt{findclose}(i)$ first uses the local table. If the match is not local, the procedure checks the subblock and top-level block summaries and uses the stored representative links to jump to the block that contains the answer. The same fixed hierarchy handles `findopen` by the symmetric search on right excess. Analogous bookkeeping of enclosing pairs supports `enclose` and `double_enclose` in the same asymptotic budget. The reductions of the operations above are direct. For nonroot nodes, $\texttt{parent}(v) = \texttt{enclose}(i_v)$. We also have $\texttt{subtree\_size}(v) = (\texttt{findclose}(i_v) - i_v + 1)/2$, $\texttt{depth}(v) = e(B, i_v) - 1$, $\texttt{first\_child}(v) = i_v + 1$ when $B[i_v + 1] = \texttt{(}$ and undefined otherwise, $\texttt{next\_sibling}(v) = \texttt{findclose}(i_v) + 1$ when that position exists and holds an open parenthesis, and $\texttt{preorder\_rank}(v) = \text{rank}_{\texttt{(}}(B, i_v)$. For $\texttt{LCA}(u, v)$, assume $i_u \le i_v$. If $i_v \le \texttt{findclose}(i_u)$, then u is an ancestor of v and the answer is u. Otherwise the two matched pairs are nonoverlapping, and $\texttt{double\_enclose}(i_u, i_v)$ returns their closest enclosing pair, namely the lowest common ancestor. □

[11]: Jacobson (1988), *Succinct Static Data Structures*
[12]: Jacobson (1989), *Space-efficient static trees and graphs*

[37]: Munro (1996), *Tables*

The lookup-table device is the four-Russians technique in the abstract sense [13], but Munro-Raman attribute the load-bearing tabulation construction to [37].

The reductions are immediate consequences of the bijection node $\leftrightarrow$ matched pair. A nonroot node's parent is the closest enclosing pair, which is what $\texttt{enclose}(i_v)$ returns. The subtree of v occupies the substring $B[i_v..\texttt{findclose}(i_v)]$, of length $2|T_v|$, so the subtree size is half the difference. The depth of v is the number of unmatched ancestor opens before i_v, hence $e(B, i_v) - 1$ under our inclusive convention. The first child of v exists exactly when the symbol at $i_v + 1$ is an open parenthesis, since otherwise $B[i_v + 1] = \texttt{)}$ matches i_v and v is a leaf. The next sibling lives at $\texttt{findclose}(i_v) + 1$ when that position exists and holds an open parenthesis, since $\texttt{findclose}(i_v)$ is the close of v's subtree, the next position belongs to the same parent, and an open there starts the next sibling. The preorder rank of v is the count of open parentheses up to i_v, which is the binary $\text{rank}_{\texttt{(}}(B, i_v)$ of Theorem 2.1.1. The i-th child is reachable by $i - 1$ successive applications of `next_sibling` starting from $\texttt{first\_child}(v)$, each costing $O(1)$ via `findclose`, for total cost $O(i)$.

The lowest common ancestor also admits a range-minimum view. During a DFS traversal of T, the lowest common ancestor of u and v is the shallowest node visited between the first occurrences of u and v in the traversal. Bender and Farach-Colton [38] use this observation to

[38]: Bender et al. (2000), *The LCA Problem Revisited*

reduce LCA to RMQ on the level array of an Euler tour, and then give an $\langle O(n), O(1)\rangle$ preprocessing and query algorithm for the special case of ± 1 range minimum queries. The excess sequence of a balanced-parenthesis traversal has the same ± 1 property. Under our inclusive convention, the excess at an open parenthesis is the node depth plus one, and at a close parenthesis it is the depth after leaving the corresponding subtree. Thus minima in the excess encode the same ancestor-boundary information used by the Euler-tour level minimum. We will use this RMQ viewpoint when discussing LCA, while the BP theorem above can already support LCA through `double_enclose`.

Example 3.2.1 For the tree of Figure 3.2, the BP sequence is $B =$ `((())()(())())` and has length 14. Node b sits at $i_b = 2$, and $\texttt{findclose}(2) = 5$, so $\texttt{subtree\_size}(b) = (5-2+1)/2 = 2$, in agreement with the subtree of b containing b and its only child e. The closest enclosing parenthesis of position $i_b = 2$ is at 1, so $\texttt{parent}(b) = a$. Finally, $e(B, 2) = 2$, and therefore $\texttt{depth}(b) = e(B, 2) - 1 = 1$.

The bound $O(i)$ for $\texttt{child}(v, i)$ is the remaining limitation of the encoding. For trees of bounded degree this is constant, but for high-degree trees a query for the i-th child of a node of degree $d \gg 1$ still pays $\Theta(i)$ findclose calls in the worst case. Munro and Raman raise this gap as an open question in their original paper. We next recover $\texttt{child}(v, i)$ in $O(1)$ while keeping constant-time parent and subtree-size access.

3.2.2 DFUDS

[36]: Benoit et al. (2005), *Representing Trees of Higher Degree*

We follow the construction of Benoit, Demaine, Munro, Raman, Raman, and Rao [36], which keeps the unary degree blocks of LOUDS but changes their order. LOUDS writes the unary degree of each node in level order. BP writes one open and one close parenthesis per node in depth-first order. DFUDS combines these choices by writing unary degree blocks in depth-first preorder. At node v of degree d_v, the traversal emits d_v open parentheses followed by one close. Concatenating these $d_v + 1$ symbols across all nodes yields a string of length $\sum_v (d_v + 1) = (n-1) + n = 2n - 1$. A leading open parenthesis is prepended to balance the string, bringing the total length to $2n$. The result is the *depth-first unary degree sequence* encoding, abbreviated DFUDS, shown in Figure 3.3.

The string is balanced by induction on the tree. A single-node tree is encoded as `()`. Now suppose that the root has d children and that the DFUDS strings of the child subtrees are balanced. The root contributes the artificial open, then d opens and one close for its own block. For each child subtree, we append its balanced DFUDS string after removing its artificial leading open. Removing that open leaves one extra close, and the d extra closes from the d child strings match the d opens in the root block. The artificial open at the beginning matches the last close of the whole string.

We identify a node v with the position p_v of the first parenthesis of its block. The artificial open at position 1 has no node, so the root starts at position 2. The address $p_v \in [1..2n]$ is the local node label used by the formulas below. When an integer preorder label in $[1..n]$ is needed, it is enough to count the blocks completed before p_v. Under our inclusive

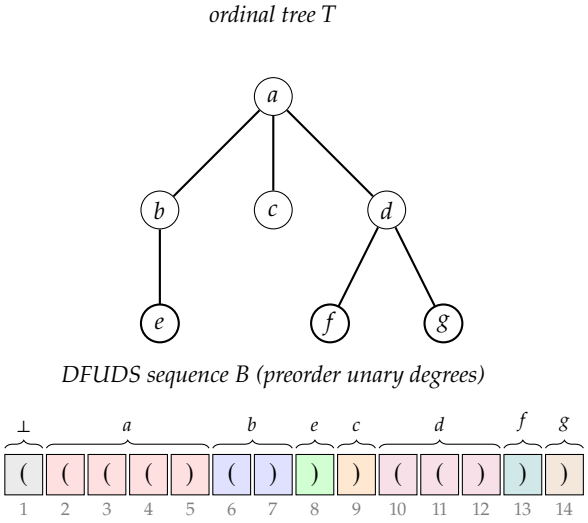

Figure 3.3: DFUDS encoding of the same seven-node ordinal tree of Figure 3.2. Each node v contributes a block of d_v open parentheses followed by one close, written in DFS preorder. A leading open parenthesis (block $\perp$) is prepended so that the resulting string of length $2n = 14$ is balanced. The colored blocks above the sequence highlight the per-node contributions of a, b, e, c, d, f, g in preorder. The position p_v of the first parenthesis of each block is the address by which the node is referenced.

rank convention, we write $r_v = \text{rank}_)(B, p_v - 1)$ for the number of close parentheses strictly before the block of v, so the preorder label of v is $r_v + 1$.

The DFUDS encoding supports degree, i-th child, parent, and subtree size in constant time on top of Theorem 3.2.1. The degree of the node at position p_v is the length of its run of opens, which is the gap from p_v to the next close in B. A single $\text{rank}_)$ followed by a $\text{select}_)$ identifies that close, giving $\texttt{degree}(v) = \text{select}_)(B, r_v + 1) - p_v$. For $1 \le i \le \texttt{degree}(v)$, the i-th child is found by counting the opens of v's block from right to left. The open at position $\text{select}_)(B, r_v+1)-i$ has its matching close immediately before the block of the i-th child, so $\texttt{child}(v, i) = \texttt{findclose}(\text{select}_)(B, r_v+1)-i)+1$. If $i > \texttt{degree}(v)$, the child is undefined. Thus the query uses one rank call, one select call, one `findclose` call, and constant arithmetic.

The parent of v is recovered from the close immediately preceding v's block. If $p_v = 2$, then v is the root. Otherwise let $q_v = \texttt{findopen}(p_v - 1)$ and $s_v = \text{rank}_)(B, q_v - 1)$. The position q_v lies inside the block of the parent. If $s_v = 0$, then the parent is the root, whose address is 2. Otherwise the parent block starts immediately after the s_v-th close, so $\texttt{parent}(v) = \text{select}_)(B, s_v) + 1$.

Subtree size is not read from the matching close of p_v. If v is a leaf, then $\texttt{subtree\_size}(v) = 1$. Otherwise the substring from p_v to $\texttt{findclose}(\texttt{enclose}(p_v))$ is the DFUDS description of the subtree rooted at v with its artificial leading open removed. Its length is $2|T_v|-1$, and therefore $\texttt{subtree\_size}(v) = (\texttt{findclose}(\texttt{enclose}(p_v)) - p_v)/2 + 1$.

Theorem 3.2.2 (DFUDS [36]) *An ordinal tree on n nodes admits a representation in $2n + o(n)$ bits that supports $\texttt{degree}(v)$, $\texttt{child}(v, i)$, $\texttt{parent}(v)$, and $\texttt{subtree\_size}(v)$ in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$. The representation is the DFUDS string of length $2n$, the auxiliary structure of Theorem 3.2.1, and binary* rank *and* select *indexes on the two parenthesis symbols.*

Sketch. The DFUDS string is balanced and has length $2n$, so Theorem 3.2.1 attaches an $o(n)$-bit auxiliary structure that supports `findclose`, `findopen`, `excess`, `enclose`, and `double_enclose` in $O(1)$ time, together

with rank and select on the open and close alphabets via Theorem 2.1.1 and Theorem 2.1.2. The four operations claimed are obtained by the formulas above, each evaluating to $O(1)$ word-RAM steps. □

DFUDS changes syntax, not the tree traversal. BP and DFUDS read nodes in preorder and use $2n$ parentheses, but a parenthesis carries different information in the two encodings. In BP, an open marks node entry and a close marks subtree exit. In DFUDS, the opens of a node's block give one handle per child, and the trailing close ends the block. This shift turns the cost of `child`(v, i) from a chain of i `next_sibling` jumps into a single `findclose` on a position computed by a rank/select pair, while preserving constant-time parent and subtree-size queries. We have now seen a level-order encoding that makes children local, a parenthesis encoding that makes subtrees local, and a unary depth-first encoding that combines the two properties for the four operations of Theorem 3.2.2. The auxiliary structures used so far are still built around separate primitives. In the next section we replace them with a uniform tree representation supporting the full operation set of Definition 3.1.1.

3.3 Fully functional encodings

The encodings above reach the succinct space target, but they do not yet give the full operation set of Definition 3.1.1. BP gives constant-time parent, subtree size, depth, siblings, preorder rank, and LCA, but pays $O(i)$ for `child`(v, i). DFUDS restores constant-time child, parent, degree, and subtree size, yet its support still rests on separate indexes for parenthesis primitives. As operations such as `leftmost_leaf`, `leaf_select`, `degree`, `child_rank`, and `level_ancestor` are added, the classical BP line accumulates auxiliary structures tied to individual queries. We now move from encoding-specific support to uniform representations. The first, due to He, Munro, and Rao [39], augments a tree covering with an $o(n)$-bit catalog of microtree patterns and supports every operation of Definition 3.1.1 in $O(1)$ time within $2n + o(n)$ bits. The second, due to Sadakane and Navarro [35], keeps the BP sequence but answers the needed excess primitives through one range min-max tree, reaching the same static bound and extending naturally to the dynamic setting.

[39]: He et al. (2007), *Succinct Ordinal Trees Based on Tree Covering*

[35]: Navarro et al. (2014), *Fully Functional Static and Dynamic Succinct Trees*

The starting point is a recursive cover of T at two carefully chosen sizes. With parameter M, the covering procedure covers the nodes of T by *minitrees*, connected subgraphs that may share only a common root. Every minitree has size between M and $3M - 4$, except for a possible smaller minitree containing the root of T. The same procedure is then applied inside each minitree with parameter M', producing a cover by *microtrees*. Geary, Raman, and Raman [40] introduced the covering algorithm and proved the size bound. He, Munro, and Rao [39] use this two-level cover in their fully functional representation. Choosing $M = \max\{\lceil \lg^4 n \rceil, 2\}$ and $M' = \max\{\lceil \lg n/24 \rceil, 2\}$ gives $O(n/\lg^4 n)$ minitrees and $O(n/\lg n)$ microtrees in total. Figure 3.4 sketches the nesting pattern of this two-level cover on a small example.

[40]: Geary et al. (2006), *Succinct Ordinal Trees with Level-Ancestor Queries*

Lemma 3.3.1 (Tree covering size bound [40]) *For any ordinal tree T on n nodes and any integer $M \geq 2$, the covering procedure covers the nodes of T with*

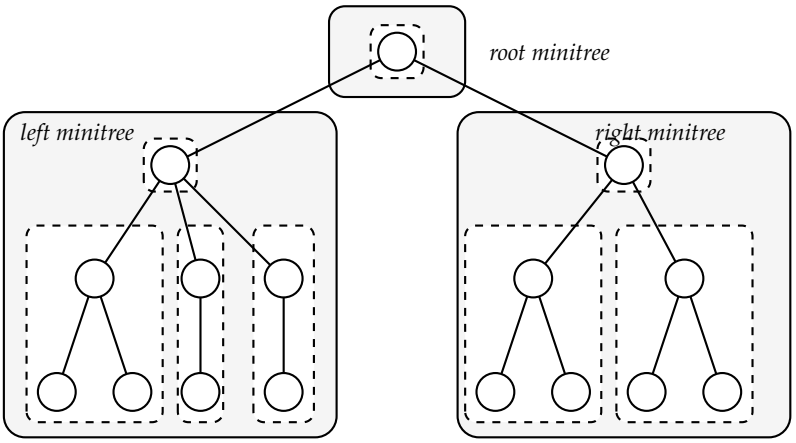

Figure 3.4: A schematic two-level tree covering of an ordinal tree. Solid rectangles show minitree regions produced with parameter M. Dashed rectangles show microtree regions produced inside them with parameter $M' < M$, including the possible smaller piece that contains the root of a recursive cover. Different pieces may intersect only at a common root. The exact size bounds are those of Lemma 3.3.1. The drawing shows nesting and boundary sharing on a tree too small for the asymptotic parameters used in the representation.

connected subgraphs of T. Any two pieces are either disjoint or intersect only at their common root. Every piece A has $|A| \leq 3M - 4$, and $|A| \geq M$ unless A contains the root of T. Consequently the cover has $\Theta(\max\{1, n/M\})$ pieces.

The size cap $3M - 4$ is what makes microtrees small enough for a catalog while keeping the number of pieces $\Theta(n/M)$ in the regime used below. The lower bound M rules out pathological splits and is essential for the count of pieces to be sublinear.

A boundary node may belong to more than one minitree or microtree, so He, Munro, and Rao address it by recording the containing minitree, the containing microtree, and the local preorder position. If a node has several such addresses, we use the lexicographically smallest one as its canonical address. Extended microtrees keep promoted boundary copies so that local navigation remains possible, and the conversion between canonical addresses and preorder indices is supported in $O(1)$ time. We use this conversion as an available constant-time primitive.

The second-level parameter $M' = \max\{\lceil \lg n/24 \rceil, 2\}$ is calibrated so that the catalog of microtree shapes is sublinear. For the asymptotic range in which $\lceil \lg n/24 \rceil \geq 2$, the size bound of Lemma 3.3.1 gives at most $3M' - 4 \leq \lceil \lg n/8 \rceil$ nodes per microtree, which is the upper bound used by He, Munro, and Rao. The elementary Catalan bound on ordinal trees of size at most h gives fewer than 4^h shapes. Substituting $h = \lceil \lg n/8 \rceil$ yields $O(n^{1/4})$ possible microtree shapes, which is the shape universe used in their accounting. Since each shape has only $O(\lg n)$ local positions, lookup tables indexed by a shape and a constant number of local positions occupy $o(n)$ bits.

The local component is a catalog over microtree-sized objects. Once a query has been reduced to data that lie inside a single microtree, the shape of that microtree and the local positions determine the answer. He, Munro, and Rao therefore precompute one set of universal tables over all possible microtree shapes. One table, indexed by a shape and two local preorder positions, returns the local node of maximum depth between those positions, which is the local subproblem needed by the height query. Another table, indexed by a shape and two local nodes, returns their lowest common ancestor when both nodes lie in the same microtree. These tables are stored once for the whole representation, not once per occurrence of a microtree, and occupy $o(n)$ bits because the shape universe has size $O(n^{1/4})$ and each shape contributes only a polylogarithmic number of local entries. This is the microtree analogue of the block lookup tables used in Theorem 2.1.1.

The remaining structures reconcile queries that cross microtree boundaries. A bitvector of length n marks the tier-1 *preorder changers*, namely

the $O(n/\lg^4 n)$ nodes whose preorder predecessor lies in a different minitree. A second bitvector marks the tier-2 preorder changers, namely the $O(n/\lg n)$ nodes whose predecessor lies in a different microtree. Both bitvectors are sparse and are stored with the rank/select dictionaries of Theorem 2.1.7 in $o(n)$ bits. The associated arrays store the local-address components of boundary nodes and the representatives needed for range maxima on the conceptual depth array. These summaries do not give a single formula for every operation. Instead, they move a query between global preorder, the relevant minitree or microtree, and the local catalog entry required by the operation. For LCA, a macro tree on the $O(n/\lg^4 n)$ minitree roots is preprocessed with the Bender-Farach-Colton LCA structure [38], taking $O(n/\lg^4 n \cdot \lg n) = o(n)$ bits. Distance then follows from LCA and depth. Analogous tier-2 macro trees handle microtree roots inside each minitree, and the same sparse-boundary accounting keeps their total space $o(n)$.

[38]: Bender et al. (2000), *The LCA Problem Revisited*

Theorem 3.3.2 (Tree covering succinct [39]) *An ordinal tree on n nodes admits a representation in $2n + o(n)$ bits that supports every operation of Definition 3.1.1 in $O(1)$ time on the word RAM with word size $\omega = \Theta(\lg n)$. The same representation supports, in $O(1)$ time within the same space bound, the additional operations*

- `child_rank`(x), *the position of x among its siblings.*
- `height`(x), *the height of the subtree rooted at x.*
- `distance`(x, y), *the length of the tree path between x and y.*
- `leftmost_leaf`(x) *and* `rightmost_leaf`(x), *the leftmost and rightmost leaves of the subtree rooted at x.*
- `leaf_rank`(x), `leaf_select`(i), *and* `leaf_size`(x), *mapping leaves to leaf-only positions, mapping those positions back, and counting leaves inside a subtree.*
- `level_leftmost`(d) *and* `level_rightmost`(d), *the leftmost and rightmost nodes at depth d.*
- `level_succ`(x) *and* `level_pred`(x), *the next and previous nodes at the same depth.*
- *the postorder rank and select operations, and the DFUDS rank and select counterparts of* `preorder_rank` *and* `preorder_select`.

Sketch. We start from the extended-microtree representation of Geary, Raman, and Raman [40]. Its main structures store the tree in $2n + o(n)$ bits. He, Munro, and Rao add only $o(n)$ further bits: sparse rank/select dictionaries for the tier-1 and tier-2 preorder changers, arrays storing the corresponding local-address components, catalog tables for microtree-local subproblems, range-extremum summaries on the conceptual depth array, and macro trees on the minitree and microtree roots. The counts established above give $O(n/\lg^4 n)$ minitrees, $O(n/\lg n)$ microtrees, and $O(n^{1/4})$ possible microtree shapes, so each family of added structures stays sublinear.

The query algorithms use these structures by operation family. Operations inherited from the Geary, Raman, and Raman representation keep their constant-time support on extended microtrees. Height reduces to a range maximum over the conceptual preorder depth array and is answered by the tiered range-extremum summaries. LCA first uses an in-microtree catalog lookup when both nodes lie in one microtree, and otherwise

moves to the appropriate tier-2 or tier-1 macro tree. Distance follows from LCA and depth. Leaf operations use marked pseudo-leaves at the mini and micro levels. DFUDS rank and select use the extended-mini-tree sums and the conversion between DFUDS numbers and local addresses. Level operations use the per-level summaries described for `level_leftmost`, `level_rightmost`, `level_succ`, and `level_pred`. In every case the algorithm performs a constant number of rank/select calls, address conversions, macro-tree queries, or catalog lookups, each of which takes $O(1)$ time.

The operation list of Definition 3.1.1 follows from the operations stated by He, Munro, and Rao. Their child, degree, depth, level-ancestor, LCA, and preorder rank/select operations are already in our list. We recover $\texttt{parent}(x)$ as $\texttt{level\_ancestor}(x, 1)$, $\texttt{first\_child}(x)$ as $\texttt{child}(x, 1)$ when $\texttt{degree}(x) > 0$, $\texttt{next\_sibling}(x)$ from $\texttt{child\_rank}(x)$ and the parent of x, $\texttt{subtree\_size}(x)$ by adding one to the number of descendants of x, and $\texttt{ancestor}(x, d)$ as $\texttt{level\_ancestor}(x, \texttt{depth}(x) - d)$. The detailed definitions of local addresses, extended microtrees, and the auxiliary arrays are those of He, Munro, and Rao [39]. □

The two-level cover predates Theorem 3.3.2 and is a result in its own right. Geary, Raman, and Raman used it to obtain a $2n + o(n)$-bit representation that supports `level_ancestor` in $O(1)$ time [40]. As presented above, LOUDS, BP, and DFUDS do not provide that operation in constant time. Farzan and Munro later developed a related recursive-decomposition framework for ordinal, cardinal, and free trees [41]. Theorem 3.3.2 extends the Geary, Raman, and Raman construction by augmenting it with the auxiliaries needed for every operation of Definition 3.1.1, while keeping the asymptotic space bound.

[40]: Geary et al. (2006), *Succinct Ordinal Trees with Level-Ancestor Queries*

[41]: Farzan et al. (2008), *A Uniform Approach Towards Succinct Representation of Trees*

A second route reaches the same bound without covering the tree. Sadakane and Navarro keep the BP sequence and attach one range min-max tree to it [35]. Their reduction expresses the operations through a small set of bitvector and excess primitives, including forward search, backward search, prefix sum, and range minimum or maximum queries. The same auxiliary structure supports these primitives, replacing the per-primitive hierarchies used by earlier BP representations.

[35]: Navarro et al. (2014), *Fully Functional Static and Dynamic Succinct Trees*

The auxiliary is the *range min-max tree*. Let $B[1..2n]$ be the BP sequence of T, and let $e(B, i)$ be the excess at position i of Section 3.2. For intuition, divide B into blocks of length $\Theta(\lg n)$ and build a complete tree whose leaves are the blocks of B, with each internal node storing the minimum and maximum excess below it. Sadakane and Navarro first implement the polylogarithmic-size building block as a complete k-ary tree with arity $k = \Theta(w/(c_0 \lg w))$ and chunks of size $w/2$, where w is the machine word and c_0 is the constant controlling the depth of that local structure. Three per-chunk arrays store the last excess, minimum excess, and maximum excess, and each occupies $O(N c_0 \lg w/w)$ bits for a segment of length N inside that building block. The in-chunk step is answered by a universal lookup table of $O(\sqrt{2^w} \lg w)$ bits. The full static construction combines these blocks with higher-level summaries and then reparameterizes the constants, giving the $O(n/\lg^c n)$ redundancy stated in Theorem 3.3.3 for any final constant $c > 0$.

The parenthesis operations reduce to these primitives in two different ways. The operations `findclose`, `findopen`, `enclose`, and `level_ancestor` become forward or backward searches for a target excess value. Such a search starts at the leaf containing the query position, walks upward until it finds a sibling subtree whose stored range contains the target, and then walks downward to localize the answer. By contrast, `rmq` and `RMQ` ask directly for a minimum or maximum excess in an interval and use the same min-max summaries. The lookup table answers the final in-chunk step in constant time. After the large-input reduction and the constant reparameterization, each primitive runs in $O(c)$ time on the word RAM, where $c > 0$ is the final constant fixing the redundancy bound.

Theorem 3.3.3 (Range min-max tree [35]) *Let $B[1..2n]$ be the balanced parenthesis sequence of an ordinal tree T on n nodes. For any constant $c > 0$, the BP sequence augmented with the range min-max tree occupies $2n + O(n/\lg^c n)$ bits and supports every operation of Definition 3.1.1 in $O(c)$ time on the word RAM with word size $\Theta(\lg n)$. The same approach extends to a dynamic ordinal tree on n nodes that supports every operation of Definition 3.1.1, together with insertions and deletions, in $O(\lg n)$ time per operation, within $2n + O(n/\lg n)$ bits of space.*

The proof of Theorem 3.3.3 is constructive. Each primitive resolves to a traversal of the range min-max tree, with the leaf step answered by the lookup table on $\Theta(\lg n)$-bit blocks. The dynamic extension is one advantage over Theorem 3.3.2. The two-level catalog used by the tree-covering scheme is hard to maintain under updates, while the range min-max tree stores the sequence in updatable blocks and refreshes the affected min-max summaries. Sadakane and Navarro [35] also present a finer dynamic tradeoff with $2n + O(n \lg\lg n/\lg n)$ bits. In that variant many operations take $O(\lg n/\lg\lg n)$ time, while updates and some degree or child-related queries have larger nonuniform bounds. We state the coarser dynamic variant in Theorem 3.3.3 because it gives a single $O(\lg n)$ bound for all supported operations.

Theorem 3.3.2 and Theorem 3.3.3 both attain $2n + o(n)$ bits with constant query time for every operation of Definition 3.1.1, the first with $O(1)$ time and the second with $O(c)$ time for any fixed constant $c > 0$. They reach this bound by two different routes. One builds a catalog of microtree patterns on a recursive cover. The other keeps the BP sequence and stores range min-max summaries of its excess sequence. Together they give the fully functional unlabeled tree representations needed in the rest of the thesis. The next section turns to the question of what changes when each node carries a label drawn from an alphabet Σ.

3.4 Labeled trees and α-operations

The encodings developed so far address ordinal trees in which a node is identified by its position in some traversal order and nothing else. Every operation of Definition 3.1.1 returns or consumes a position. No operation carries a node label. Theorem 3.3.2 and Theorem 3.3.3 both close the unlabeled question with $2n + o(n)$ bits and constant query time. The natural next step is the case in which each node carries a label drawn from an alphabet Σ of size σ. Once labels enter, the basic navigation

queries have label-aware counterparts: instead of asking for the parent of x, one asks for the closest ancestor of x whose label is a specified $\alpha \in \Sigma$, and instead of counting all descendants of x, one counts the descendants whose label is α. Setting $\sigma = 1$ recovers the same navigation, up to the depth convention in which $\texttt{depth}_\alpha(v)$ counts v itself.

We follow the terminology of He, Munro, and Zhou [42], who write *α-node* for a node whose label is α and refer to the label-aware variants as *α-operations*. The α-rank of a node x in a list is the number of α-nodes to the left of x in that list. The five α-operations below are tree analogues of the rank_c and select_c primitives that drive the wavelet tree of Section 2.3 on a sequence over an alphabet of size σ.

[42]: He et al. (2014), *A framework for succinct labeled ordinal trees over large alphabets*

Definition 3.4.1 (α-operations on labeled ordinal trees) *Let T be an ordinal tree on n nodes whose nodes are labeled with symbols from an alphabet Σ of size σ. For every $\alpha \in \Sigma$ and every node $v \in T$, a representation of the labeled tree should support the following operations.*

- $\texttt{parent}_\alpha(v)$, *the closest α-labeled ancestor of v (undefined when no α-ancestor exists).*
- $\texttt{depth}_\alpha(v)$, *the number of α-labeled nodes on the path from v to the root, counting v itself when its label is α.*
- $\texttt{child\_select}_\alpha(v, i)$, *the i-th α-labeled child of v in left-to-right order.*
- $\texttt{nbdesc}_\alpha(v)$, *the number of α-labeled nodes in the subtree rooted at v.*
- $\texttt{level\_anc}_\alpha(v, d)$, *the α-labeled ancestor w of v with $\texttt{depth}_\alpha(v) - \texttt{depth}_\alpha(w) = d$.*

When $\sigma = 1$, every node is an α-node for the unique label, so these operations recover the corresponding unlabeled navigation queries, with $\texttt{depth}_\alpha(v) = \texttt{depth}(v) + 1$ under the inclusive convention above.

Two routes from Section 3.3 to a labeled representation are immediate but inadequate. The first builds, for every $\alpha \in \Sigma$, a separate 0/1-labeled index of T in which the α-nodes are marked with 1 and all other nodes with 0. This gives constant-time access to the α-filtered queries on each copy, but it stores σ indexed trees on the same n nodes, for $\Theta(\sigma n)$ bits even before lower-order terms. The second route stores one unlabeled structure for T and keeps the labels in an explicit array of $\lceil \lg \sigma \rceil$ bits per node. This uses $n \lg \sigma + 2n + o(n)$ bits, but without an index over the labels a query such as $\texttt{level\_anc}_\alpha$ or $\texttt{nbdesc}_\alpha$ may scan a path or subtree of size $\Theta(n)$. Neither route gives $n \lg \sigma + O(n)$ bits together with sublogarithmic query time.

The information-theoretic lower bound for an n-node ordinal tree whose nodes are labeled with symbols from an alphabet of size σ is $n(\lg \sigma + 2) - O(\lg n)$ bits, by combining the unlabeled bound of Theorem 3.1.1 with the $\lg(\sigma^n) = n \lg \sigma$ bits required to store one label per node. A representation that meets this lower bound to within a low-order term, while supporting every α-operation of Definition 3.4.1 in $O(1)$ time, would close the labeled question in the same sense as Theorem 3.3.2 closed the unlabeled one. Geary, Raman, and Raman achieve $O(1)$ time within a controlled additive overhead, and their bound is the technical centerpiece of the section.

Theorem 3.4.1 (Labeled ordinal trees [40, 42]) *An ordinal tree on n nodes whose nodes carry labels from an alphabet Σ of size σ admits a representation in*

$$n(\lg \sigma + 2) \; + \; O\left(\frac{\sigma\, n \, \lg\lg\lg n}{\lg\lg n}\right)$$

bits that supports the α-operations of Definition 3.4.1, together with the labeled counterparts of the preorder and postorder rank *and* select, *in $O(1)$ time on the word RAM under the logarithmic word-size convention.*

Sketch. The construction modifies Theorem 3.3.2 to integrate the labels into the catalog of microtree shapes. Reduce the second-level parameter to $M' = \max(\lceil \lg n/(24 \lg \sigma) \rceil, 2)$ so that, whenever a microtree is answered by local table lookup, its labels fit alongside its tree structure within $\lceil |\mu| \lg \sigma \rceil + 2|\mu|$ bits while keeping the lookup table sublinear. For microtree-local queries that are handled by explicit label filters, store σ extra bitvectors per microtree, the α-th of which marks the nodes labeled α. At the canonical root of each microtree, store further bitvectors encoding the distribution of children by label and the per-label skip pointers needed to handle $\texttt{parent}_\alpha$ and $\texttt{level\_anc}_\alpha$ across microtree boundaries. The per-microtree label-aware bookkeeping sums to $o(n)$ bits over all microtree roots once M' is set as above. The macro tree on minitree roots is replicated once per $\alpha \in \Sigma$, with each macro edge weighted by the number of α-labeled nodes in the skipped portion of the original tree. The weights are polylogarithmic in n, so the polylog-weight level-ancestor algorithm answers $\texttt{level\_anc}_\alpha$ on each copy in $O(1)$ time. The σ copies of the macro tree contribute the $O(\sigma n \lg\lg\lg n/\lg\lg n)$ additive overhead of the bound. The remaining α-operations reduce to a constant number of in-microtree lookups, rank/select calls on the per-label microtree bitvectors, and a single macro-level query. □

Theorem 3.4.1 settles the labeled question for very small alphabets. When $\sigma = o(\lg\lg n)$, the displayed overhead is lower order than $n(\lg \sigma + 2)$, so the representation matches the lower bound to within a low-order correction. Outside that regime, the same upper bound no longer shows how to stay near the information-theoretic cost. If σ is polynomial in n, the displayed additive term can be much larger than the label entropy term $n \lg \sigma$, so the bound no longer explains how to store large-alphabet labels near their information-theoretic cost.

The threshold $\sigma = o(\lg\lg n)$ is what makes the additive overhead lower order in Theorem 3.4.1. For polynomial alphabets, the displayed overhead grows far beyond $n \lg \sigma$, so the stated bound ceases to be informative.

The set-collection applications that motivate the later chapters use labels drawn from the universe, so the alphabet can be much larger than the small range covered by Theorem 3.4.1. Geary, Raman, and Raman leave open whether labeled ordinal trees can remain space-efficient for larger alphabets. He, Munro, and Zhou [42] answer this question with a representation that occupies $nH_0(PLS_T) + O(n)$ bits, where $H_0(PLS_T)$ is the zeroth-order empirical entropy of the preorder label sequence of T and is bounded above by $\lg \sigma$, while supporting the α-operations of Definition 3.4.1 in $O(1)$ time when $\sigma = O(\text{polylog}(n))$ and in $O(\lg\lg \sigma)$ time for general Σ. The remaining sections of this chapter develop the answer.

[42]: He et al. (2014), *A framework for succinct labeled ordinal trees over large alphabets*

3.5 Tree extraction

The representation we seek must map every α-operation on a labeled ordinal tree T to a small number of operations on auxiliary structures whose size depends on n and on the entropy of the labels, not on σ and n jointly. A primitive that isolates, for each label α, the part of T relevant to that label, and does so without storing a separate tree per label, is the technical device that realises the reduction. Tree extraction is that primitive. Given a subset X of the nodes of T containing the root, the X-extraction of T is a new ordinal tree T_X on exactly the nodes of X, whose shape encodes the ancestor, order, and depth relations among those nodes inherited from T.

Definition 3.5.1 (X-extraction) *Let T be an ordinal tree on n nodes and let $X \subseteq V(T)$ be a subset containing the root. The X-extraction of T is the ordinal tree T_X obtained from T by deleting, one by one in level order, every node $v \notin X$. When v is deleted, the children of v are inserted into the list of children of the parent of v, in place of v, preserving their original left-to-right order among themselves. The deletion leaves a tree on exactly the nodes of X, and the assignment $v \mapsto v$ is a natural bijection between $V(T) \cap X$ and $V(T_X)$.*

The deletion rule above is the delete operation of tree edit distance used by Tai [43]. It deletes a non-root node together with its edge to the parent, and the subtree below the deleted node slides into the vacated slot. Bille's survey records the equivalent rule used here, where the children of the deleted node enter the parent's child list as a contiguous left-to-right subsequence [44]. He, Munro, and Zhou use the same deletion rule as a data-structure primitive on ordinal trees, state the preservation results for X-extraction, reduce path queries to depth queries on extracted forests, and give the forest form and proofs used below [45, 46]. Figure 3.5 shows the operation on a concrete sixteen-node tree.

Tai's delete operation removes a node together with its edge to the parent and re-attaches the orphaned children at the same slot. Bille's survey writes the rule in the form we use here, promoting the children as a left-to-right subsequence. He, Munro, and Zhou recast it as a data-structure primitive on ordinal trees.

[43]: Tai (1979), *The tree-to-tree correction problem*

[44]: Bille (2005), *A survey on tree edit distance and related problems*

[45]: He et al. (2011), *Path queries in weighted trees*

[46]: He et al. (2016), *Data structures for path queries*

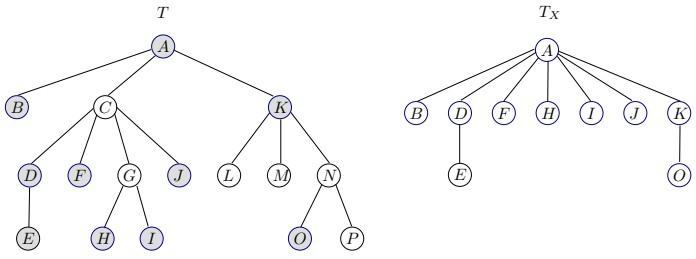

Figure 3.5: X-extraction on a sixteen-node ordinal tree, reproduced from Gagie, He, and Navarro [9]. On the left, T has $X = \{A, B, D, E, F, H, I, J, K, O\}$ as the shaded nodes. On the right, the extracted tree T_X contains exactly those nodes. The deletion of each unshaded node promotes its children into the sibling list of its parent, and the natural bijection between X and $V(T_X)$ sends every shaded node to the position it occupies on the right.

We now record the three preservation properties of T_X that justify calling it a structural projection of T onto the nodes of X. The first says that ancestry transfers between T and T_X. The second says that preorder and postorder transfer. The third relates the depth of the nearest retained ancestor in the extracted tree to the number of retained nodes on the original root path.

He, Munro, and Zhou define forest extraction by deleting non-X nodes bottom to top and observe that the result is independent of deletion order [46]. Their labeled-tree framework uses level order for the tree case [42].

Lemma 3.5.1 (Ancestry preservation [46]) *Let T be an ordinal tree and let $X \subseteq V(T)$ contain the root of T. For every pair $u, v \in X$, the node u is an ancestor of v in T if and only if u is an ancestor of v in T_X.*

Proof. Fix a level-order enumeration $z_1, z_2, \ldots, z_k$ of the nodes of T not in X, and write $T^{(0)} = T$ and $T^{(i)} = T^{(i-1)}$ with z_i deleted, so that $T^{(k)} = T_X$. We show, by induction on i, that ancestry among the nodes of X is the same in $T^{(i)}$ as in T. The claim is immediate for $i = 0$.

Assume the claim for $i - 1$ and consider the single deletion of $z = z_i$ in $T^{(i-1)}$. Let p be the parent of z in $T^{(i-1)}$ and let $c_1, \ldots, c_d$ be the children of z in $T^{(i-1)}$, in left-to-right order. After the deletion, p acquires $c_1, \ldots, c_d$ as consecutive children in place of z, and every other parent-child edge is untouched. Take $u, v \in X$ with u an ancestor of v in $T^{(i-1)}$. The unique path from v up to u in $T^{(i-1)}$ either avoids z, in which case the path is intact in $T^{(i)}$ and u remains an ancestor of v, or passes through z, in which case the sub-path of the form $v \to \cdots \to c_j \to z \to p \to \cdots \to u$ becomes $v \to \cdots \to c_j \to p \to \cdots \to u$ in $T^{(i)}$, a path in $T^{(i)}$ that still makes u an ancestor of v. Conversely, if u is an ancestor of v in $T^{(i)}$ with $u, v \in X$, the promotion rule is the only way a new edge is created, and it merely splices p between z's old position and z's children. Walking backwards along the ancestry path in $T^{(i)}$ and reinserting z whenever an edge of the form $p \to c_j$ was produced by the deletion reconstructs a path in $T^{(i-1)}$ witnessing the same ancestry. Both directions extend the claim from $i - 1$ to i, and the induction closes at $i = k$. □

Lemma 3.5.2 (Traversal-order preservation [42, 46]) *Let T be an ordinal tree and let $X \subseteq V(T)$ contain the root. For every pair $u, v \in X$, u precedes v in the preorder traversal of T if and only if u precedes v in the preorder traversal of T_X. The same equivalence holds for postorder.*

Proof. It is enough to inspect one deletion. Let z be a non-root node with parent p and children $c_1, \ldots, c_d$ in left-to-right order. In the preorder traversal before the deletion, the block contributed by the subtree of z is z followed by the preorder blocks of the subtrees rooted at $c_1, \ldots, c_d$. After the deletion, the same child blocks occupy the slot formerly occupied by z, in the same left-to-right order, and the only removed entry is z. All traversal entries outside the subtree of z keep their relative order. Thus the preorder sequence of the surviving nodes is the old preorder sequence with z removed.

In postorder, the same local block consists of the postorder blocks of the subtrees rooted at $c_1, \ldots, c_d$, followed by z. After the deletion, those child blocks again occupy the same slot and the same order, and the only removed entry is z. Therefore the postorder sequence of the surviving nodes is the old postorder sequence with z removed. Iterating over the deletion sequence gives both traversal equivalences for the final tree T_X. □

Lemma 3.5.3 (Depth identity [46]) *Let T be an ordinal tree and let $X \subseteq V(T)$ contain the root, under the convention that a node is its own 0-th ancestor. For every $v \in V(T)$, let $w = \mathrm{anc}_X(T, v)$ be the lowest node of X on the root-to-v path, and let w_X be the image of w under the natural bijection between X and $V(T_X)$. Write $\mathrm{dep}^+_X(T, v)$ for the number of nodes of X on the root-to-w path, including w. Then*

$$\mathrm{dep}_{T_X}(w_X) + 1 \;=\; \mathrm{dep}^+_X(T, v).$$

Equivalently, if a dummy root is added above T_X, the depth of w_X in the dummy-rooted tree is exactly $\mathrm{dep}^+_X(T, v)$.

Proof. Since w is the lowest node of X on the root-to-v path, the nodes counted by $\mathrm{dep}^+_X(T, v)$ are exactly w together with the nodes of X that are strict ancestors of w in T. By Lemma 3.5.1, a node $x \in X$ is a strict ancestor of w in T if and only if its image x_X is a strict ancestor of w_X in T_X. Thus the strict ancestors of w_X in T_X are precisely the images of the strict X-ancestors of w in T. The number of such strict ancestors is $\mathrm{dep}_{T_X}(w_X)$ under the zero-based depth convention of Section 3.1. Adding the node w itself gives $\mathrm{dep}_{T_X}(w_X) + 1 = \mathrm{dep}^+_X(T, v)$. □

Lemma 3.5.1, Lemma 3.5.2, and Lemma 3.5.3 together make T_X a structural projection of T onto X. Ancestry is preserved, so any navigational query on T that only consults ancestor or descendant relations among X-nodes can be answered on T_X. Preorder and postorder are preserved, so subtree-rank and subtree-select primitives on X-nodes transfer. The depth identity says that, once the extraction is viewed below a dummy root, the depth of the image of the nearest retained ancestor counts the retained nodes on the original root path up to that ancestor. This is the form the binary partition of Section 3.7 exploits to reduce path counting to depth queries on extracted forests. These three properties recur as building blocks in the two representations developed in the remainder of this chapter.

3.6 α-operations on labeled trees

The three preservation lemmas of Section 3.5 make the extracted tree T_X a structural projection of the ordinal tree T onto the subset $X \subseteq V(T)$. Ancestry transfers in both directions, preorder and postorder transfer, and the depth identity counts retained ancestors after the extraction is placed below a dummy root. We now put this projection to work on the α-operations problem that Section 3.4 left open, where the bound of Theorem 3.4.1 does not explain how to keep large alphabets near their information-theoretic cost.

Definition 3.6.1 (α-operations on the ancestor axis) *Let T be an ordinal tree on n nodes, each node carrying a label from $[1..u]$. For a node v of T, a label $\alpha \in [1..u]$, and an integer $i \geq 1$, we define three α-operations on the path from v to the root:*

- $\texttt{parent}_\alpha(v)$*: the lowest ancestor of v whose label is α, or $\perp$ if no ancestor of v carries label α.*
- $\mathrm{rank}_\alpha(v)$*: the number of ancestors of v whose label is α, with the convention that the root counts as an ancestor of v when v is the root.*
- $\mathrm{select}_\alpha(v, i)$*: the i-th ancestor of v with label α, counted from the top down along the root-to-v path, or $\perp$ if fewer than i such ancestors exist.*

For every label $\alpha \in [1..u]$, we build an extracted tree T_α that retains only the nodes of T whose behaviour is relevant to α. Let X_α consist of a fresh root r_α, every node of T whose label is α, and every parent of such a node, where r_α is attached as the sole parent of the original root of T in the augmented tree. The X_α-extraction of the augmented tree, as given

He, Munro, and Zhou [42] list a larger family of α-operations, including $\texttt{depth}_\alpha$, $\texttt{level\_anc}_\alpha$, and descendant-axis queries. We restrict the exposition to the three ancestor-axis operations required by the set-collection constructions of later chapters; the descendant-axis operations reduce to the same machinery.

by Definition 3.5.1, is the desired T_α. Each node of T_α carries a single bit of label information, namely the marker 1 if its corresponding node in T is an α-node, and the marker 0 otherwise; in particular the root r_α of T_α always carries marker 0. The combined size of the family $\{T_\alpha\}_{\alpha\in[1..u]}$ is linear in n, since each α-node of T contributes one 1-marked node and at most one further 0-marked parent to T_α, while the fresh root r_α adds a single 0-marked node, so $|T_\alpha| \le 2n_\alpha + 1$ and hence $\sum_{\alpha\in[1..u]} |T_\alpha| \le 2n + u$, where n_α counts the α-nodes of T. By Lemma 3.5.1 the α-ancestors of a node $v \in V(T)$ correspond bijectively, through the natural injection of $V(T) \cap X_\alpha$ into $V(T_\alpha)$, to the 1-marked ancestors of the image $v' \in V(T_\alpha)$ of the nearest X_α-node on or above v. The three α-operations on T reduce to unlabeled ancestor operations on T_α restricted to marker 1.

Storing one tree per label is wasteful in constants. We concatenate the family into a single ordinal tree $\mathcal{T}$ with a fresh root $\mathcal{R}$, whose children are the roots of $T_1, T_2, \ldots, T_u$ in that order. The merged tree $\mathcal{T}$ has at most $2n + u$ nodes, and the preorder traversal of each T_α occurs as a contiguous substring of the preorder traversal of $\mathcal{T}$; the same holds for the DFUDS traversal once the new root r_α of each T_α is removed. In parallel we concatenate the leaf-indicator bitvectors $L_\alpha[1..n_\alpha]$, whose i-th bit is 1 iff the i-th 1-marked node in preorder of T_α corresponds to a leaf of T, into a single bitvector $\mathcal{L}$ of length n, ordered by $\alpha = 1, 2, \ldots, u$. In the representation we build below, $\mathcal{T}$ plays the role of a single 0/1-labeled ordinal tree on which we support α-operations in the marker alphabet $\{0, 1\}$, as Figure 3.6 makes concrete on a five-node example.

The representation rests on a decomposition of the workload across four building blocks, in the form given by He, Munro, and Zhou for general labeled ordinal trees.

Lemma 3.6.1 (Framework decomposition [42]) *Let T be an ordinal tree on n nodes with labels drawn from $[1..u]$, let $PLS_T[1..n]$ be the preorder label sequence of T, and let $\mathcal{T}$ and $\mathcal{L}$ be the merged tree and merged leaf-indicator bitvector built above. Suppose that four data structures are available:*

- $\mathcal{D}_1$**:** *an unlabeled ordinal tree on the shape of T, occupying $\mathcal{S}_1(n)$ bits and supporting the standard unlabeled navigational operations on T in constant time;*
- $\mathcal{D}_2$**:** *a representation of PLS_T as a string over $[1..u]$, occupying $\mathcal{S}_2(PLS_T)$ bits and supporting* rank_α *and* select_α *for every $\alpha \in [1..u]$;*
- $\mathcal{D}_3$**:** *a 0/1-labeled ordinal tree on $\mathcal{T}$, occupying $\mathcal{S}_3(2n + u)$ bits and supporting the α-operations in the marker alphabet $\{0, 1\}$ together with the unlabeled navigational operations in constant time;*
- $\mathcal{D}_4$**:** *the bitvector $\mathcal{L}$ of length n, occupying $\mathcal{S}_4(n)$ bits and supporting* rank_b *and* select_b *for $b \in \{0, 1\}$ in constant time.*

The four structures together represent T in $\mathcal{S}_1(n)+\mathcal{S}_2(PLS_T)+\mathcal{S}_3(2n+u)+\mathcal{S}_4(n)$ bits and support every α-operation of Definition 3.6.1 with a constant number of calls to the operations of $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3, \mathcal{D}_4$.

Proof. We instantiate each $\mathcal{D}_i$, then describe the navigation from T to T_α, and finally reduce each operation of Definition 3.6.1 to a constant number of calls to the four blocks. The unlabeled navigational operations used in

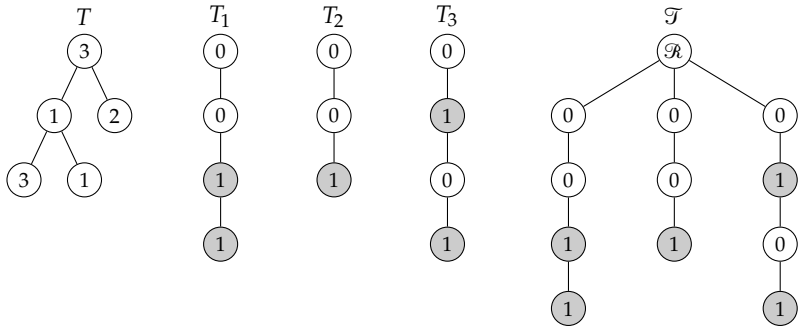

Figure 3.6: From left to right, an ordinal tree T on five nodes with labels in $\{1, 2, 3\}$; the three extracted trees T_1, T_2, T_3 built on the X_α-extractions described in the text, with 0/1 markers drawn as empty and shaded circles; and the merged tree $\mathcal{T}$ obtained by appending the T_α as subtrees of a fresh root $\mathcal{R}$. The α-operations on T are answered by the 0/1 α-operations on the single tree $\mathcal{T}$, which is the role of the building block $\mathcal{D}_3$ of Lemma 3.6.1.

$\mathcal{D}_1$ and $\mathcal{D}_3$ include parent, depth, level-ancestor, and lowest-common-ancestor; all of them are supported in constant time by the tree-covering representation of Theorem 3.3.2.

We realise $\mathcal{D}_1$ on the shape of T via Theorem 3.3.2, which gives $2n + o(n)$ bits with constant-time unlabeled navigational operations. For $\mathcal{D}_2$ we store PLS_T as a string of length n over $[1..u]$ using the sequence representation of Belazzougui and Navarro [47], which takes $nH_0(PLS_T) + o(nH_0(PLS_T)) + o(n) = O(n \lg u)$ bits and supports rank_α and select_α in $O(\lg \lg_\omega u)$ time under the word RAM with $\omega = \Omega(\lg n)$. This representation builds on the succinct indexes of Barbay, He, Munro, and Srinivasa Rao [48] and the large-alphabet string rank/select of Golynski, Munro, and Rao [49], improving their $O(\lg \lg u)$ time bound to the word-RAM-aware $O(\lg \lg_\omega u)$. For $\mathcal{D}_3$ we apply the binary-alphabet instance of the framework of He, Munro, and Zhou [42] to the merged tree $\mathcal{T}$. In this instance the marker alphabet is $\{0, 1\}$, so the structure occupies $O(|\mathcal{T}|) = O(n + u)$ bits and supports the marker-restricted α-operations needed below in constant time. The recursion on the framework is not circular because the binary-alphabet instance is proved directly, where the string index at $\mathcal{D}_2$ reduces to a bitvector and the time blows down to $O(1)$. For $\mathcal{D}_4$ we store $\mathcal{L}$ by the bitvector of Clark and Munro [50], which gives $n + o(n)$ bits with constant-time rank_b and select_b. Summing, the representation occupies $(2n + o(n)) + O(n \lg u) + O(n + u) + (n + o(n)) = O(n \lg u)$ bits, equivalently $O(n)$ words.

[47]: Belazzougui et al. (2015), *Optimal lower and upper bounds for representing sequences*

[48]: Barbay et al. (2011), *Succinct indexes for strings, binary relations and multi-labeled trees*

[49]: Golynski et al. (2006), *Rank/select operations on large alphabets: a tool for text indexing*

[42]: He et al. (2014), *A framework for succinct labeled ordinal trees over large alphabets*

[50]: Clark et al. (1996), *Efficient suffix trees on secondary storage (extended abstract)*

For a node $x \in V(T)$ and a label $\alpha \in [1..u]$ we define a partial map $\phi_\alpha(x)$ that returns the image $x' \in V(T_\alpha)$ when $x \in X_\alpha$ and is undefined otherwise. If x has label α, then $x \in X_\alpha$. The image x' is the marker-1 node of T_α whose marker-1 preorder rank equals the number of α-nodes up to x in the preorder label sequence of T. This takes a rank_α call on $\mathcal{D}_2$ and a marker-1 preorder selection on $\mathcal{D}_3$. If x is not an α-node but has at least one α-descendant, $x \in X_\alpha$ as the parent of some α-node. Let p and q be the first and last α-descendants of x in preorder, computed by rank_α and select_α on $\mathcal{D}_2$, and let p' and q' be their images in T_α. Then x' is the lowest common ancestor of p' and q' in T_α, because x is exactly the lowest retained node that contains both extreme α-descendants. If x has no α-descendant, $\phi_\alpha(x)$ is undefined, and the existence test costs one rank_α query on $\mathcal{D}_2$. The conversion runs in $O(\lg \lg_\omega u)$ time, dominated by the rank_α and select_α calls on $\mathcal{D}_2$.

If v has label α, then $\phi_\alpha(v) = v'$ is defined and $\texttt{parent}_\alpha(v)$ is the image in T of $\texttt{parent}_1(T_\alpha, v')$, recovered by one select_α on PLS_T. If v is not an α-node, let u be the α-predecessor of v in PLS_T, computed by one rank_α followed by one select_α on $\mathcal{D}_2$. If no such u exists, v has no α-ancestor

and we return $\perp$. Otherwise let $w_{\text{anc}} = \texttt{LCA}(u, v)$ on $\mathcal{D}_1$. If w_{anc} is an α-node, it is the answer. Otherwise w_{anc} has at least one α-descendant, so $\phi_\alpha(w) = w'$ is defined for the first α-descendant w of w_{anc} in preorder, and we return the image in T of $\texttt{parent}_1(T_\alpha, w')$ via one select_α on PLS_T. Every α-ancestor of v contains v in its subtree by definition and contains u in its subtree because u is an α-position with preorder no larger than that of v, so every α-ancestor of v is also an ancestor of u and therefore lies on the path from w_{anc} to the root. The path from w_{anc} down to v contains no α-node, since such a node would be an α-position later than u and no later than v, contradicting maximality of u.

Let y be v if v has label α, and $\texttt{parent}_\alpha(v)$ otherwise. If $y = \perp$, then $\text{rank}_\alpha(v) = 0$ and $\text{select}_\alpha(v, i) = \perp$ for every $i \geq 1$. Otherwise $y \in X_\alpha$ and $y' = \phi_\alpha(y)$ is defined, and $\text{rank}_\alpha(v)$ equals the marker-1 depth of y' in T_α, computed in $O(1)$ on $\mathcal{D}_3$, by the bijection of Lemma 3.5.1 between α-ancestors of y in T and 1-marked ancestors of y' in T_α. For $\text{select}_\alpha(v, i)$, if $i > \text{rank}_\alpha(v)$ we return $\perp$. Otherwise the same bijection sends the i-th α-ancestor of v from the top to the marker-1 ancestor of y' with marker rank i, computed in $O(1)$ on $\mathcal{D}_3$, whose image in T is recovered by one select_α on PLS_T.

Each α-operation costs a constant number of calls to $\mathcal{D}_1$, $\mathcal{D}_3$, $\mathcal{D}_4$, each of which runs in $O(1)$ time by the constant-time specifications above, and a constant number of calls to $\mathcal{D}_2$, each of which runs in $O(\lg \lg_\omega u)$ time. The bottleneck is $\mathcal{D}_2$, so the total cost is $O(\lg \lg_\omega u)$. □

Theorem 3.6.2 (α-operations via tree extraction [9, 42]) *Let T be an ordinal tree on n nodes, each carrying a label from $[1..u]$, and assume the word RAM model with word size $\omega = \Omega(\lg n)$. There exists a representation of T in $O(n)$ words of space that supports $\texttt{parent}_\alpha(v)$, $\text{rank}_\alpha(v)$, and $\text{select}_\alpha(v, i)$ of Definition 3.6.1 in $O(\lg \lg_\omega u)$ time, and that can be built in $O(n \lg u)$ time.*

Proof. Lemma 3.6.1 gives the decomposition; the four-phase argument of its proof instantiates the building blocks and performs the reduction, yielding the stated space and time bounds. □

The representation of Theorem 3.6.2 answers the ancestor-axis α-operations in time sublogarithmic in the alphabet size and in space proportional to n words, which is the precision the downstream constructions of Cap. 6 require for membership in labeled deletion trees. Path queries that aggregate over ranges of labels rather than a single label are the subject of the next section.

3.7 Path queries

He, Munro, and Zhou [46] also handle node-to-node paths and path-median queries. The set-collection constructions of later chapters only need the root-to-node case, which already admits the sharper time bounds of the present section.

The α-operations of Definition 3.6.1 aggregate information along the root-to-v path when the query is pinned to a single label. Many downstream uses of labeled trees ask a broader question. How many ancestors of v carry a label in a query range $[p..q]$, and which ancestor carries the i-th smallest label. We call these *path queries*, and we restrict attention to the root-to-node version, which is what the set-collection constructions of later chapters need. The construction develops in two steps, each with its own query time bound, and this section covers the first of them.

3.7.1 Binary partition

The closing of Section 2.3 promised that the hierarchical binary partition of the alphabet, which organises rank and select on a sequence through a chain of bitmaps, would transfer from sequences to labeled ordinal trees. We cash the promise in now. The wavelet tree of Grossi, Gupta, and Vitter [31] splits the alphabet $[1..u]$ at each recursive level by a midpoint, stores at every range a bitmap that records, for each symbol of the underlying subsequence, whether it falls in the left or right half of the range, and answers rank_c and select_c through a root-to-leaf traversal whose per-level cost is a single binary rank or select. We replicate this plan on a labeled ordinal tree T on n nodes with labels drawn from $[1..u]$. The alphabet is partitioned by the same range tree, and at every range $[a..b]$ we store a 0/1-labeled extracted tree $T_{a,b}$ whose 0 markers flag the nodes with labels in the left half and whose 1 markers flag those with labels in the right half. Level transitions move between extracted trees through a single rank or select on the marker sequence, in constant time, and the three preservation lemmas of Section 3.5 guarantee that depths and ancestries transfer across the transitions in the form the queries need.

[31]: Grossi et al. (2003), *High-order entropy-compressed text indexes*

Definition 3.7.1 (Path queries) *Let T be an ordinal tree on n nodes with labels in $[1..u]$, and let $v \in V(T)$. For a range $[p..q] \subseteq [1..u]$ and an integer $i \geq 1$, we define two operations on the root-to-v path of T:*

- `path_count`$(v, [p..q])$*: the number of ancestors of v, counted with multiplicity by label, whose label belongs to $[p..q]$.*
- `path_select`(v, i)*: the ancestor of v whose label is the i-th smallest among the labels appearing on the root-to-v path. Return $\perp$ if i exceeds the length of the path.*

Under the convention that a node is its own 0-th ancestor, the root-to-v path includes v itself.

The alphabet is organised by a *conceptual range tree* on $[1..u]$. The root range is $[1..u]$. A non-leaf range $[a..b]$ with $a < b$ has two child ranges $[a..m]$ and $[m+1..b]$, where $m = \lfloor (a+b)/2 \rfloor$. The recursion stops at the u leaf ranges of the form $[\alpha..\alpha]$, one for every value $\alpha \in [1..u]$. The conceptual range tree has height $\lceil \lg u \rceil$ and is balanced up to a unit imbalance in sibling lengths. No material storage is devoted to the range tree itself; each range is an abstract index under which we store the labeled tree associated with it.

For each range $[a..b]$ in the conceptual range tree we store an extracted ordinal tree $T_{a,b}$. Let $R_{a,b}$ denote the set of nodes of T whose label lies in $[a..b]$, and let $F_{a,b}$ be the $R_{a,b}$-extraction of T in the sense of Definition 3.5.1, where $R_{a,b}$ is augmented by the root of T to meet the root-inclusion requirement of the definition. The object $T_{a,b}$ is $F_{a,b}$ with a dummy root $\rho_{a,b}$ attached above, following He, Munro, and Zhou [46], so that $T_{a,b}$ is a genuine ordinal tree rather than a forest and its nondummy nodes correspond bijectively to the nodes of $R_{a,b}$. At the top of the recursion, $T_{1,u}$ contains every node of T under the dummy root, and its preservation properties reduce to those of T itself. At every nonleaf range $[a..b]$ with children $[a..m]$ and $[m+1..b]$, each nondummy node of $T_{a,b}$ carries a single marker bit. The marker is 0 if the label of the corresponding node of T lies in $[a..m]$ and 1 if it lies in $[m+1..b]$. The child trees $T_{a,m}$ and

[46]: He et al. (2016), *Data structures for path queries*

$T_{m+1,b}$ are, up to a relabelling of the dummy root, the extractions of $T_{a,b}$ onto its marker-0 and marker-1 nondummy nodes, respectively, under the root-extraction convention that the dummy root is preserved. At a leaf range $[\alpha..\alpha]$, $T_{\alpha,\alpha}$ collects the nodes of T whose label equals α. The full hierarchy of extracted trees, displayed in Figure 3.7, makes the level-by-level transition visible.

Each $T_{a,b}$ is accessed through three navigation primitives on every node, namely the depth of the node within $T_{a,b}$, a link back to the corresponding node of T, and a pair of links leading to the corresponding images in the two children $T_{a,m}$ and $T_{m+1,b}$. In the succinct encoding each primitive is realised as a marker-0 or marker-1 α-operation of Definition 3.6.1 applied to $T_{a,b}$, delivered in $O(1)$ time on the pointer machine by Theorem 3.6.2 at $\sigma = 2$. The abstract rule that justifies the navigation is the preservation machinery of Section 3.5. By Lemma 3.5.1 the image v' in $T_{a,m}$ of a node $v \in V(T_{a,b})$ whose label falls in $[a..m]$ is the natural bijective image of v through the X_0-extraction; if the label of v falls in $[m+1..b]$ instead, v' is the lowest marker-0 ancestor of v in $T_{a,b}$, and the same formula, with marker 1, produces the image in $T_{m+1,b}$. The binary alphabet of markers makes every such transition a single $\texttt{parent}_\alpha$ or rank_α call on $T_{a,b}$.

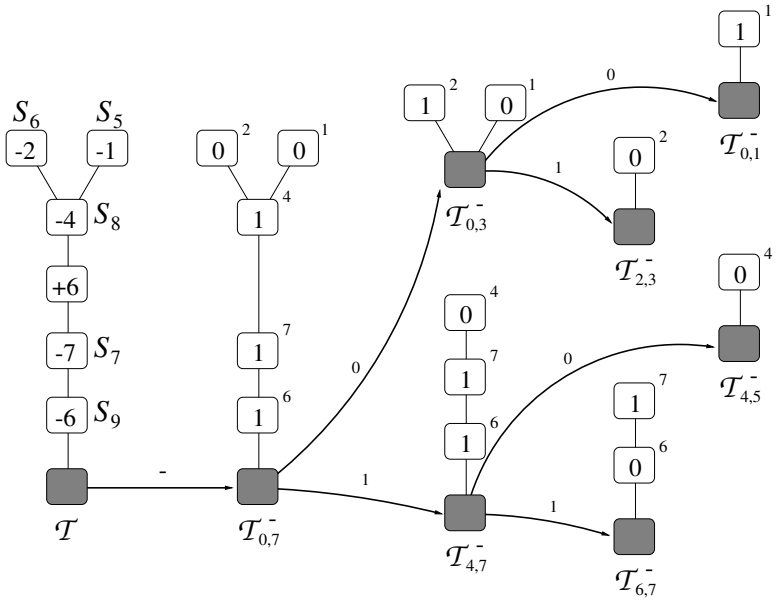

Figure 3.7: Hierarchical binary partition of a labeled ordinal tree for root-to-node path queries, reproduced from Gagie, He, and Navarro [9]. Starting from the root range, each intermediate tree $\mathcal{T}_{a,b}$ is 0/1-labeled by the midpoint of its range. Bold arrows lead to the child trees $\mathcal{T}_{a,m}$ on marker 0 and $\mathcal{T}_{m+1,b}$ on marker 1. The binary alphabet at every level makes each level transition cost $O(1)$ via a single rank or select on the marker sequence.

Theorem 3.7.1 (Path queries on the pointer machine [9, 46]) *Let T be an ordinal tree on n nodes with labels in $[1..u]$. On the pointer machine, T can be represented in $O(n)$ words of space that support the operations $\texttt{path\_count}(v, [p..q])$ and $\texttt{path\_select}(v, i)$ of Definition 3.7.1 in $O(\lg u)$ time, and that can be built in $O(n \lg u)$ time.*

Proof. We instantiate the range tree and the family $\{T_{a,b}\}$ as described above, then analyse space, then describe the two query algorithms, then bound their time.

Every $T_{a,b}$ is a 0/1-labeled ordinal tree on $|R_{a,b}| + 1$ nodes, counting the dummy root, where $R_{a,b}$ is the set of nodes of T with label in $[a..b]$. We encode each $T_{a,b}$ through the labeled-tree primitive of Theorem 3.6.2 specialised to alphabet size $\sigma = 2$, which encodes a σ-labeled ordinal tree on N nodes in $O(N \lg \sigma)$ bits and supports the level-transition queries in

constant time on the pointer machine. At $\sigma = 2$ this reduces to $O(|R_{a,b}|)$ bits per range. The level-by-level accounting is a telescope. At a fixed level ℓ of the conceptual range tree, the ranges partition $[1..u]$ into at most 2^ℓ disjoint sub-ranges, and accordingly partition the labeled nodes of T into disjoint sets, so

$$\sum_{[a..b] \text{ at level } \ell} |R_{a,b}| \;=\; n,$$

and the total size of the trees at level ℓ, including the dummy roots, is $n + O(2^\ell)$. A range of length 1 with no α-labeled node carries only the dummy root and need not be materialised, so we omit empty ranges and every materialised level contributes $O(n)$ bits. Across the $\lceil \lg u \rceil$ levels the partition occupies $O(n \lg u)$ bits in total, which under $\omega \geq \lg n$ is $O(n)$ words. This is the succinct reading of the partition adopted by Gagie, He, and Navarro [9], who treat each $T_{a,b}$ as a bit-level succinct tree to keep the aggregate cost at $O(n)$ words. The original pointer-machine construction of He, Munro, and Zhou [46] materialises each node of $T_{a,b}$ as an $O(1)$-word record holding explicit pointers to T and to the two child trees, which inflates the space to $O(n \lg u)$ words; the word bound stated above matches once one substitutes the bit-level encoding for the pointer record, as we do here.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

Given $v \in V(T)$ and a query range $[p..q] \subseteq [1..u]$, we move from v to its image $v_{1,u}$ in $T_{1,u}$ and recurse on the range tree. The recursion acts on a pair $([a..b], w)$ where $w \in V(T_{a,b})$ is the image of the closest $R_{a,b}$-ancestor of v weakly above v, and returns the number of ancestors of v in T with label in $[a..b] \cap [p..q]$. If $[a..b] \subseteq [p..q]$, every label in $[a..b]$ already qualifies, and the recursion returns $\mathrm{dep}_{T_{a,b}}(w)$ after excluding the dummy root from the count. By Lemma 3.5.3 applied to $X = R_{a,b} \cup \{\text{root of } T\}$ and the node v, this depth equals the number of ancestors of v in T whose label belongs to $[a..b]$. If $[a..b] \cap [p..q] = \emptyset$, no ancestor with label in $[a..b]$ qualifies, and the recursion returns 0. In the remaining case, $[a..b]$ properly overlaps $[p..q]$, so the recursion continues on $([a..m], w_0)$ and $([m+1..b], w_1)$. The nodes w_0 and w_1 are the images of w in $T_{a,m}$ and $T_{m+1,b}$, each obtained from w in $O(1)$ time by a single $\mathtt{parent}_\alpha$ call on $T_{a,b}$ with $\alpha \in \{0, 1\}$ under Theorem 3.6.2. The answer is the sum of the two recursive answers. Correctness is an induction on the range tree. At $[1..u]$ the recursion is invoked with the image of v and the query range $[p..q]$, and the result is the number of ancestors of v in T with label in $[1..u] \cap [p..q] = [p..q]$, which is exactly $\mathtt{path\_count}(v, [p..q])$.

Given v and i, we drive the same recursion on the range tree, now tracking the rank i of the label to locate. The recursion acts on $([a..b], w, j)$ and returns the ancestor of v whose label is the j-th smallest among the ancestors with label in $[a..b]$, or $\perp$ if no such ancestor exists. If $a = b$, the range is a singleton $\{\alpha\}$. When $j \leq \mathrm{dep}_{T_{\alpha,\alpha}}(w)$, one select_α call on $T_{a,b}$ locates the j-th marker-α ancestor of v, and the label bijection maps it back to T. When j is larger than that depth, the requested ancestor does not exist and the recursion returns $\perp$. If $a < b$, let $d_L = \mathrm{dep}_{T_{a,m}}(w_0)$ be the number of ancestors of v with label in $[a..m]$. For $j \leq d_L$ the recursion continues into $([a..m], w_0, j)$, and for $j > d_L$ it continues into $([m+1..b], w_1, j - d_L)$. The recursion is invoked with $([1..u], v_{1,u}, i)$. Correctness is again a range-tree induction. The decision at each nonleaf

range uses the depth identity of Lemma 3.5.3 to count ancestors in $[a..m]$, then subtracts to shift the rank when recursing into the right child.

Path counting touches every range $[a..b]$ on the frontier between ranges contained in $[p..q]$ and ranges disjoint from it. The frontier of a canonical decomposition of a range on a balanced range tree has size $O(\lg u)$, and the recursion spends $O(1)$ time per range through a constant number of α-operations on $T_{a,b}$, so path counting runs in $O(\lg u)$ time. Path selection descends a single root-to-leaf path of the range tree, of length $\lceil \lg u \rceil$, spending $O(1)$ time per range through the same primitive. Both queries run in $O(\lg u)$ time. □

3.7.2 Sublogarithmic refinement

The bound of Theorem 3.7.1 is the tree analogue of the $O(\lg u)$ cost that a wavelet tree pays on sequences, one level of the binary alphabet partition at a time. On sequences, Ferragina, Manzini, Mäkinen, and Navarro [33] refined that cost to $O(\lg u / \lg \lg n)$ under the word RAM, by increasing the branching factor of the partition to $f = \lceil \lg^{\epsilon} n \rceil$ and supporting the resulting f-ary rank and select in constant time per level through packed-word arithmetic. We ask whether the same refinement transfers to labeled ordinal trees, and the answer, due to He, Munro, and Zhou [46], is that it does, subject to two word-RAM primitives on f-labeled sequences that replace binary rank and select.

[33]: Ferragina et al. (2007), *Compressed Representations of Sequences and Full-Text Indexes*

[46]: He et al. (2016), *Data structures for path queries*

We fix a constant $\epsilon \in (0, 1)$ and set the fanout to $f = \lceil \lg^{\epsilon} n \rceil$. The conceptual range tree on $[1..u]$ is built by splitting each nonleaf range into f subranges of as nearly equal length as possible, following He, Munro, and Zhou [46], so that at level ℓ there are at most $f^{\ell-1}$ ranges and the height is $h = \lceil \log_f u \rceil$. A direct computation gives

$$h = \frac{\lg u}{\lg f} = \frac{\lg u}{\epsilon \lg \lg n} = \Theta\left(\frac{\lg u}{\lg \lg n}\right).$$

Path counting calibrates the fanout to $f = \lceil \lg^{\epsilon} n \rceil$, giving height $\Theta(\lg u / \lg \lg n) = \Theta(\lg_{\omega} u)$. Path selection will instead use $f' = \lceil \lg^{\epsilon'} u \rceil$, giving height $\Theta(\lg u / \lg \lg u)$. The two denominators agree only when u is polynomial in n; outside that regime the two queries genuinely separate.

[46]: He et al. (2016), *Data structures for path queries*

Under the word RAM with $\omega = \Omega(\lg n)$, the quantity $\lg u / \lg \lg n$ is $\Theta(\lg_{\omega} u)$ when $\omega = \Theta(\lg n)$, so $h = \Theta(\lg_{\omega} u)$ and the binary-partition height of Theorem 3.7.1 drops from $\lceil \lg u \rceil$ to $\lceil \log_f u \rceil$. The saving is paid for at every level, where the alphabet of markers is now $[1..f]$. The level transition of Theorem 3.7.1, which cost $O(1)$ via a single rank on a bitvector, has to be replaced by a constant-time operation on an f-labeled sequence.

The first primitive counts, in a sequence over a small integer alphabet, how many entries up to a given position fall below a threshold.

Lemma 3.7.2 (Counting below a threshold [51]) *Let $S[1..n]$ be a sequence over the alphabet $[1..f]$ with $f = O(\lg^{\epsilon} n)$ for some constant $\epsilon \in (0, 1)$. Assume that S is stored in a representation from which any $\lceil \lg^{\lambda} n \rceil$ consecutive entries are accessible in $O(1)$ time, for some constant $\lambda \in (\epsilon, 1)$. There exists an auxiliary structure of $o(n)$ bits that supports, in $O(1)$ time, the query*

$$\mathtt{count}_{\beta}(S, i) = |\{\, j \le i : S[j] \le \beta \,\}|$$

for every $\beta \in [1..f]$ and every $i \in [1..n]$.

Sketch. Partition S into blocks of $\lceil \lg^2 n \rceil$ entries and further into subblocks of $\lceil \lg^\lambda n \rceil$ entries. Store a two-dimensional array $A[1..n/\lceil \lg^2 n \rceil, 1..f]$ that holds, for each block, the count of entries not exceeding β up to the end of the block, and an analogous array B that holds the same counts within each block up to the end of the subblock. A global lookup table C records, for every possible subblock pattern, every prefix length inside the subblock, and every threshold β, the count of entries of the pattern up to that prefix that are at most β. A query at position i decomposes i as a block index plus a subblock index plus a subblock offset and reads one entry from each of A, B, C in constant time. The space cost of A and B is $o(n)$ bits because $f = O(\lg^\epsilon n)$ keeps the second dimension subpolynomial, and the table C is subpolynomial in n because each subblock carries only $O(\lg^{\lambda+\epsilon} n)$ bits of information. □

The second primitive counts, at any position, how many earlier entries carry the same value as the entry at that position.

Lemma 3.7.3 (Constant-time partial rank [52]) *Let $S[1..n]$ be a sequence over the alphabet $[1..f]$. There is a representation of S in $O(n \lg f)$ bits supporting, in $O(1)$ time, the query*

$$\texttt{partial\_rank}(S, i) \;=\; |\{\, j \le i : S[j] = S[i] \,\}|.$$

Sketch. The representation is a standard rank-indexed succinct encoding of the sequence, equivalent to the rank structure used as a substructure in ball-inheritance for 2-d range reporting in the word RAM. For every position i, the value $\texttt{partial\_rank}(S, i)$ equals the rank of $S[i]$ up to i among all entries equal to $S[i]$, and since the partial rank only counts the symbol already at i it can be stored implicitly, using $O(\lg f)$ bits per position in a packed layout, and answered by one table lookup in constant time. □

At each level $\ell \in \{1, 2, \ldots, h\}$ of the f-ary range tree we merge the family $\{T_{a,b}\}$ at that level into a single labeled ordinal tree T_ℓ, following the construction of He, Munro, and Zhou [46]. The ranges at level ℓ partition the n weighted nodes of T into disjoint groups, so the forests $\{F_{a,b}\}$ at level ℓ are node-disjoint; collecting them under a common dummy root in the order imposed by the ranges produces T_ℓ, whose n nondummy nodes (alongside the dummy root) correspond bijectively to the nodes of T. Each nondummy node of T_ℓ carries a label from $[1..f]$, namely the index of the child range at level $\ell + 1$ to which the node descends, so the labels of T_ℓ record the level-ℓ to level-$(\ell + 1)$ transition of every node. We store T_ℓ via Theorem 3.6.2 at alphabet size $\sigma = f$, which consumes $O(|T_\ell| \lg f) = O(n \lg f)$ bits and supports the marker-based α-operations of Definition 3.6.1 in time $O(\lg \lg_\omega f) = O(1)$, because $\omega = \Theta(\lg n)$ and $f = O(\lg^\epsilon n)$ give $\lg_\omega f = (\epsilon \lg \lg n)/\lg \lg n = \Theta(1)$. We also index the preorder label sequence of T_ℓ by Lemma 3.7.2 and Lemma 3.7.3; both fit within the $O(n \lg f)$-bit budget of the primary encoding.

[46]: He et al. (2016), *Data structures for path queries*

Theorem 3.7.4 (Path queries on the word RAM [46]) *Let T be an ordinal tree on n nodes with labels in $[1..u]$, under the word RAM model with word size $\omega = \Omega(\lg n)$. There exists a representation of T in $O(n)$ words of space that supports the operation $\texttt{path\_count}(v, [p..q])$ of Definition 3.7.1 in $O(\lg_\omega u)$*

time and the operation `path_select`(v, i) *of Definition 3.7.1 in* $O(\lg u / \lg \lg u)$ *time.*

Proof. The statement combines three ingredients from He, Munro, and Zhou [46]: a linear-space encoding that realises the range tree of Theorem 3.7.1 through Theorem 3.6.2 rather than through word-record pointers, a fanout refinement for path counting, and a separate fanout refinement for path selection. Both refinements replace per-node navigation by packed-word operations. The two queries calibrate the fanout differently, so their encodings and analyses proceed in parallel.

[46]: He et al. (2016), *Data structures for path queries*

Fix $\epsilon \in (0, 1)$ and set $f = \lceil \lg^{\epsilon} n \rceil$. The f-ary conceptual range tree on $[1..u]$ has height $h = \lceil \log_f u \rceil$. For each level $\ell \in \{1, \ldots, h\}$ store the merged tree T_ℓ through Theorem 3.6.2 at alphabet size f, using $O(n \lg f)$ bits since T_ℓ has $n + 1$ nodes. Summing over levels,

$$\sum_{\ell=1}^{h} O(n \lg f) \;=\; O(nh \lg f) \;=\; O\left(n \cdot \frac{\lg u}{\lg f} \cdot \lg f\right) \;=\; O(n \lg u) \text{ bits},$$

which is $O(n)$ words under $\omega = \Omega(\lg n)$. The auxiliary index of Lemma 3.7.2, applied to the preorder label sequence of T_ℓ, contributes $o(n)$ bits per level, hence $o(n \lg_\omega u)$ bits overall. The auxiliary representation of Lemma 3.7.3 contributes $O(n \lg f)$ bits per level, hence $O(n \lg u)$ bits overall. Both are subsumed by the primary encoding.

Given $v \in V(T)$ and $[p..q] \subseteq [1..u]$, the recursion of Theorem 3.7.1 runs on the f-ary range tree. On input $([a..b], w)$ at level ℓ, with w the image in $T_{a,b}$ of the deepest $R_{a,b}$-ancestor of v, the recursion returns the number of ancestors of v with label in $[a..b] \cap [p..q]$. If $[a..b] \subseteq [p..q]$, return $\mathrm{dep}_{T_{a,b}}(w)$, which counts the ancestors of v with label in $[a..b]$ by Lemma 3.5.3. If $[a..b] \cap [p..q] = \emptyset$, return 0. Otherwise recurse on the child ranges $[a_1..b_1], \ldots, [a_f..b_f]$ that intersect $[p..q]$.

Each recursive call needs the image w_i of w in the child tree T_{a_i,b_i}. The label of w in T_ℓ identifies which child range w descends into, so the preorder rank of w_i in $T_{\ell+1}$ is obtained from that of w in T_ℓ by a rank_i call on the label sequence of T_ℓ. This call decomposes into one Lemma 3.7.2 difference and one Lemma 3.7.3 correction, each $O(1)$. The depth $\mathrm{dep}_{T_{a,b}}(w)$ in the containment case is computed through Theorem 3.6.2 on T_ℓ in $O(\lg \lg_\omega f) = O(1)$ time, since $\omega = \Theta(\lg n)$ and $f = O(\lg^{\epsilon} n)$ give $\lg_\omega f = \Theta(1)$. The canonical decomposition of $[p..q]$ visits $O(h)$ ranges, so path counting runs in $O(h) = O(\lg u / \lg \lg n) = O(\lg_\omega u)$ time.

For selection, fix $\epsilon' \in (0, 1)$, set $f' = \lceil \lg^{\epsilon'} u \rceil$, and build a separate conceptual range tree on $[1..u]$ with branching factor f' and height $h' = \lceil \log_{f'} u \rceil + 1 = \Theta(\lg u / \lg \lg u)$. At every nonbottom level ℓ, store the merged tree T'_ℓ through Theorem 3.6.2 at alphabet size f'. The telescope $\sum_\ell O(n \lg f') = O(n \lg u)$ bits is $O(n)$ words.

Selection requires one further auxiliary structure per level, an array $X[1..n_1, 1..\Delta]$ where n_1 is the number of tier-1 roots of T'_ℓ under the tree-covering framework of [40, 41] that underlies Lemma 3.7.3, and $\Delta = O(\lg^{\epsilon'} u)$ is the number of sections into which a packed $\lceil \lg n \rceil$-bit counter word is split. For each tier-1 root r_i^1 and section index p, the entry $X[i, p]$ prepends one marker bit to $\mathit{sect}_p(A[r_i^1, \beta])$ for every $\beta \in [1..f']$ and concatenates the f' padded fragments into a single machine word,

[40]: Geary et al. (2006), *Succinct Ordinal Trees with Level-Ancestor Queries*
[41]: Farzan et al. (2008), *A Uniform Approach Towards Succinct Representation of Trees*

where $A[r_i^1, \beta]$ counts the β-bounded labels on the path from r_i^1 to the root. The total bit cost is $n_1 \cdot \Delta \cdot O(\lg n) = O(n \lg^{\epsilon'} u / \lg n) = o(n)$ per level. Aggregate space stays $O(n)$ words.

Given v and i, descend a single root-to-leaf branch of the f'-ary range tree tracking the residual rank. At a range $[a..b]$ of level ℓ with child ranges $[a_1..b_1], \ldots, [a_{f'}..b_{f'}]$, the recursion must locate the smallest γ for which the ancestors of v with label in $[a..b_\gamma]$ number at least i. By Lemma 3.5.3, this count is $\sum_{j \le \gamma} \mathrm{dep}_{T_{a_j,b_j}}(w_j)$, where w_j is the image of w in T_{a_j,b_j}. The array X packs the sections of these partial sums across the f' children into $O(1)$ machine words, and a constant number of word-parallel comparisons identify two candidates α_1, α_2 with $\gamma \in [\alpha_1, \alpha_2)$. When $\alpha_2 - \alpha_1 \le 1$, γ is read off in $O(1)$; otherwise a binary search of cost $O(\lg f')$ disambiguates. Across the h' levels the binary search triggers at most $\Delta - 1$ times, and non-search operations stay $O(1)$ per level, so the total time is

$$O(h') + \Delta \cdot O(\lg f') \;=\; O\left(\frac{\lg u}{\lg \lg u}\right) + O\left(\lg^{\epsilon'} u \cdot \lg \lg u\right) \;=\; O\left(\frac{\lg u}{\lg \lg u}\right),$$

since $\epsilon' < 1$. At the bottom level the label is fixed by the singleton range, and the corresponding ancestor of v follows from the stored preorder rank. □

The linear-space cost recorded in Theorem 3.7.4 is a consequence of the thesis-wide convention of measuring space in machine words. He, Munro, and Zhou sharpen the same construction to an entropy-compressed bit-level bound driven by the zeroth-order entropy $H(W_T)$ of the multiset of node labels of T, which satisfies $H(W_T) \le \lg u$ with equality only when the labels appearing in T are all distinct. Their refinement of the path-counting encoding uses $n(H(W_T) + 2) + o(n)$ bits when $u = O(\lg^{\epsilon} n)$ for some constant $\epsilon \in (0, 1)$, and $nH(W_T) + O(n \lg u / \lg \lg n)$ bits otherwise, while retaining the path-counting time of Theorem 3.7.4. Their refinement of the path-selection encoding uses $n(H(W_T) + 2) + o(n)$ bits when $u = O(1)$, and $nH(W_T) + O(n \lg u / \lg \lg u)$ bits otherwise, while retaining the path-selection time of Theorem 3.7.4. The question whether these bit-level refinements compose with the compression measures introduced by the set-collection constructions of later chapters is taken up in the open-problem discussion of Chapter 7.

The three theorems of this chapter supply the labeled-tree machinery that the downstream constructions of the thesis need. The α-operations of Theorem 3.6.2 run in $O(\lg \lg_\omega u)$ time, the binary-partition path queries of Theorem 3.7.1 in $O(\lg u)$ time, and the sublogarithmic word-RAM refinement of Theorem 3.7.4 answers path counting in $O(\lg_\omega u)$ time and path selection in $O(\lg u / \lg \lg u)$ time. Later chapters of the thesis appeal to these three theorems when building compressed representations for collections of sets.

4 Hierarchical Encodings

4.1 Setup and baseline 54
4.2 Insertion compressibility . 55
4.3 Symmetric-difference compressibility 61

[2]: Alanko et al. (2025), *Compact data structures for collections of sets*

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

Two compressibility regimes have emerged from the literature on collections of correlated sets. Each turns a counting bound into a labelled ordinal tree, on which the tree-extraction primitives of Section 3.5 answer the five operations of Definition 1.2.2 in sublogarithmic time per query.

The per-set baseline of Section 4.1 ignores every relationship among the sets of $\mathcal{S}$. When the collection has structure, two regimes from the literature drop strictly below the baseline. The insertion regime of Alanko et al. [2] and Gagie, He, and Navarro [9] encodes each set against a directional reference, a smallest superset or a largest subset already in the collection, and pays in the binomial cost of one and in the deletion or insertion count of the other. The indel regime of Gagie, He, and Navarro generalises to undirected references measured by symmetric difference, with a thirty-year prehistory in inverted indexes, fast Boolean matrix multiplication, web-graph compression, and coloured de Bruijn graphs. Both regimes turn into labelled ordinal trees on which the tree-extraction primitives of Section 3.5 answer the five operations of Definition 1.2.2.

4.1 Setup and baseline

Chapter 1 fixed the problem and its operations; Chapter 2 showed how to meet the per-set baseline with sparse bitvectors. The present chapter asks what strictly less than that baseline buys us when $\mathcal{S}$ has structure, and which combinatorial quantity tracks the gain.

We return to the setting of Section 1.2. A collection $(\mathcal{S}, U)$ consists of a totally ordered universe $U = [1..u]$ of size u and a family $\mathcal{S} = \{S_1, \ldots, S_m\}$ of m finite subsets of U, with $n = \sum_{i=1}^{m} |S_i|$ counting elements across sets with multiplicity (Definition 1.2.1). A representation of $\mathcal{S}$ must support the five operations of Definition 1.2.2, namely member, access, rank, predecessor, and successor. The independent-encoding baseline of Definition 1.2.3,

$$H_{wc}(\mathcal{S}) \;=\; \sum_{i=1}^{m} \lg \binom{u}{|S_i|},$$

is the per-set information content of the collection when each S_i is viewed as an isolated subset of U. No inter-set relationship contributes to this cost.

The baseline is achievable up to lower-order terms. Applying Theorem 2.2.2 to the characteristic bitvector of each S_i separately stores the whole collection in

$$H_{wc}(\mathcal{S}) \;+\; O(n + m \lg n) \quad \text{bits},$$

supporting access on any $S \in \mathcal{S}$ in $O(1)$ time, rank in $O(1 + \lg(u/|S|))$ time, and member as an access at the rank position [27]. Predecessor and successor reduce to one rank plus one access. The $O(n)$ slack accounts for the remainder arrays in the m per-set Elias-Fano layouts (Subsection 2.2.2); the $O(m \lg n)$ slack pays for a directory of pointers into the m separate bitvectors, so that each set can be addressed in $O(1)$ time.

[27]: Okanohara et al. (2007), *Practical entropy-compressed rank/select dictionary*

In the applications of Section 1.1, $\mathcal{S}$ is never a bag of unrelated sets. Adjacency lists of neighbouring web nodes share most of their links, and k-mer colour sets change by one element between consecutive k-mers. The baseline ignores this overlap entirely.

The set-by-set representation is the right starting point when the collection really is a bag of unrelated sets. For every S_i, there are $\binom{u}{|S_i|}$ subsets of U of that size, and distinguishing them costs $\lg \binom{u}{|S_i|}$ bits. When $\mathcal{S}$ is genuinely unstructured, no encoding can do better than summing the per-set information contents. The difficulty is that collections arising

in practice are almost never unstructured. Two sets with symmetric difference of size one pay full price twice under H_{wc}, and nothing in the baseline notices that one is essentially a copy of the other. If $S_i \subset S_j$, describing S_i as a subset of S_j costs $\lg \binom{|S_j|}{|S_i|}$ bits, strictly less than $\lg \binom{u}{|S_i|}$ when $|S_j| < u$. If the symmetric difference $S_i \triangle S_j$ is small, a description of S_j as an edit of S_i costs only $|S_i \triangle S_j| \lg u$ bits. Every such relationship is invisible to H_{wc}.

The rest of this chapter answers both the combinatorial and the structural side of the question, taking each compressibility measure end to end. We assume from here on that $U = \bigcup_{i=1}^{m} S_i$, so that every universe element belongs to at least one set of the collection. Universe elements outside $\bigcup_i S_i$, when present in the input, are handled by applying the results to the restriction $U' = \bigcup_i S_i$. Two measures follow. Insertion compressibility (Section 4.2) encodes every set against a smaller reference and realises the resulting compression through a labelled ordinal tree on which the tree-extraction primitives of Section 3.5 answer the five operations of Definition 1.2.2. Symmetric-difference compressibility (Section 4.3) drops the requirement of a containment between a set and its reference and pays in symmetric-difference labels of two signs, with a parallel structural realisation. Throughout, cost is measured in bits of information content for the combinatorial side and in words of space and query time for the structural side.

The convention $U = \bigcup_i S_i$ is the one adopted by Alanko et al. [2] and by Gagie, He, and Navarro [9]; under it every universe element has a nonempty pattern of membership across $\mathcal{S}$, which sharpens the combinatorial analysis of inter-set structure.

4.2 Insertion compressibility

Given a set S and a reference set R, how many bits are needed to describe S once R is known? Two primitive cases admit a clean answer. When $S \subseteq R$, describing S reduces to choosing $|S|$ positions out of the $|R|$ elements of R, at a cost of $\lg \binom{|R|}{|S|}$ bits. When $R \subseteq S$, describing S reduces to listing the $|S \setminus R|$ elements that must be added to R to obtain S, each drawable from $U \setminus R$. The two cases are duals of one another. One specifies which elements of a larger reference are retained, the other specifies which elements outside a smaller reference are inserted.

The question is posed on a single pair (S, R). The next two subsections apply each primitive to the whole collection, obtaining two compressibility measures whose definitions disagree on which direction of the containment is assumed but whose information-theoretic flavour is the same.

Raised to a whole collection $\mathcal{S}$, each primitive yields a compressibility measure. Choosing the reference $R = p(S)$ to be a smallest proper superset of S in $\mathcal{S}$ instantiates the first primitive and gives the containment entropy $L(\mathcal{S})$ of Alanko, Bille, Gørtz, Navarro, and Puglisi [2]. Choosing $R = p'(S)$ to be a largest strict subset of S in $\mathcal{S}$ instantiates the second and gives the insertion compressibility $I(\mathcal{S})$ of Gagie, He, and Navarro [9]. The remainder of this section develops the two primitives as counting bounds, applies each to $\mathcal{S}$ to obtain the corresponding measure and its known space realisation, and overlays the tree-extraction primitives of Section 3.5 on the insertion tree to answer the five operations of Definition 1.2.2.

[2]: Alanko et al. (2025), *Compact data structures for collections of sets*

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

4.2.1 Reference encoding

We isolate the two primitives as purely combinatorial counting bounds, independently of any data-structure realisation. Throughout, U is the universe of size u and $S, R \subseteq U$ are finite.

The two primitives do not exhaust the space of reference encodings. When R and S are incomparable, one records the symmetric difference $S \triangle R$; this is taken up in the next section.

Proposition 4.2.1 (Subset primitive) *If $S \subseteq R \subseteq U$, then S can be described in $\lg \binom{|R|}{|S|}$ bits given R. The bound is tight over the class of subsets of R of size $|S|$.*

Proof. There are exactly $\binom{|R|}{|S|}$ subsets of R of size $|S|$, and S is one of them. An encoding restricted to this class uses $\lg \binom{|R|}{|S|}$ bits by indexing. Since every subset of R of size $|S|$ is a valid value of S, no shorter prefix-free code exists. □

The subset primitive strictly improves on the independent cost $\lg \binom{u}{|S|}$ of Definition 1.2.3 whenever $|R| < u$. Indeed $\binom{|R|}{|S|} \leq \binom{u}{|S|}$ with equality only when $|R| = u$, so $R = U$ recovers the baseline. Every proper reference set yields a strict saving.

Proposition 4.2.2 (Superset primitive) *If $R \subseteq S \subseteq U$, then S can be described by specifying the $|S \setminus R|$ elements of $S \setminus R$. An encoding with alphabet $U \setminus R$ uses $|S \setminus R| \lg(u - |R|)$ bits, and one with alphabet U uses $|S \setminus R| \lg u$ bits.*

Proof. The elements of $S \setminus R$ are distinct members of $U \setminus R$. Encoding each as an integer in $[1..u - |R|]$ costs $\lg(u - |R|)$ bits; encoding each as an integer in $[1..u]$ costs $\lg u$ bits. The second encoding discards the knowledge of R and is looser. □

The two primitives describe S given R in opposite directions. The subset primitive measures S as a fraction of a larger reference; the superset primitive measures how many elements must be added to a smaller reference. Neither exhausts the possibilities when R and S are incomparable. The symmetric-difference primitive, where R plays the role of a close but unordered neighbour of S, is developed in the next section.

4.2.2 Containment entropy

Alanko et al. write n_i for $|S_i|$, p_i for $|p(S_i)|$, n for the total size, and s for the number of sets. We keep the thesis notation from Definition 1.2.1.

[2]: Alanko et al. (2025), *Compact data structures for collections of sets*

Alanko et al. [2] apply Proposition 4.2.1 to every set in $\mathcal{S}$ at once. For each S_i, the reference is a smallest proper superset of S_i among the sets of $\mathcal{S}$. If none exists, the reference is U itself. The choice is canonical enough to induce a tree on $\mathcal{S} \cup \{U\}$.

Definition 4.2.1 (Containment hierarchy [2]) *Let $\mathcal{S} = \{S_1, \ldots, S_m\}$ be a collection over universe U. The containment hierarchy of $\mathcal{S}$ has U at the root, and for each $S_i \in \mathcal{S}$ the parent $p(S_i)$ is a smallest set in $\mathcal{S}$ strictly containing S_i; if no such set exists, then $p(S_i) = U$.*

Definition 4.2.2 (Containment entropy [2]) *The containment entropy of $\mathcal{S}$ is*

$$L(\mathcal{S}) \;=\; \sum_{i=1}^{m} \lg \binom{|p(S_i)|}{|S_i|},$$

where $p(S_i)$ is the parent of S_i in the containment hierarchy.

The definition is an instance-by-instance application of Proposition 4.2.1. Every S_i is a subset of its parent $p(S_i)$, so the subset primitive gives a description of length $\lg \binom{|p(S_i)|}{|S_i|}$ bits. Summing over i yields the containment entropy. When the hierarchy is trivial and every $p(S_i)$ equals U, the containment entropy collapses to the baseline.

Proposition 4.2.3 (Containment bound[2]) *For every collection $\mathcal{S}$,*

$$L(\mathcal{S}) \;\le\; H_{wc}(\mathcal{S}).$$

Proof. Since $p(S_i) \subseteq U$, we have $|p(S_i)| \le u$, and hence $\binom{|p(S_i)|}{|S_i|} \le \binom{u}{|S_i|}$. Taking logarithms and summing over i gives $L(\mathcal{S}) \le \sum_i \lg \binom{u}{|S_i|} = H_{wc}(\mathcal{S})$. □

Translating the bound into space requires a device that shortens every root-to-leaf path in the hierarchy. A direct representation stores each S_i as a sparse bitvector of length $|p(S_i)|$ indicating which elements of the parent are retained. Queries on S_i then walk the chain of ancestors up to U, which can be as long as the hierarchy itself. Alanko et al. introduce a path-compressed variant in which the parent of S_i skips all intermediate ancestors whose size is at most $2|S_i|$.

Definition 4.2.3 (Contracted hierarchy [2]) *The contracted hierarchy of $\mathcal{S}$ has U at the root. For each S_i, its parent $c(S_i)$ is the highest ancestor S_t of $p(S_i)$ in the containment hierarchy satisfying $|S_t| \le 2|S_i|$; if no such ancestor exists, $c(S_i) = p(S_i)$.*

Proposition 4.2.4 (Depth of the contracted hierarchy [2]) *In the contracted hierarchy of $\mathcal{S}$, every node S_i has depth $O(\log(u/|S_i|))$.*

Sketch. By Lemma 4 of [2], consecutive ancestors on the contracted path from S_i to U at least double in size every two steps, bounding the path length by $O(\log(u/|S_i|))$. □

Encoding S_i as a sparse bitvector relative to $c(S_i)$ inflates the bit cost by at most a constant factor and leaves $L(\mathcal{S})$ unchanged asymptotically (Lemma 5 of [2]); the sparse-bitvector machinery is the one built in Section 2.2.

Theorem 4.2.5 (Containment-bounded representation [2]) *Let $\mathcal{S}$ be a collection of m sets over a universe of size u, with total size $n = \sum_i |S_i|$ as in Definition 1.2.1. The contracted hierarchy of $\mathcal{S}$ can be stored in*

$$L(\mathcal{S}) + O(n + m \lg n) \quad \textit{bits}$$

using one sparse bitvector per edge via Theorem 2.2.2, supporting on every $S \in \mathcal{S}$ the five operations of Definition 1.2.2 in time $O(\log(u/|S|))$.

The $O(n + m \lg n)$ slack is the familiar cost accounted for in the baseline of Section 4.1. Each of the m sparse bitvectors spends $O(|S_i|)$ additive bits in its Elias-Fano layout, summing to $O(n)$, and $O(\lg n)$ bits per set are spent on a directory that addresses the m separate structures.

Alanko et al. observe in Section 3 of [2] that the containment hierarchy is not the cheapest possible. When many sets are subsets of both S_i

and S_j, introducing a new set $S_i \cap S_j$ as an internal node could shorten further the encoding, even though $S_i \cap S_j$ is not in $\mathcal{S}$. They remark that an advantage of $L(\mathcal{S})$ is that it is computable in polynomial time from $\mathcal{S}$, whereas more general notions that allow the introduction of new sets may not be. The question of how to augment the hierarchy with synthetic nodes, in particular which augmentation minimises the total encoding cost, is left open in [2].

4.2.3 Insertion compressibility

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

Gagie, He, and Navarro [9] call the measure insertion compressibility because every set is described by the elements *inserted* into a smaller reference. The dual of containment entropy describes the elements *deleted* from a larger reference to obtain the set. The two are the same primitive applied in opposite directions.

Gagie, He, and Navarro [9] apply Proposition 4.2.2 to every set in $\mathcal{S}$. For each $S \in \mathcal{S}$, the reference is a largest strict subset of S in $\mathcal{S}$. If none exists, the reference is the empty set.

Definition 4.2.4 (Insertion compressibility [9]) *Let $\mathcal{S}$ be a collection. For each $S \in \mathcal{S}$, let $p'(S)$ be a largest strict subset of S in $\mathcal{S}$, or $\emptyset$ if no such subset exists. The insertion compressibility of $\mathcal{S}$ is*

$$I(\mathcal{S}) \;=\; \sum_{S \in \mathcal{S}} |S \setminus p'(S)| \;=\; \sum_{S \in \mathcal{S}} \big(|S| - |p'(S)|\big).$$

The description length implied by the superset primitive is $|S \setminus p'(S)| \lg u$ bits per set. Dropping the $\lg u$ factor, $I(\mathcal{S})$ counts the total number of inserted elements across all sets, which is bounded by the total size of $\mathcal{S}$.

Proposition 4.2.6 (Insertion bound [9]) *For every collection $\mathcal{S}$ with total size $n = \sum_i |S_i|$,*

$$I(\mathcal{S}) \;\le\; n.$$

Proof. By Definition 4.2.4, $I(\mathcal{S}) = \sum_{S \in \mathcal{S}} |S \setminus p'(S)|$. For each $S \in \mathcal{S}$, $|S \setminus p'(S)| \le |S|$, so $I(\mathcal{S}) \le \sum_{S \in \mathcal{S}} |S| = n$. □

Equality holds when $p'(S) = \emptyset$ for every S, which happens whenever no two sets of $\mathcal{S}$ are in a containment relation. At the opposite extreme, a deep chain $S_1 \subset S_2 \subset \cdots \subset S_m$ gives $I(\mathcal{S}) = |S_m|$, a quantity that can be far below n when the chain is long. A direct comparison with $H_{wc}(\mathcal{S})$ is unit-dependent: the insertion representation pays $I(\mathcal{S}) \lg u$ bits against $H_{wc}(\mathcal{S}) = \sum_i \lg \binom{u}{|S_i|}$ bits, and neither dominates the other across all collections. The value of $I(\mathcal{S})$ is that it exposes sequential containment structure, which is invisible to the per-set baseline.

The dual construction that realises $I(\mathcal{S})$ structurally is the insertion graph of [9]. Its nodes are the sets of $\mathcal{S}$ together with $\emptyset$, and each S has a single outgoing edge to $p'(S)$ of weight $|S \setminus p'(S)|$. The edge weights sum to $I(\mathcal{S})$ by Definition 4.2.4. Since p' is a functional parent map and $|p'(S)| < |S|$ strictly, the graph has no cycles and every path eventually reaches $\emptyset$; it is a forest rooted at $\emptyset$.

The chain expansion is the move that makes the insertion graph usable as a labelled ordinal tree. Weights are replaced by element labels; the multiset of labels on the root-to-$v(S)$ path is exactly the set S.

The graph is not directly a labelled ordinal tree, because its edges carry numeric weights rather than element labels. Gagie et al. expand each weighted edge into a chain.

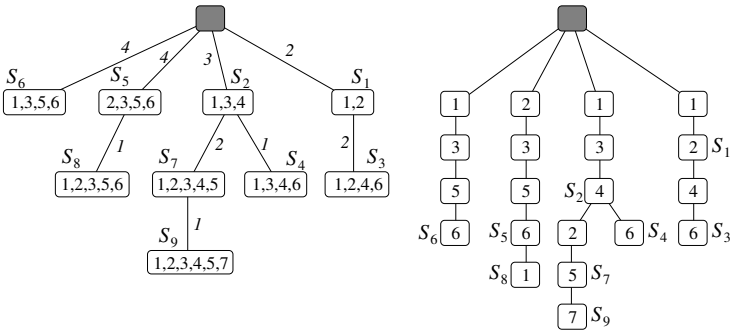

Figure 4.1: Reproduced from Gagie, He, and Navarro [9]. Left: an insertion graph for a collection $\mathcal{S} = \{S_1, \ldots, S_9\}$, with edge weights written as slanted values. The edge weights sum to $I(\mathcal{S}) = 20$. Right: the corresponding insertion tree obtained by expanding each edge $(v(p'(S)), v(S))$ of weight $w = |S \setminus p'(S)|$ into a chain of w labelled edges. The tree has $1 + I(\mathcal{S}) = 21$ nodes. Each set S_i labels the chain endpoint $v(S_i)$.

Definition 4.2.5 (Insertion tree [9]) *Let $\mathcal{S}$ be a collection, with $p'(S)$ as in Definition 4.2.4. The insertion tree of $\mathcal{S}$ is the rooted labelled ordinal tree defined by the following rules. The root is $v(\emptyset)$. For every $S \in \mathcal{S}$ there is a node $v(S)$, and the tree contains a chain of $|S \setminus p'(S)|$ edges from $v(p'(S))$ to $v(S)$, each labelled with a distinct element of $S \setminus p'(S)$, in arbitrary order. The tree may contain additional internal nodes introduced by these chains but not associated with any set of $\mathcal{S}$.*

Proposition 4.2.7 (Insertion tree structure[9]) *The insertion tree of $\mathcal{S}$ has $1 + I(\mathcal{S})$ nodes. Every edge label lies in $[1..u]$, and for every $S \in \mathcal{S}$ the set of labels on the path from $v(\emptyset)$ to $v(S)$ is exactly S.*

Proof. The root contributes one node. Each $S \in \mathcal{S}$ contributes a chain of $|S \setminus p'(S)|$ edges, hence $|S \setminus p'(S)|$ new nodes. Summing over $\mathcal{S}$ adds $\sum_S |S \setminus p'(S)| = I(\mathcal{S})$ nodes. Every label lies in $U = [1..u]$ by construction. An induction on depth shows that the labels on the root-to-$v(S)$ path accumulate exactly the elements of $p'(S) \cup (S \setminus p'(S)) = S$: the induction base is the root, which represents $\emptyset$, and each step adds $S \setminus p'(S)$ to $p'(S)$. □

Figure 4.1 shows the insertion graph for a nine-set collection and the insertion tree obtained from it by chain expansion. The edge weights on the left add up to $I(\mathcal{S}) = 20$, and the tree on the right has 21 nodes in total. The labels on the root-to-$v(S_i)$ path reproduce the content of S_i.

The insertion tree carries labels from $[1..u]$ and no deletion marks. Compared to an encoding that lists for each S_i the elements of $S_i \setminus p'(S_i)$ as an unordered collection, the tree is only a structural reorganisation: no additional information is stored, and the total label count is $I(\mathcal{S})$.

Proposition 4.2.8 (Insertion-graph construction [9]) *Given a collection $\mathcal{S}$ of m sets with total size n over a universe of size u, the insertion graph and its chain expansion into the insertion tree can be built in $O(n \lg u + mn)$ time on a word RAM with $\omega = \Omega(\lg n)$.*

Sketch. Sort the sets by increasing size and the elements of every set in increasing order, at aggregate cost $O(n \lg u)$. Insert the sets into a growing graph that starts from the single node $\emptyset$. For each set S, a DFS from $\emptyset$ identifies its largest strict subset $p'(S)$ among the already-inserted sets through merge-based containment checks, each costing time linear in the two sets involved. Summed over all insertions the traversal cost is $O(mn)$, and chain expansion of the graph into the insertion tree adds $O(I(\mathcal{S})) \subseteq O(n)$. □

4.2.4 Insertion-tree queries

The insertion tree is the one representation of this chapter whose access reaches the sublogarithmic bound of Theorem 3.7.4. The indel construction of the next section pays $O(\lg u)$ for access for a different structural reason.

The insertion tree of Definition 4.2.5 is a labeled ordinal tree of $1 + I(\mathcal{S})$ nodes, with labels in $[1..u]$ and root $v(\emptyset)$. Its defining property, recorded in Proposition 4.2.7 and illustrated in Figure 4.1, is that the labels on the path from $v(\emptyset)$ to $v(S)$ are exactly the elements of S. Every operation of Definition 1.2.2 on S thus becomes a question about the ancestors of $v(S)$.

We overlay two indexes on the tree. The α-operations index of Theorem 3.6.2 supports $\texttt{parent}_\alpha$ for every label $\alpha \in [1..u]$. The path-queries index of Theorem 3.7.4 supports `path_count` and `path_select` for arbitrary label ranges. Each index stores a labeled ordinal tree of $|T|$ nodes in $O(|T|)$ words, and $|T| = 1 + I(\mathcal{S})$ by Proposition 4.2.7, so the overlay occupies $O(I(\mathcal{S}))$ words in total. Every operation below calls one of the two primitives once.

Membership asks whether $x \in S$. Since the elements of S are exactly the labels above $v(S)$, the test reduces to $\texttt{parent}_x(v(S)) \neq \perp$, which the α-operations index answers in $O(\lg \lg_\omega u)$ time.

Rank asks for $|\{y \in S : y \leq x\}|$. Because the ancestors of $v(S)$ form S, this count equals the number of ancestors of $v(S)$ whose label lies in $[1..x]$, which $\texttt{path\_count}(v(S), [1..x])$ returns in $O(\lg_\omega u)$ time.

Access asks for the i-th smallest element of S. This is the ancestor of $v(S)$ whose label is the i-th smallest along the path, returned by $\texttt{path\_select}(v(S), i)$ in $O(\lg u / \lg \lg u)$ time.

Predecessor and successor follow the rank-then-access reduction recalled in Section 1.2. The predecessor of x in S is $\text{access}(S, \text{rank}(S, x))$ when $\text{rank}(S, x) \geq 1$ and undefined otherwise; the successor is $\text{access}(S, \text{rank}(S, x-1) + 1)$ when that index does not exceed $|S|$. Each query performs one rank and one access, and the access term $O(\lg u / \lg \lg u)$ dominates.

Theorem 4.2.9 (Insertion tree representation [9]) *Let $\mathcal{S}$ be a collection of subsets of $[1..u]$ and assume the word RAM model with word size $\omega = \Omega(\lg n)$. The insertion tree of $\mathcal{S}$ admits a representation in $O(I(\mathcal{S}))$ words of space supporting, for every $S \in \mathcal{S}$, the operations of Definition 1.2.2 within the following time bounds.*

- $\text{member}(S, x)$ *in time* $O(\lg \lg_\omega u)$.
- $\text{rank}(S, x)$ *in time* $O(\lg_\omega u)$.
- $\text{access}(S, i)$ *in time* $O(\lg u / \lg \lg u)$.
- $\text{predecessor}(S, x)$ *in time* $O(\lg u / \lg \lg u)$.
- $\text{successor}(S, x)$ *in time* $O(\lg u / \lg \lg u)$.

Chain-like containment is what the insertion tree compresses. When $S \subsetneq S'$ and $|S' \setminus S|$ is small, the chain from $v(S')$ to $v(S)$ is short and contributes little to $I(\mathcal{S})$. Two sets close in symmetric difference but incomparable under inclusion see no such saving. Each of them sits as a chain descending directly from $v(\emptyset)$, and both contribute their full size to $I(\mathcal{S})$. The next section develops a compressibility measure that rewards symmetric-difference proximity directly, irrespective of inclusion.

4.3 Symmetric-difference compressibility

Containment and insertion, developed in Section 4.2, both pick a directional reference. The containment hierarchy assigns each $S \in \mathcal{S}$ a parent $p(S) \supseteq S$ and pays $\lg \binom{|p(S)|}{|S|}$ bits; the insertion hierarchy assigns a parent $p'(S) \subseteq S$ and pays $|S \setminus p'(S)| \lg u$ bits. When two sets $A, B \in \mathcal{S}$ are incomparable, neither $A \subseteq B$ nor $B \subseteq A$ holds, so neither primitive gives any saving over the per-set baseline $H_{wc}(\mathcal{S})$.

The subset and superset primitives of Subsection 4.2.1 both require a directional relationship between S and its reference. Neither applies when S and R are incomparable. The symmetric-difference primitive is the smallest natural extension that covers this case.

The symmetric difference $|A \triangle B| = |A \setminus B| + |B \setminus A|$ is the symmetric alternative. It measures proximity without requiring a containment. Gagie, He, and Navarro [9] turn this quantity into a compressibility measure $\Delta(\mathcal{S})$, and give the first representation of size $O(\Delta(\mathcal{S}))$ words that answers the five operations of Definition 1.2.2 directly on the compressed form. This section develops $\Delta(\mathcal{S})$ from its prehistory in information retrieval, matrix multiplication, and graph compression, constructs the minimum-weight symdiff graph by an adapted Prim algorithm, introduces the indel tree as the structural object that carries queries, and overlays the tree-extraction primitives of Section 3.5 on it to answer the five operations.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

4.3.1 Historical context

The idea of encoding a set as the symmetric difference with a close neighbour predates the compressibility measure by three decades. It was rediscovered in four distinct communities, each time motivated by a specific application and each time without full awareness of the previous occurrences.

The earliest systematic treatment is by Bookstein and Klein [26], motivated by inverted indexes in information retrieval. For each term of a text collection, a posting bitvector records the segments in which the term occurs; semantically correlated terms, such as the pair *Security* and *Council*, produce posting bitvectors with heavy overlap. Bookstein and Klein form the complete weighted graph on the posting bitvectors, with edge weight equal to the Hamming distance, introduce an artificial zero vector as an extra node, and take a minimum spanning tree of the augmented graph. Every posting bitvector is then encoded as the XOR with its parent in the tree (for non-root bitvectors) or directly (for root bitvectors). They report that this preprocessing nearly doubles the compression saving on the Hebrew Bible corpus when combined with sparse-bitvector coders.

Bookstein and Klein [26] treat each bitvector as a set and identify the Hamming distance between two bitvectors with the cardinality of the symmetric difference between the corresponding sets.

[26]: Bookstein et al. (1991), *Compression of correlated bit-vectors*

The same broader idea of encoding each object via its difference from a neighbour reappears a decade later in two different communities. Buchsbaum, Caldwell, Church, Fowler, and Muthukrishnan [53] tackle the compression of massive tables in engineering contexts through differential dependencies, storing each column as a delta against a greedily chosen predecessor column. In a parallel development, Björklund and Lingas [54] study fast Boolean matrix multiplication for highly clustered data. Gagie, He, and Navarro [9] credit Björklund and Lingas with the first observation that matrix multiplication can be carried out in time bounded by the total weight of a spanning forest of the row similarity graph, since the symmetric difference between two rows carries all the information needed to update the product incrementally. Exact MST

The Hamming distance between two binary row vectors equals $|A \triangle B|$ when the rows are read as characteristic vectors over the column universe. The MST argument is Bookstein and Klein's; the application to matrix multiplication is Björklund and Lingas's.

[53]: Buchsbaum et al. (2000), *Engineering the compression of massive tables: an experimental approach*

[54]: Björklund et al. (2001), *Fast Boolean Matrix Multiplication for Highly Clustered Data*

construction was costly in their setting, and they used approximate MSTs instead.

Boldi and Vigna's [5] reference compression restricts candidate references to URL-sorted neighbours within a window W, so the resulting structure is an engineering approximation of a minimum-weight spanning forest rather than the true minimum. The gain in compression ratio is dramatic in practice.

[5]: Boldi et al. (2004), *The webgraph framework I: compression techniques*

The web-graph compression literature gives the most celebrated application. Boldi and Vigna [5] observe that when the rows of the adjacency matrix of a web graph are sorted by URL, consecutive rows share almost all of their outgoing edges. The WebGraph framework stores each adjacency list against a chosen reference list drawn from a bounded backreference window of size W, recording by a copy list which successors of the reference to retain and by a list of extra nodes which successors to add. The backreference choice is a heuristic approximation of a minimum-weight spanning forest of the row similarity graph, tuned to keep the decoding depth bounded. Gagie, He, and Navarro [9] note in retrospect that Boldi and Vigna were not aware of Bookstein and Klein's prior construction. Boldi and Vigna report compression ratios on the order of three to four bits per edge on real web crawls, an order-of-magnitude improvement over adjacency-list representations.

In Mantis [8], each k-mer in a de Bruijn graph carries a colour set recording the genomes in which the k-mer occurs. Colour sets of neighbouring k-mers differ by one or two colours, so the MST encoding yields several orders of magnitude improvement over per-k-mer storage.

[8]: Almodaresi et al. (2020), *An efficient, scalable, and exact representation of high-dimensional color information enabled using de Bruijn graph search*

[55]: Alves et al. (2025), *Accelerating Graph Neural Networks Using a Novel Computation-Friendly Matrix Compression Format*

The technique reaches bioinformatics with Almodaresi, Pandey, Ferdman, Johnson, and Patro [8], who compress the colour sets of coloured de Bruijn graphs in the Mantis tool. They restrict the similarity graph to edges between colour sets of neighbouring vertices in the de Bruijn graph, take a minimum spanning tree of the resulting graph, and encode every colour set on that MST. A more recent instance is due to Alves, Moustafa, Benkner, Francisco, Gansterer, and Russo [55], who apply the same idea to compress binary adjacency matrices for graph neural network acceleration. Gagie, He, and Navarro [9] take Alves et al. as their direct predecessor, and the two-rooted symdiff graph of Definition 4.3.1 is a variant of the one-rooted model used there.

The elevation to a compressibility measure is what separates Gagie, He, and Navarro's treatment [9] from its predecessors. A thirty-year sequence of independent rediscoveries becomes a single combinatorial quantity $\Delta(\mathcal{S})$ with a clean definition.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

Over thirty years of independent rediscoveries, the MST-on-Hamming construction remained a compression heuristic. Gagie, He, and Navarro [9] elevate it to a compressibility measure $\Delta(\mathcal{S})$ tracking exactly the weight of the minimum spanning tree, prove that $\Delta(\mathcal{S}) \leq I(\mathcal{S})$ holds for every collection, give a construction algorithm whose running time depends on the MST weight rather than on the total pairwise symdiff volume, and exhibit the first representation of size $O(\Delta(\mathcal{S}))$ words that answers member, access, rank, predecessor, and successor directly on the compressed form. Prior work only compressed; no prior work offered a queryable compressed form.

The rest of this section develops the combinatorial content of $\Delta(\mathcal{S})$. The construction algorithm of Gagie et al. is presented in condensed form, the indel tree carries the structural content into a labelled ordinal tree, and the query structure overlaid on it answers the five operations of Definition 1.2.2.

4.3.2 Symdiff graph

The special nodes $\emptyset$ and U are not elements of $\mathcal{S}$; they are auxiliary roots that anchor the two trees of the symdiff graph. The choice to allow *two* roots is what enables both insertion-like and deletion-like reductions in a single structure.

The object underlying the symmetric-difference primitive is a weighted directed graph on $\mathcal{S}$ augmented with two auxiliary nodes, $\emptyset$ and U. Every set of $\mathcal{S}$ points to exactly one neighbour, and the weight of the outgoing edge records the symmetric-difference cost of describing the set from its neighbour.

Definition 4.3.1 (Symdiff graph [9]) *A symdiff graph on a collection $\mathcal{S}$ is a weighted directed graph on the node set $\mathcal{S} \cup \{\emptyset, U\}$, where $U = \bigcup_{S \in \mathcal{S}} S$ is the universe spanned by $\mathcal{S}$. Every $S \in \mathcal{S}$ has exactly one outgoing edge; its target $p(S) \in \mathcal{S} \cup \{\emptyset, U\}$ is the neighbour of S, and the weight of the edge $(S, p(S))$ is $|S \triangle p(S)|$. The graph is required to satisfy the reachability condition that from every $S \in \mathcal{S}$ one of $\emptyset$ or U is reachable by following outgoing edges. The weight of the graph is the sum of the edge weights.*

Given a symdiff graph, each $S \in \mathcal{S}$ is reconstructed from its neighbour $p(S)$ by inserting the elements of $S \setminus p(S)$ and deleting those of $p(S) \setminus S$. The total number of labels stored across the whole collection equals the weight of the graph. The reachability condition ensures that the decoding terminates at one of the two canonical anchors, $\emptyset$ or U, whose content is known a priori.

Definition 4.3.2 (Symdiff compressibility [9]) *The symmetric-difference compressibility of $\mathcal{S}$ is*

$$\Delta(\mathcal{S}) \;=\; \min\{\, W(G) : G \text{ is a symdiff graph on } \mathcal{S} \,\},$$

where $W(G)$ denotes the weight of G.

The minimisation problem admits a clean structural description. A symdiff graph has $|\mathcal{S}|$ outgoing edges and at most $|\mathcal{S}| + 2$ nodes, with the two roots $\emptyset$ and U having no outgoing edges. The reachability condition forces the graph to consist of two rooted trees, one anchored at $\emptyset$ and one at U; every $S \in \mathcal{S}$ sits in exactly one of them. Minimising the total weight over all such two-tree configurations is captured by a single spanning-tree problem on an augmented graph.

Lemma 4.3.1 (Minimum-weight symdiff graph[9]) *Let K be the complete undirected graph on $\mathcal{S} \cup \{\emptyset, U\}$ with edge weights $w(A, B) = |A \triangle B|$ for $A, B \in \mathcal{S} \cup \{\emptyset, U\}$, except for the edge between $\emptyset$ and U, which is given weight 0. The weight of a minimum spanning tree of K equals $\Delta(\mathcal{S})$, and the two trees achieving $\Delta(\mathcal{S})$ are obtained by removing the zero-weight edge between $\emptyset$ and U from the MST.*

Proof. Every minimum spanning tree of K contains the edge between $\emptyset$ and U because it is the lightest in K. Removing this zero-weight edge disconnects the MST into two trees, one containing $\emptyset$ and the other containing U; the two trees span $\mathcal{S} \cup \{\emptyset, U\}$ and their total weight equals that of the MST. Conversely, any pair of trees rooted at $\emptyset$ and U together spanning $\mathcal{S} \cup \{\emptyset, U\}$, plus the zero-weight edge between $\emptyset$ and U, gives a spanning tree of K of the same weight. The minimum over pairs of rooted trees therefore coincides with the MST weight, and the minimum-weight two-tree configuration is recovered by deleting the zero-weight edge. □

The relation to insertion compressibility is immediate. Every insertion tree of Definition 4.2.5, read as a functional parent map with the extra node $\emptyset$ playing the role of its own root, gives a valid symdiff graph in which every set S points to its largest strict subset $p'(S)$.

Proposition 4.3.2 (Symdiff-insertion inequality[9]) *For every collection $\mathcal{S}$,*

$$\Delta(\mathcal{S}) \;\le\; I(\mathcal{S}).$$

Proof. The insertion graph of Subsection 4.2.3 has one outgoing edge from each $S \in \mathcal{S}$ to its largest strict subset $p'(S) \in \mathcal{S} \cup \{\emptyset\}$. Every S reaches $\emptyset$ along this chain, so the graph satisfies Definition 4.3.1 on the node set $\mathcal{S} \cup \{\emptyset, U\}$, with U an auxiliary node carrying no incident edges. Because $p'(S) \subseteq S$, no deletions contribute and $|S \triangle p'(S)| = |S \setminus p'(S)|$. Summing gives a symdiff graph of total weight $I(\mathcal{S})$, so $\Delta(\mathcal{S}) \leq I(\mathcal{S})$. □

The inequality can be strict. Collections with many incomparable pairs whose symmetric difference is small are where $\Delta(\mathcal{S})$ outperforms $I(\mathcal{S})$. When no containment is available, the insertion hierarchy is forced to treat the set as a fresh root, paying $|S|$ in full, whereas the symdiff graph connects the set to its nearest neighbour at cost $|S \triangle \cdot|$.

4.3.3 Adapted Prim construction

A naive computation materialises all $\binom{m+2}{2}$ pairwise symdiff weights for the classical Prim on the dense graph K. This is wasteful when $\Delta(\mathcal{S}) \ll m^2$. The algorithm of Gagie et al. [9] discovers the weights incrementally from smallest to largest, and stops iterating on a pair as soon as its weight exceeds the current MST threshold.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

The MST characterisation of Lemma 4.3.1 reduces the computation of $\Delta(\mathcal{S})$ to a minimum spanning tree on the augmented complete graph K with $m+2$ nodes. Evaluating all pairwise symmetric differences explicitly costs $O(m^2 u)$ in the worst case, which is extravagant when $\Delta(\mathcal{S})$ is much smaller than m^2. Gagie, He, and Navarro [9] show that the MST can be computed by an adapted Prim algorithm that evaluates each pairwise symmetric difference only partially, stopping as soon as the comparison exceeds the current MST frontier threshold. The running time is a function of how much pairwise symdiff volume actually lies below the heaviest MST edge, not of the total volume.

Suspended Prim. The adaptation rests on a simple invariant of Prim's algorithm. At any point of Prim's execution, let k be one more than the maximum edge weight included in the MST so far. Prim will never add an edge of weight at least k while any edge of weight strictly less than k remains available from the current MST frontier. Consequently, Prim is correct when it is allowed to operate on the subset of edges of weight strictly less than k, suspending whenever that subset is exhausted and resuming after new edges of weight up to k become known.

Proximity iterator. The weight discovery is driven by a proximity iterator built on a generalised suffix tree of the concatenation of all sets. The sets are sorted in increasing order of their elements in time $O(n \lg u)$; each set S_i is appended a unique terminator $\$_i$; the concatenation $T(\mathcal{S}) = S_1\$_1 \cdots S_m\$_m$ has length $O(n)$. Its generalised suffix tree [56] is built in $O(n)$ time [57] with constant-time lowest-common-ancestor queries [38]. The LCA of two leaves returns the longest common prefix of the corresponding suffixes, which translates into a merge step that identifies the next element of $S_i \triangle S_j$ in constant time. The guarantee is a single-step iterator that advances by one element of the symmetric difference at each call.

[56]: Apostolico (1985), *The myriad virtues of subword trees*

[57]: Farach-Colton et al. (2000), *On the sorting-complexity of suffix tree construction*

[38]: Bender et al. (2000), *The LCA Problem Revisited*

Lemma 4.3.3 (Proximity iterator[9]) *After an $O(n \lg u)$-time preprocessing of $\mathcal{S}$, there is a data structure supporting the following operation. Given two pointers p, q into a pair of sorted sets S, S', each initialised to the first element, a single call advances p or q by one position and either reports one new element*

of $S \triangle S'$ or detects that both pointers have reached their terminators. After k successful calls on the pair (S, S'), either both pointers point to terminators, in which case $|S \triangle S'| < k$, or k distinct elements of $S \triangle S'$ have been reported, in which case $|S \triangle S'| \geq k$. Each call runs in $O(1)$ time.

With the iterator in place, Prim is run in phases indexed by an integer threshold k. Phase k advances every still-active iterator by one step; pairs that exhaust both pointers have their final weight revealed and are fed to Prim, which consumes all available edges of weight strictly less than k before the next phase begins. The pseudocode appears in Algorithm 1.

Algorithm 1: Adapted Prim for $\Delta(\mathcal{S})$ and the minimum-weight symdiff graph, after Gagie, He, and Navarro [9].

Input: A collection $\mathcal{S}$ (sorted) with the proximity iterator of Lemma 4.3.3 preprocessed.
Output: The minimum-weight symdiff graph on $\mathcal{S} \cup \{\emptyset, U\}$.

```
V ← {∅, U} with the zero-weight edge (∅, U) in the MST; V' ← S;
Bag B ← all pairs of nodes in V ∪ V' except (∅, U), with weight
  marked unknown; each pair carries an iterator at p = q = 1;
for k = 1, 2, 3, ... do
    foreach edge (A, B) ∈ B still in play do
        advance the proximity iterator of (A, B) by one step;
        if both pointers of (A, B) reached their terminators then
            record w(A, B) := k − 1 in Prim's structures and remove
              (A, B) from B;
    while Prim's frontier contains an edge of weight < k into V' do
        extract the lightest such edge (u, v), move v from V' to V,
          add (u, v) to the MST;
    if V' = ∅ then
        return the MST;
```

Theorem 4.3.4 (Construction of $\Delta(\mathcal{S})$[9]) *Let $\mathcal{S}$ be a collection of m sets over a universe of size u, with total size n. Let ℓ be the maximum edge weight in the minimum-weight symdiff graph on $\mathcal{S}$. The adapted Prim algorithm computes $\Delta(\mathcal{S})$ and the minimum-weight symdiff graph in time*

$$O\left(n \lg u + \sum_{S,S' \in \mathcal{S}} \min(\ell, |S \triangle S'|)\right) \subseteq O\big(n \lg u + \min(m^2\ell, mn)\big).$$

Proof. The $O(n \lg u)$ term is the preprocessing cost of Lemma 4.3.3. For the second term, every pair $(S, S') \in \mathcal{S}^2$ has its proximity iterator advanced $\min(\ell, |S \triangle S'|)$ times. At most ℓ times because the outer loop suspends Prim once the threshold reaches ℓ and the MST is complete, and at most $|S \triangle S'|$ times because once the iterator exhausts the symmetric difference both pointers reach their terminators. The total iterator work therefore equals $\sum_{S,S'} \min(\ell, |S \triangle S'|)$. Prim's own bookkeeping is dominated by this cost. The two worst-case bounds arise from $\min(\ell, |S \triangle S'|) \leq \ell$ and from $|S \triangle S'| \leq |S| + |S'|$. □

The adapted Prim improves strictly on the naive $O(m^2 u)$ whenever $\ell \ll u$, that is, whenever the minimum-weight symdiff graph is dominated by short edges. For the collections where $\Delta(\mathcal{S})$ is interesting in practice,

such as adjacency lists of web graphs or colour sets of de Bruijn graphs, the MST edges have weight far below u, and the construction cost is governed by $m\ell$ rather than by mu.

The same iterator accelerates the construction of the insertion graph of Subsection 4.2.3, which Proposition 4.2.8 bounded at $O(n \lg u + mn)$ time via merge-based DFS containment checks. Given a new set S, we maintain a bag of already-inserted candidates $S' \in \mathcal{S}$ and advance the iterator of Lemma 4.3.3 across all pairs (S, S') in lockstep. As soon as the iterator reports an element lying in $S' \setminus S$, the candidate violates $S' \subset S$ and leaves the bag; the iteration continues until the first surviving candidate reaches both terminators, identifying $p'(S)$ and the elements of $S \setminus p'(S)$. The per-set cost is $O(m \cdot |S \setminus p'(S)|)$, summing to $O(m \cdot I(\mathcal{S}))$ across all insertions on top of the $O(n \lg u)$ preprocessing of the suffix tree.

Proposition 4.3.5 (Improved insertion-graph construction [9]) *The insertion graph and the insertion tree of a collection $\mathcal{S}$ with m sets and total size n over a universe of size u can be built in $O(n \lg u + m \cdot I(\mathcal{S})) \subseteq O(n \lg u + mn)$ time on a word RAM with ω-bit words.*

4.3.4 Indel trees

Where the insertion tree of Definition 4.2.5 carries only positive labels drawn from $U = [1..u]$, the indel tree carries labels from $[-u..-1] \cup [1..u]$. Doubling the alphabet is the price of allowing deletions.

The minimum-weight symdiff graph splits into two trees anchored at $\emptyset$ and U, and each edge $(p(S), S)$ carries a weight that decomposes as $|S \triangle p(S)| = |S \setminus p(S)| + |p(S) \setminus S|$ insertions and deletions. Unlike the insertion tree of Definition 4.2.5, whose edges carry only insertions, the indel tree of [9] carries both kinds of edit. Translating a weighted edge into a chain of labelled edges therefore requires two label classes, one for insertions and one for deletions.

Definition 4.3.3 (Indel trees [9]) *Let $\mathcal{S}$ be a collection, with $p(S)$ the neighbour of S in a minimum-weight symdiff graph of $\mathcal{S}$. The indel trees of $\mathcal{S}$ are two rooted labelled ordinal trees. The root of one tree is $v(\emptyset)$, the root of the other is $v(U)$. For every $S \in \mathcal{S}$ there is a node $v(S)$, lying in the same tree as $v(p(S))$. The edge connecting $v(p(S))$ to $v(S)$ is replaced by a chain of $|S \triangle p(S)|$ edges, each labelled with a distinct element of the insertion set $S \setminus p(S)$ or the deletion set $p(S) \setminus S$, in arbitrary order. Labels from the insertion set are written $+x$ for $x \in S \setminus p(S)$; labels from the deletion set are written $-y$ for $y \in p(S) \setminus S$. The label alphabet is $[-u..-1] \cup [1..u]$, of size $2u$ (Gagie et al. write this as $[-u..u]$; the label 0 is unused since $0 \notin U$).*

Proposition 4.3.6 (Indel-tree structure[9]) *The two indel trees of $\mathcal{S}$ together contain $\Delta(\mathcal{S}) + 2$ nodes. For every $S \in \mathcal{S}$, the content of S is recovered from the root of its tree by walking the path from the root to $v(S)$ and applying each label in order. An insertion label $+x$ adds x to the running set, and a deletion label $-y$ removes y.*

Proof. The two roots contribute two nodes. Each $S \in \mathcal{S}$ contributes a chain of $|S \triangle p(S)|$ edges, hence $|S \triangle p(S)|$ internal nodes. Summing over $\mathcal{S}$ adds $\sum_S |S \triangle p(S)| = \Delta(\mathcal{S})$ nodes. The decoding semantics follows by induction on depth. At the root the running set is $\emptyset$ or U as given. An insertion label $+x$ increments the running set by x; a deletion label $-y$

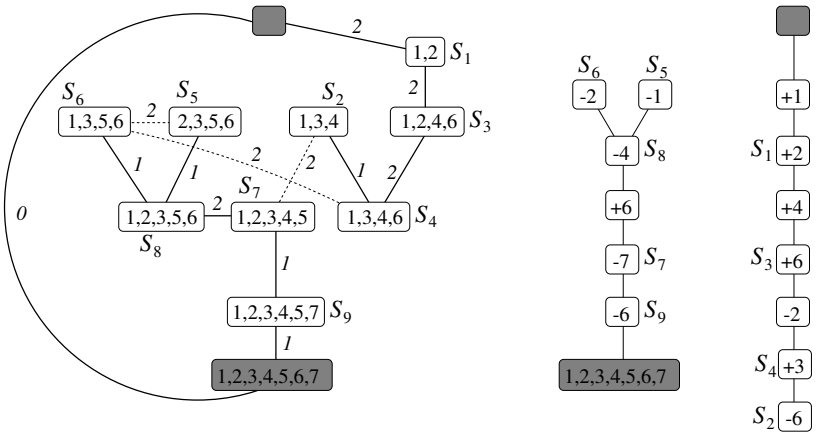

Figure 4.2: Reproduced from Gagie, He, and Navarro [9]. Left: a minimum-weight symdiff graph on $\mathcal{S} = \{S_1, \ldots, S_9\}$ with $\Delta(\mathcal{S}) = 13$. Dashed edges are available in the complete graph K but not used by the MST; they are the edges whose weight exceeds the largest MST edge weight $\ell = 2$. Right: the two indel trees rooted at U (drawn upside down) and at $\emptyset$. Chain labels $+x$ are insertions, $-x$ are deletions relative to the parent.

decrements it by y. By Definition 4.3.1, the labels on the chain from $v(p(S))$ to $v(S)$ realise exactly the symmetric difference between $p(S)$ and S, so the running set at $v(S)$ equals $p(S) \cup (S \setminus p(S)) \setminus (p(S) \setminus S) = S$. □

Figure 4.2 shows the minimum-weight symdiff graph of a nine-set collection, together with the two indel trees obtained by chain expansion. The MST edges have total weight $\Delta(\mathcal{S}) = 13$, and the two trees together have 15 nodes.

The indel trees realise $\Delta(\mathcal{S})$ as a pair of labelled ordinal trees over an alphabet of size $2u$. Every set of $\mathcal{S}$ is recovered as a root-to-$v(S)$ path, and the total label count is $\Delta(\mathcal{S})$.

4.3.5 Indel-tree queries

The indel trees of Definition 4.3.3 pick up exactly the case that the insertion tree could not compress. Each edge $(v(p(S)), v(S))$ is expanded into a chain of $|S \triangle p(S)|$ edges, labelled with the distinct elements of $S \setminus p(S)$ (marked $+x$) and of $p(S) \setminus S$ (marked $-y$), as recorded in Proposition 4.3.6 and illustrated by Figure 4.2. The collection splits across two trees rooted at $v(\emptyset)$ and $v(U)$, and the label alphabet becomes $[-u..-1] \cup [1..u]$. Every operation of Definition 1.2.2 on S must now reconstruct the running set at $v(S)$ from the sequence of insertions and deletions along the root-to-$v(S)$ path.

The insertion tree forced a single class of edge marks, all positive. The indel tree carries two classes, $+x$ for insertions and $-x$ for deletions, at the price of a doubled label alphabet. Asymptotically $\lg(2u) = \lg u + 1$, so every time bound that depends on $\lg u$ survives the doubling unchanged.

We overlay the two indexes of Section 3.5 on each of the two trees. The α-operations index of Theorem 3.6.2, instantiated at alphabet $[-u..-1] \cup [1..u]$, supports $\texttt{parent}_\alpha$ for every signed label α. The path-queries index of Theorem 3.7.4, instantiated at the same alphabet, supports `path_count` and `path_select` for arbitrary label ranges. Each index stores a labeled ordinal tree of $|T|$ nodes in $O(|T|)$ words, and $|T^\emptyset| + |T^U| = \Delta(\mathcal{S}) + 2$ by Proposition 4.3.6, so the overlay occupies $O(\Delta(\mathcal{S}))$ words in total. The alphabet size $2u$ enters the query bounds only through $\lg(2u) = \lg u + 1$, which leaves every asymptotic in $\lg u$ and $\lg \lg_\omega u$ unchanged.

Membership on (S, x) reduces to two α-operations. Let $u^+ = \texttt{parent}_{+x}(v(S))$ and $u^- = \texttt{parent}_{-x}(v(S))$, both returned by the α-operations index. If both exist, the last edit that affected x on the root-to-$v(S)$ path is the deeper of the two, so $x \in S$ iff $\text{depth}(u^+) > \text{depth}(u^-)$. If only u^+ exists, x was inserted and never removed, so $x \in S$. If only u^- exists, x was

removed and never reinstated, so $x \notin S$. If neither exists, no label for x appears on the path, so $x \in S$ iff $v(S)$ descends from $v(U)$, since the elements of U are implicitly present at the root of that tree. The two α-operations and the depth comparison run in $O(\lg \lg_\omega u)$ time.

Rank on (S, x) is a difference of two path counts. The ancestors of $v(S)$ with label in $[1..x]$ record the elements of $[1..x]$ inserted along the path, and the ancestors with label in $[-x.. -1]$ record those deleted; the rank is the excess of insertions over deletions. When $v(S)$ descends from $v(U)$, the root holds all of $[1..x]$ implicitly, so the excess must be added to x. Both counts are single calls to `path_count` of Theorem 3.7.4, and the total cost is $O(\lg_\omega u)$.

Access is the one operation where the reductions used for the insertion tree break down. Path selection on the signed alphabet returns a position that mixes insertions and deletions, not the i-th surviving element of S. A separate algorithm, developed in the coordinated-descent paragraph below, achieves access(S, i) in $O(\lg u)$ time by descending two parallel hierarchies in lockstep.

Predecessor and successor follow the rank-then-access reduction recalled in Section 1.2. Each performs one rank and one access, and the access term $O(\lg u)$ dominates.

Theorem 4.3.7 (Indel tree representation [9]) *Let $\mathcal{S}$ be a collection of subsets of $[1..u]$ and assume the word RAM model with word size $\omega = \Omega(\lg n)$. The two indel trees of $\mathcal{S}$ admit a joint representation in $O(\Delta(\mathcal{S}))$ words of space supporting, for every $S \in \mathcal{S}$, the operations of Definition 1.2.2 within the following time bounds.*

- member(S, x) *in time* $O(\lg \lg_\omega u)$.
- rank(S, x) *in time* $O(\lg_\omega u)$.
- access(S, i) *in time* $O(\lg u)$.
- predecessor(S, x) *in time* $O(\lg u)$.
- successor(S, x) *in time* $O(\lg u)$.

Coordinated descent

Access on (S, i) asks for the i-th smallest element x of S. A direct binary search on $[1..u]$ driven by rank queries on $v(S)$ would probe $\lceil \lg u \rceil$ candidate values and spend $O(\lg_\omega u)$ per probe, for a total of $O(\lg u \cdot \lg_\omega u)$. The algorithm of this subsection removes the extra $\lg_\omega u$ factor by replacing external rank probes with an internal descent on hierarchical partitions built by tree extraction, so that every level takes $O(1)$ time.

For each indel tree $\mathcal{T}$ rooted at $v(\emptyset)$ or $v(U)$, extract two subtrees by the binary labels $\{+, -\}$. The first subtree $\mathcal{T}^+$ retains only the nodes with insertion labels, and the second $\mathcal{T}^-$ retains only those with deletion labels. Both are obtained through the tree-extraction operation of Definition 3.5.1 with the associated image map $f_{\mathcal{T}}$. Within each of $\mathcal{T}^+$ and $\mathcal{T}^-$, apply the hierarchical binary universe partition of Subsection 3.7.1 on $[1..u]$, halving the label range at every level, producing a family of binary-labeled trees $\mathcal{T}^+_{a,b}$ and $\mathcal{T}^-_{a,b}$ indexed by the current range. Figure 4.3 diagrams the extraction for one of the two hierarchies; $\mathcal{T}^+$ is the parallel instance built

on insertion marks. The representation of Theorem 3.6.2 at alphabet size $\sigma = 2$ stores each level in $O(|\mathcal{T}^{\pm}|)$ bits; telescoping over the $O(\lg u)$ levels gives $O(\Delta(\mathcal{S}) \lg u)$ bits of additional state, which is $O(\Delta(\mathcal{S}))$ words.

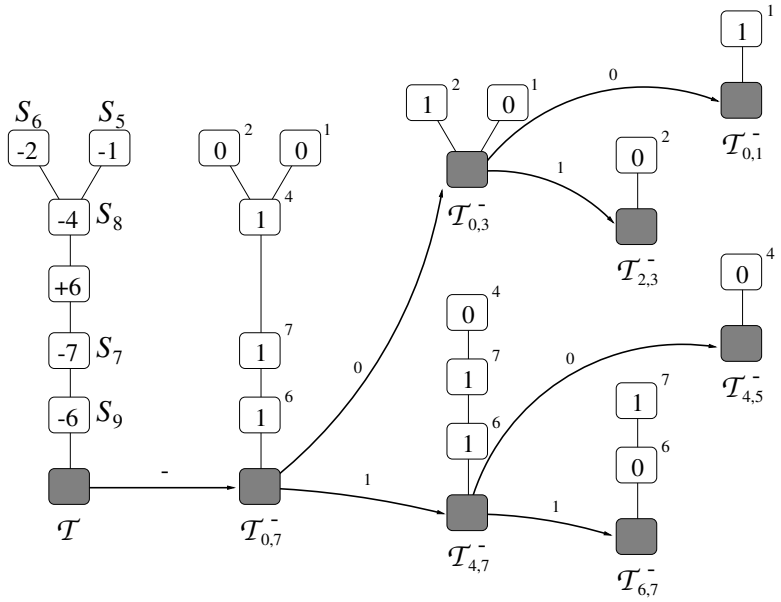

Figure 4.3: Hierarchical extraction of one indel-tree hierarchy, reproduced from Gagie, He, and Navarro [9]. The diagram shows GHN's worked example $\mathcal{T}^- = \mathcal{T}^-_{0,7}$ obtained from the indel tree rooted at $v(U)$ of Figure 4.2 (drawn upside down); the subscript [0..7] reflects GHN's 0-indexed eight-element universe, while the thesis prose uses $[1..u]$. At each intermediate range $[a..b]$, the 0/1-labelled tree $\mathcal{T}^-_{a,b}$ extracts to $\mathcal{T}^-_{a,m}$ on label 0 and to $\mathcal{T}^-_{m+1,b}$ on label 1, following the bold rightward arrows. The same construction yields $\mathcal{T}^+$ on insertion marks; the coordinated descent of this subsection tracks images v^+ and v^- in two parallel instances of this structure.

Algorithm 2 states the loop. Two facts drive the analysis. First, both rank_0 calls run in $O(1)$ by Theorem 3.6.2 at $\sigma = 2$, since the alphabet of $\mathcal{T}^{\pm}_{a,b}$ is $\{0, 1\}$. Second, the count $s^+ - s^-$ records the number of elements of S with value in $[a..m]$ when $\mathcal{T}$ is rooted at $v(\emptyset)$; the additive correction $b - a + 1$ when $\mathcal{T}$ is rooted at $v(U)$ accounts for the elements of $[a..b]$ that start as implicitly present at the root and are removed only by deletions along the path.

Correctness follows from the loop invariant that, at every level, i is the rank of x among the elements of S in $[a..b]$. At initialisation the invariant holds because x is the i-th smallest element of S and $[a..b] = [1..u]$. The branch rule preserves it because, by the insertion-minus-deletion observation of Gagie et al., every element of S in $[a..m]$ corresponds to exactly one net insertion on the path from the root of $\mathcal{T}$ to v; the left-half count $s^+ - s^-$ (with the $v(U)$-root correction $b - a + 1$) realises this counting as an instance of the depth identity of Lemma 3.5.3 applied to $\mathcal{T}^{\pm}_{a,b}$, where $\text{rank}_0(v^{\pm})$ equals the depth of the image of v in the corresponding extracted subtree.

Each level performs two $O(1)$ rank queries, two $O(1)$ tree-extraction remappings, and one arithmetic comparison, for $O(1)$ time overall. The number of levels is $\lceil \lg u \rceil$, so the algorithm runs in $O(\lg u)$ time.

The $O(\lg u)$ bound is structural rather than a matter of calibration. The sublogarithmic path-selection mechanism of Theorem 3.7.4 hinges on an auxiliary array indexed by the tier-1 roots [46] of a single labeled hierarchy, which packs partial rank sections across the f' child ranges of every tier-1 root into a constant number of machine words and answers each level by word-parallel comparison. The coordinated descent of this subsection tracks two independent images $v^+ \in \mathcal{T}^+_{a,b}$ and $v^- \in \mathcal{T}^-_{a,b}$ at every level, and the per-level count is a difference $s^+ - s^-$ drawn from the two hierarchies. A joint auxiliary array would need to be indexed simultaneously by tier-1 roots of $\mathcal{T}^+$ and of $\mathcal{T}^-$ with compatible packing

Path counting on the signed alphabet factors through a single partition of $[-u..u]$, so one hierarchy suffices and Theorem 3.7.4 applies directly. Path selection does not, because the tier-1 auxiliary array sits inside a single hierarchy.

[46]: He et al. (2016), *Data structures for path queries*

Algorithm 2: Coordinated descent for access(S, i) on indel trees, after Gagie, He, and Navarro [9].

Input: Indel tree $\mathcal{T}$ rooted at $v(\emptyset)$ or $v(U)$, node $v = v(S)$, rank $i \in [1..|S|]$.
Output: The i-th smallest element of S.

$a \leftarrow 1$; $b \leftarrow u$;
$v^+ \leftarrow f_{\mathcal{T}}(v, +) \in \mathcal{T}^+_{1,u}$;
$v^- \leftarrow f_{\mathcal{T}}(v, -) \in \mathcal{T}^-_{1,u}$;
while $a < b$ **do**
  $m \leftarrow \lfloor (a + b)/2 \rfloor$;
  $s^+ \leftarrow \mathrm{rank}_0(v^+)$ in $\mathcal{T}^+_{a,b}$;
  $s^- \leftarrow \mathrm{rank}_0(v^-)$ in $\mathcal{T}^-_{a,b}$;
  $c \leftarrow s^+ - s^-$;
  if $\mathcal{T}$ *is rooted at* $v(U)$ **then**
    $c \leftarrow c + (b - a + 1)$;
  if $i \le c$ **then**
    $b \leftarrow m$;
    $v^+ \leftarrow f^+_{a,b}(v^+, 0)$; $v^- \leftarrow f^-_{a,b}(v^-, 0)$;
  else
    $a \leftarrow m + 1$;
    $v^+ \leftarrow f^+_{a,b}(v^+, 1)$; $v^- \leftarrow f^-_{a,b}(v^-, 1)$;
    $i \leftarrow i - c$;
return a;

of the difference, which no known construction supports. The obstruction is the signed-label decomposition itself.

The two regimes of this chapter, insertion and indel, both pay the per-set baseline $H_{wc}(\mathcal{S})$ in the worst case. Neither says how far the achieved cost sits from the information-theoretic optimum across all collections. The next chapter develops a lower bound based on the membership patterns of universe elements, sharpens the picture of attainable compression, and introduces a polynomial-time greedy construction that augments the candidate hierarchy with synthetic union nodes.

5 Set-Union Matching

5.1 Information theoretic bound 71
5.2 Union matching 76
5.3 Queries 83

The encodings of Chapter 4 exploit relations that can be seen between sets. If $Q \subseteq P$, we store what disappears from P to obtain Q. If B differs little from A, we store the small change. If several sets are close along a tree, we share the common parts. These constructions save space after a relation has been chosen. We first want a benchmark that depends only on the membership table of the collection.

The membership table records, for each element of U, which sets contain it. Elements with the same membership row are interchangeable, so U splits into Venn regions. Once the sizes of these regions are fixed, describing the collection amounts to assigning a region label to each labelled element of U. Counting these assignments gives the first bound of the chapter.

The count treats the collection as one global string over Venn-region labels. It gives a clean lower bound, but it gives no route for queries. Membership, rank, and access should be answered by reading a small set of local changes. If a query has to reconstruct the entire global string first, the encoding has lost the property that made the hierarchies of Chapter 4 useful. We therefore look for a local step whose cost still reflects the Venn-region count.

Containment gives such a step. From a parent P to a child $Q \subseteq P$, the edge stores the deleted elements $P \setminus Q$, and path queries count or select those deletions. Two incomparable sets have no containment edge between them, but their union contains both. Adding the union creates a common parent and turns the pair into two containment edges. This move can be scored locally, repeated across the collection, and stored in the query structure.

The same idea admits a wider search space. We could add references beyond binary unions, or allow one reference to split into several children. Appendix A studies that wider model. It shows which compression opportunities the pairwise route discards and why gaining them requires a harder reference-selection problem.

5.1 Information theoretic bound

Fix the Venn regions that occur in $\mathcal{S}$ and fix their sizes. After this information is known, the remaining freedom is the assignment of the labelled elements of U to those regions. Each assignment determines the collection. An element placed in the region labelled by $\tau \subseteq [m]$ belongs exactly to the sets S_i with $i \in \tau$. Conversely, every collection with the same occurring regions and the same region sizes gives one assignment. The number of assignments is therefore the size of the class that the representation must distinguish. Its logarithm is a lower bound in bits, and enumerative coding reaches the same count up to ceilings. This benchmark is conditional, since it counts only the assignment of elements

For $\mathcal{S} = \{S_1 = \{a, b\}, S_2 = \{b, c\}\}$ over $U = \{a, b, c\}$, the partition has three Venn regions, $\{a\}$ in S_1 only, $\{b\}$ in both S_1 and S_2, and $\{c\}$ in S_2 only.

to regions after the decoder receives the region labels and their sizes. A complete data structure must also store those labels and sizes, together with the auxiliary indices used by the queries.

5.1.1 Membership patterns

For each element $x \in U$, the membership table gives a subset of indices. This subset records exactly the sets that contain x. Equal subsets place elements in the same Venn region, and different subsets place them in different regions. The counting argument uses both the subset and the region it determines.

Definition 5.1.1 (Membership pattern) *The membership pattern of an element $x \in U$ relative to $\mathcal{S} = \{S_1, \ldots, S_m\}$ is the set*

$$\sigma(x) \;=\; \{\, i \in [m] : x \in S_i \,\}.$$

A Venn region should be indivisible by the sets available to the collection. The standard algebraic terminology for such an indivisible nonzero element is atom.

Definition 5.1.2 (Atom [58]) *Let $\mathcal{B}$ be a Boolean algebra with least element 0. An atom of $\mathcal{B}$ is an element $a \neq 0$ such that the only elements $b \in \mathcal{B}$ satisfying $b \leq a$ are $b = 0$ and $b = a$.*

For $\mathcal{S} = \{S_1, \ldots, S_m\} \subseteq 2^U$, let $\mathcal{B}(\mathcal{S})$ be the Boolean algebra of subsets of U generated by $S_1, \ldots, S_m$. For every nonempty $\tau \subseteq [m]$, set

$$A_\tau \;=\; \{\, x \in U : \sigma(x) = \tau \,\}.$$

If $A_\tau \neq \emptyset$, we call A_τ the atom of pattern τ.

The definition applies because the sets $S_1, \ldots, S_m$ act on each nonempty A_τ all at once. For each index i, either $A_\tau \subseteq S_i$ or $A_\tau \cap S_i = \emptyset$, according to whether $i \in \tau$. The same all or nothing behaviour holds for every set obtained from $S_1, \ldots, S_m$ by unions, intersections, and complements. Thus no set in $\mathcal{B}(\mathcal{S})$ can split a nonempty A_τ into two nonempty parts.

In the running example, $\sigma(a) = \{1\}$, $\sigma(b) = \{1, 2\}$, and $\sigma(c) = \{2\}$. The atoms are $A_{\{1\}} = \{a\}$, $A_{\{1,2\}} = \{b\}$, and $A_{\{2\}} = \{c\}$, all of size one.

The convention $U = \bigcup_i S_i$ fixed in Section 4.1 removes the empty pattern. Every $x \in U$ belongs to at least one S_i, so $\sigma(x) \neq \emptyset$. The possible patterns are the nonempty subsets of $[m]$, hence at most $2^m - 1$ sets A_τ can be nonempty. If $u = 0$, all multinomial coefficients are one and every bit bound is zero. We assume $u > 0$ from now on. The nonempty sets A_τ form a partition of U, and their cardinalities sum to u.

Let $R(\mathcal{S}) = \{\, \tau \subseteq [m] : \tau \neq \emptyset,\ A_\tau \neq \emptyset \,\}$ denote the set of realised patterns and write $k = |R(\mathcal{S})|$. This number ranges from 1, when all sets coincide, to $2^m - 1$, when every nonempty Venn region is realised. Enumerate $R(\mathcal{S}) = \{\tau_1, \ldots, \tau_k\}$ and set $c_{\tau_j} = |A_{\tau_j}|$. Fixing $R(\mathcal{S})$ and the vector $(c_{\tau_j})_{j=1}^{k}$ still leaves the labelled elements of U to place into the pattern classes. Every labelling of the u elements by patterns in $R(\mathcal{S})$ with prescribed counts $(c_\tau)_{\tau \in R(\mathcal{S})}$ produces one collection, and every collection with these parameters gives the labelling $x \mapsto \sigma(x)$. If the decoder receives these parameters outside the bit count, a fixed length representation must

distinguish all such labellings. The same lower bound applies to the longest codeword of any prefix code for the class.

If the decoder receives only $|U|$ and m, the count vector is no longer fixed. Each labelled element of U can receive any of the $2^m - 1$ nonempty patterns, independently of the other elements. The number of indexed collections with universe coverage U is therefore exactly $(2^m - 1)^u$, so any fixed length representation that distinguishes all such collections uses at least

$$\lceil \lg(2^m - 1)^u \rceil \;=\; \lceil u \lg(2^m - 1) \rceil$$

bits. Equivalently, every prefix code for this class has a codeword of at least this length. This coarse bound depends only on u and m. Supplying the realised patterns and their counts narrows the class from all pattern strings to the class with fixed counts whose exact size is computed next.

5.1.2 Atom bound

Once the realised patterns and their counts are fixed, only the positions of the patterns on the labelled universe remain. A collection with these parameters is a labelling of the u elements of U by patterns in $R(\mathcal{S})$, using each pattern τ exactly c_τ times. The number of such labellings is the multinomial coefficient

$$\binom{u}{(c_\tau)_{\tau \in R(\mathcal{S})}} \;=\; \frac{u!}{\prod_{\tau \in R(\mathcal{S})} c_\tau!},$$

obtained by ordering the u elements in $u!$ ways and dividing by the $\prod_\tau c_\tau!$ permutations within each pattern class that produce the same labelling.

The atom counts (c_τ) are strictly finer side information than the row sizes ($|S_i|$). The row sizes follow from the atom counts via $|S_i| = \sum_{\tau \ni i} c_\tau$, but multiple atom count configurations can yield the same row sizes.

Proposition 5.1.1 (Atom bound) *Let $R(\mathcal{S}) = \{\tau_1, \ldots, \tau_k\}$ be the realised patterns of $\mathcal{S}$ and $c_{\tau_j} = |A_{\tau_j}|$ the atom counts. Set*

$$L^*(\mathcal{S}) \;=\; \lg \binom{u}{c_{\tau_1}, \ldots, c_{\tau_k}}.$$

Given the side information $(R(\mathcal{S}), (c_{\tau_j})_{j=1}^k)$ outside the bit count, every fixed length representation that distinguishes all indexed collections sharing these parameters uses at least $\lceil L^*(\mathcal{S}) \rceil$ bits. The longest codeword of every prefix code for the same class has length at least $\lceil L^*(\mathcal{S}) \rceil$ bits.

Proof. Let $\mathcal{C}(R, (c_\tau))$ denote the class of indexed collections

$$\mathcal{S}' = \{S'_1, \ldots, S'_m\}$$

over U whose realised pattern set is R and whose atom counts match (c_τ). The map sending $\mathcal{S}' \mapsto \ell_{\mathcal{S}'}$ with $\ell_{\mathcal{S}'}(x) = \{i \in [m] : x \in S'_i\}$ takes $\mathcal{C}(R, (c_\tau))$ into the set of pattern labellings $\ell \colon U \to R$ in which τ occurs exactly c_τ times. Its inverse sends a labelling ℓ back to the collection with $S'_i = \{x \in U : i \in \ell(x)\}$, whose realised pattern set is the image of ℓ (which equals R since every $\tau \in R$ has $c_\tau \geq 1$) and whose atom counts are (c_τ). The two maps are inverses, hence $|\mathcal{C}(R, (c_\tau))| = \binom{u}{(c_\tau)}$.

Any prefix code that assigns a distinct codeword to every member of $\mathcal{C}(R, (c_\tau))$ has a codeword of length at least $\lceil \lg |\mathcal{C}(R, (c_\tau))| \rceil = \lceil L^*(\mathcal{S}) \rceil$ bits. Enumerating the members of $\mathcal{C}(R, (c_\tau))$ in a fixed order gives a fixed length code of $\lceil \lg |\mathcal{C}(R, (c_\tau))| \rceil$ bits. □

Enumerative coding of fixed count classes is treated in Cover and Thomas [59, Ch. 11].

Proposition 5.1.1 gives a conditional benchmark. It charges the choice of positions inside fixed pattern classes, and it leaves the cost of describing those classes to the complete representation.

5.1.3 Coarser bounds

The bound $L^*(\mathcal{S})$ assumes that the realised patterns and their counts are outside the bit count. If the decoder receives less information, the representation must distinguish a larger class. Two weaker decoder promises give coarser benchmarks. The first keeps only the row sizes $|S_i|$ and gives the independent row size benchmark $H_{wc}(\mathcal{S})$. The second keeps only the universe size and gives the coverage count $u \lg(2^m - 1)$.

The first weakening keeps row sizes and forgets the rest of the pattern data. Once the sets A_τ have prescribed sizes, every row size is forced by $|S_i| = \sum_{\tau \ni i} c_\tau$. A code that knows only the row sizes sees a larger class, because it also has to distinguish row tuples whose pattern counts differ from those of $\mathcal{S}$.

Proposition 5.1.2 *For every collection $\mathcal{S}$ with row sizes $(|S_i|)_{i=1}^m$,*

$$L^*(\mathcal{S}) \;\le\; H_{wc}(\mathcal{S}).$$

Proof. Let R be the realised pattern set and (c_τ) the atom counts of $\mathcal{S}$. Consider one of the collections counted by the multinomial defining $L^*(\mathcal{S})$, and write $(A'_\tau)_{\tau \in R}$ for its sets of elements with pattern τ. It determines an m-tuple of rows $(S'_1, \dots, S'_m)$ by

$$S'_i \;=\; \bigcup_{\tau \ni i} A'_\tau.$$

The row sizes satisfy $|S'_i| = \sum_{\tau \ni i} c_\tau = |S_i|$, so the tuple belongs to the class counted by $H_{wc}(\mathcal{S})$.

The map is injective. Given the m-tuple $(S'_1, \dots, S'_m)$, the set A'_τ is recovered as $\bigcap_{i \in \tau} S'_i \cap \bigcap_{i \notin \tau} (U \setminus S'_i)$. Thus the partition $(A'_\tau)_{\tau \in R}$ is determined by the tuple, and distinct partitions map to distinct tuples.

Comparing the cardinality of the domain with the cardinality of the target gives

$$\binom{u}{(c_\tau)} \;\le\; \prod_{i=1}^{m} \binom{u}{|S_i|},$$

and taking logarithms to base 2 yields $L^*(\mathcal{S}) \le H_{wc}(\mathcal{S})$. □

The second weakening forgets the row sizes as well. If k pattern labels occur, then the sequences counted by $L^*(\mathcal{S})$ are exactly the strings over this alphabet with prescribed frequencies. They form a subset of all k^u strings over the same alphabet. Replacing k by its largest possible value $2^m - 1$ gives a bound that depends only on u and m.

Proposition 5.1.3 *If $\mathcal{S}$ realises k patterns, with $1 \le k \le 2^m - 1$, then*

$$L^*(\mathcal{S}) \;\le\; u \lg k \;\le\; u \lg(2^m - 1).$$

Proof. The multinomial coefficient satisfies

$$\binom{u}{(c_\tau)} = \frac{u!}{\prod_\tau c_\tau!} \le k^u$$

since the left side counts pattern sequences of length u over an alphabet of size k with prescribed frequencies, and k^u counts all pattern sequences of length u over the same alphabet. Taking logarithms gives $L^*(\mathcal{S}) \le u \lg k$. Under the convention of Section 4.1, the empty pattern is excluded, so $k \le 2^m - 1$ and the second inequality follows. □

5.1.4 Achievability and its limits

$L^*(\mathcal{S})$ is the logarithm of a finite class of pattern sequences. The same class can be used as an address space. Once the decoder knows the realised patterns and their counts, the encoder lists the membership patterns in the order of the universe and stores the enumerative index of that sequence. This reaches the lower bound up to the ceiling. The statement concerns only the codeword length, and it excludes the cost of storing the side information and the indices used by queries.

Bit space tightness concerns the codeword length alone. The achieving encoding stores one integer identifying the full pattern sequence.

Proposition 5.1.4 (Achievability of L^*) *For every collection $\mathcal{S}$ under the convention $U = \bigcup_i S_i$, there is an encoding of $\mathcal{S}$ of length exactly $\lceil L^*(\mathcal{S}) \rceil$ bits given the side information $(R(\mathcal{S}), (c_\tau)_{\tau \in R(\mathcal{S})})$.*

Proof. Order the universe as $U = \{x_1, \dots, x_u\}$ and form the pattern sequence $\pi(\mathcal{S}) = (\sigma(x_1), \dots, \sigma(x_u))$ over the alphabet $R(\mathcal{S})$. By the bijection in the proof of Proposition 5.1.1, the map $\mathcal{S} \mapsto \pi(\mathcal{S})$ is a bijection between indexed collections with parameters $(R, (c_\tau))$ and sequences of length u over $R(\mathcal{S})$ with prescribed frequencies (c_τ). Enumerative coding assigns a distinct codeword of length $\lceil \lg \binom{u}{(c_\tau)} \rceil = \lceil L^*(\mathcal{S}) \rceil$ bits to each sequence.

The decoder, given the side information and the codeword, reconstructs $\pi(\mathcal{S})$ and then sets $S_i = \{x_j : i \in \sigma(x_j)\}$ for each $i \in [m]$, recovering $\mathcal{S}$. □

The code of Proposition 5.1.4 stores an index of the whole sequence $(\sigma(x_1), \dots, \sigma(x_u))$ among all sequences with the prescribed pattern counts. A membership query for S_i and x needs the pattern of x and tests whether it contains i. A rank query for S_i up to position x needs the number of earlier patterns that contain i. The single index gives neither value without decoding the sequence or adding separate indices. Bit optimality therefore does not by itself give a representation for the five operations of Definition 1.2.2.

To support these operations without decoding the pattern sequence, the representation has to recover each set through small changes. Containment gives such a change. From a parent P to a child $Q \subseteq P$, the edge stores the deleted elements $P \setminus Q$, and path queries count or select those deletions. Two incomparable sets lack such a parent inside the pair itself.

Their union contains both, and using it as a synthetic parent turns the pair into two containment edges.

The union parent also exposes a larger search space. Other synthetic references appear if we close $\mathcal{S}$ under union and intersection, and a parent with three or more children can encode a joint Venn pattern in one multinomial. In that larger space the flat atom code reappears as a one level hierarchy of arity m, while binary hierarchies can remain separated from it by a linear gap. Appendix A studies this comparison and the reference selection problem left by the binary route. We keep the binary union step in the chapter because it gives containment edges that can be turned into a query structure.

5.2 Union matching

We have just seen how to pair two incomparable sets A and B under their union $M = A \cup B$, reading the two parent-child edges as containments with deletion elements $M \setminus A$ and $M \setminus B$. Iterating this step over $\mathcal{S}$ produces a rooted forest whose leaves carry the input sets, whose internal nodes carry synthetic unions, and whose every parent-child edge is a containment. Each input set is then recovered from its component root by removing the elements along one root-to-leaf path.

The local building block is a binary merge. Two current top-level nodes with set labels A and B become the children of a new node with label $M = A \cup B$. The two new edges are containments by construction, and the description of the merge depends only on how the elements of M split among the three regions $A \cap B$, $A \setminus B$, and $B \setminus A$. We propose Set-Union Matching, abbreviated SUM, which iterates this merge level by level. Each level scores every pair of current top-level nodes by the change a merge would induce in a global cost functional, then selects disjoint pairs through a weighted matching, with unpaired nodes passing to the next level unchanged. The construction terminates after $\lceil \lg m \rceil$ levels and returns the cheapest forest seen so far.

5.2.1 Merge cost

Fix two sets $A, B \subseteq U$ and write $M = A \cup B$, $k = |A \cap B|$, $l = |A \setminus B|$, and $r = |B \setminus A|$. Then $|M| = k + l + r$. Once M is known, recovering A and B is the same as assigning every element of M to one of the three regions $A \cap B$, $A \setminus B$, and $B \setminus A$. Two binary choices encode this ternary label, because once two of the three regions are identified the third is determined by elimination. First, a bitvector of length $|M|$ marks the k elements in the intersection, selecting one of $\binom{|M|}{k}$ possibilities. After those elements are removed, a second bitvector of length $l + r$ marks the l elements in $A \setminus B$, selecting one of $\binom{l+r}{l}$ possibilities. The remaining r elements belong to $B \setminus A$ and require no third bitvector. The product of the two counts is the multinomial coefficient $\binom{|M|}{k,l,r}$. A short header records k and l, after which $r = |M| - k - l$ follows.

Write the elements of M in the order inherited from U as $m_1 < m_2 < \cdots < m_{|M|}$. The example in Figure 5.1 uses the seven positions $m_1, \ldots, m_7$

to show how the two bitvectors are indexed. The first bitvector isolates the intersection positions, and the second bitvector is indexed by the compacted sequence of elements that remain.

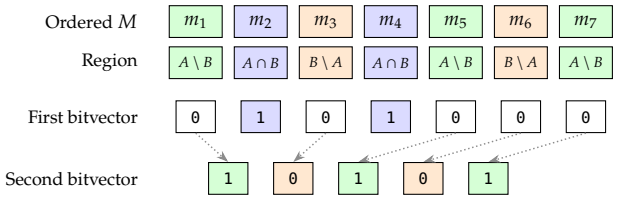

Figure 5.1: The two bitvectors used to encode the three regions of M. The top row fixes the order of M. The first bitvector marks the k positions of $A \cap B$. The unmarked positions are compacted, and the second bitvector marks the l positions of $A \setminus B$. The remaining positions belong to $B \setminus A$. The number of choices is $\binom{|M|}{k}\binom{l+r}{l} = \binom{|M|}{k,l,r}$.

Definition 5.2.1 (Merge cost) *For $A, B \subseteq U$ with $M = A \cup B$, $k = |A \cap B|$, $l = |A \setminus B|$, $r = |B \setminus A|$, the merge cost of the pair (A, B) is*

$$w(A, B) \;=\; \lg \binom{|M|}{k, l, r} \;+\; c(|M|, k), \tag{5.1}$$

where $c(|M|, k) = \lceil \lg(|M| + 1) \rceil + \lceil \lg(|M| - k + 1) \rceil$ encodes k and l.

When $k = 0$, the first bitvector is fixed and the multinomial part is $\binom{l+r}{l}$. We still store the two-field header so that one decoder handles all pairs.

The multinomial in (5.1) is bounded above by $\binom{|M|}{|A|}\binom{|M|}{|B|}$, the cost of encoding A and B as two separate bitvectors of length $|M|$, one marking the elements of A and one marking the elements of B. This is the local analogue of the $H_{wc}(\mathcal{S})$ baseline, where each S_i is encoded independently as a bitvector of length u. Every joint labelling of M by the three regions $A \cap B$, $A \setminus B$, $B \setminus A$ determines both bitvectors uniquely, so the count $\binom{|M|}{k,l,r}$ injects into their product.

The merge cost is local, but the representation must describe the whole collection. Each time we merge two current occurrences a and b with set labels P_a and P_b, we create a new parent occurrence c with set label $P_c = P_a \cup P_b$ and keep a and b as its two children. Repeating this operation turns the indexed input occurrences into the leaves of rooted binary components. The internal occurrences are precisely the synthetic unions introduced by the construction. A component root has no parent that can describe it, while an internal occurrence is described through its parent-child decomposition. The global cost must therefore charge roots and internal occurrences in different ways.

Definition 5.2.2 (SUM forest) *A SUM forest on $\mathcal{S}$ is a rooted forest F in which every internal node has exactly two children. We call the nodes of F occurrences, because two distinct nodes may carry the same set label. Each occurrence a carries a set label $P_a \subseteq U$. The leaves of F are exactly m distinguished occurrences $a_1, \ldots, a_m$ with $P_{a_i} = S_i$, kept distinct even when two input sets coincide. For every internal occurrence c with children a and b, $P_c = P_a \cup P_b$.*

A SUM forest records which union occurrences have been introduced. To decode it, we start from each root and move downward. At an internal occurrence c, once the decoder knows the set label P_c, the merge description charged by $w(P_{\text{left}(c)}, P_{\text{right}(c)})$ recovers the set labels of its two child occurrences. A root occurrence has no parent from which it can be recovered. The representation therefore stores each root label P_a directly as a subset of U, at cost $\lg \binom{u}{|P_a|}$. This gives two charges, one for starting a component and one for splitting a known occurrence.

Different choices of merges produce different SUM forests, so the construction needs a single value with which to compare them. We assign to a forest the sum of all root charges and all merge charges. This value is denoted by $\Phi(F)$.

Definition 5.2.3 (SUM cost) *The cost of a SUM forest F on $\mathcal{S}$ is*

$$\Phi(F) \;=\; \sum_{a \in \mathrm{roots}(F)} \lg \binom{u}{|P_a|} \;+\; \sum_{c \in \mathrm{internal}(F)} w(P_{\mathrm{left}(c)}, P_{\mathrm{right}(c)}). \tag{5.2}$$

In (5.2)*, the first sum is the support cost, with each root occurrence encoded against U; the second is the structural cost, with each internal occurrence contributing the merge cost of its decomposition. The terms* $\mathrm{left}(c)$ *and* $\mathrm{right}(c)$ *denote the two child occurrences of c.*

$\Phi(F)$ is a conditional cost. It counts root supports and local split descriptions after the decoder already knows the forest skeleton, the local cardinalities that determine the fixed-size and fixed-composition classes, and the map from the distinguished leaf occurrences a_i to the input indices i. The root terms count supports of known sizes, the multinomial terms count splits of known compositions, and the header term in w records the local fields used by the uniform decoder. The bits needed to encode the forest itself and the auxiliary indexes that support queries are accounted for separately in Section 5.3.

5.2.2 Greedy matching

$\Phi(F)$ ranks SUM forests by cost, but the construction still needs a rule for choosing which merges to perform. The rule assigns a score to every pair of current roots and selects compatible pairs through a weighted matching at each level.

We score a candidate merge by the change it induces in the forest cost Φ. Take two current root occurrences a and b with set labels $A = P_a$ and $B = P_b$. Replacing them with a new root occurrence labelled $M = A \cup B$ adds a root charge for M as a subset of U and removes the old root charges for a and b. The merge description then pays $w(A, B)$ to recover the two child labels from M. The resulting difference is the merge score.

Definition 5.2.4 (Merge score) *Let a and b be two root occurrences of a SUM forest, set $A = P_a$, $B = P_b$, and $M = A \cup B$. The merge score of the occurrence pair (a, b) is*

$$\Delta\Phi(a, b) \;=\; \lg \binom{u}{|M|} \;+\; w(A, B) \;-\; \lg \binom{u}{|A|} \;-\; \lg \binom{u}{|B|}. \tag{5.3}$$

In (5.3), the term $\lg \binom{u}{|M|}$ stores the new root label as a subset of U, and $w(A, B)$ stores the split of M into its two child labels. The two negative terms remove the former root charges of the two occurrences. Hence $\Delta\Phi(a, b) < 0$ means that this replacement lowers Φ, while $\Delta\Phi(a, b) > 0$ means that it raises Φ.

A level may contain several merges as long as no two pairs share a root. Each merge changes Φ only through the support charges of its two roots

and the new support and merge charges introduced by their parent. Disjoint pairs of merges therefore modify disjoint terms of Φ, and the total change at the level is the sum of the individual scores $\Delta\Phi(a, b)$. For a prescribed number q of pairs, the level cost is minimised by a minimum-weight matching of cardinality q in the complete graph on the current root occurrences, with edge weight $\Delta\Phi(a, b)$ on the edge $\{a, b\}$. SUM takes the maximum-cardinality choice, $q = \lfloor r/2 \rfloor$, when the level starts with r roots. The next level has $\lceil r/2 \rceil$ roots, so at most $\lceil \lg m \rceil$ levels are produced.

Since the matching has cardinality $\lfloor r/2 \rfloor$, it may contain pairs with positive score, and a level can raise Φ above the previous one. SUM therefore records the forest at every level and returns the cheapest recorded forest. The first recorded forest is the trivial F_0 on m isolated roots, with cost $\Phi(F_0) = \sum_i \lg \binom{u}{|S_i|} = H_{wc}(\mathcal{S})$. The independent encoding therefore remains available as a recorded option.

The score compares the cost of two encodings of the pair (A, B) over U. As separate roots, A and B are encoded as two binary strings of length u with $|A|$ and $|B|$ ones, giving $\binom{u}{|A|} \cdot \binom{u}{|B|}$ choices. As a single union root, the pair is encoded as one string of length u over the alphabet $\{U \setminus M, A \cap B, A \setminus B, B \setminus A\}$ with symbol counts $u - |M|, k, l, r$, giving $\binom{u}{u-|M|,k,l,r}$ choices.

For a candidate occurrence pair (a, b) with labels $A = P_a$ and $B = P_b$, the algebraic saving of the merge is $-\Delta\Phi(a, b)$. Substituting (5.1) into (5.3) gives

$$\begin{aligned} -\Delta\Phi(a, b) = \lg \binom{u}{|A|} \; + \; \lg \binom{u}{|B|} \\ - \; \lg \binom{u}{|M|} \; - \; \lg \binom{|M|}{k, l, r} \; - \; c(|M|, k). \end{aligned} \tag{5.4}$$

The two negative logarithms in (5.4) combine because the product $\binom{u}{|M|}\binom{|M|}{k,l,r}$ counts the same choices in two ways. We may choose the support M and then split it into the three regions inside M, or we may partition all u universe elements directly into $U \setminus M$, $A \cap B$, $A \setminus B$, and $B \setminus A$. Hence

$$\begin{aligned} -\Delta\Phi(a, b) = \lg \binom{u}{|A|} \; + \; \lg \binom{u}{|B|} \\ - \; \lg \binom{u}{u - |M|, k, l, r} \; - \; c(|M|, k). \end{aligned} \tag{5.5}$$

Algorithm 3 performs the level-by-level construction and returns the cheapest forest visited along the way. It maintains a working forest F, updated at each level by a minimum-weight matching of maximum cardinality on its current roots, alongside a recorded forest F^* initialised to the trivial F_0. The invariant $\Phi^* = \min_{0 \le s \le t} \Phi(F_s)$ holds at the end of level t, where F_s is the working forest after s levels.

Let r_t be the number of roots at the start of level t. A sorted-set implementation of Algorithm 3 first computes $\Delta\Phi(a, b)$ for every unordered pair of current roots. Computing one score amounts to finding $|P_a \cap P_b|$, $|P_a \setminus P_b|$, and $|P_b \setminus P_a|$, so a linear merge of the two sorted set labels costs $O(|P_a| + |P_b|)$ time. Let $N = \sum_i |S_i|$. Every current root label is a union

Algorithm 3: Set-Union Matching.

Input: Collection $\mathcal{S} = \{S_1, \ldots, S_m\}$ over universe U.
Output: A SUM forest F^* achieving the smallest value of Φ across all levels $0, 1, \ldots, \lceil \lg m \rceil$.

```
F ← trivial forest F_0 on S;
F* ← F; Φ* ← Φ(F);
for t = 1, 2, ..., ⌈lg m⌉ do
    compute ΔΦ(a, b) for every pair of current root occurrences a, b in F;
    M_t ← minimum weight matching of maximum cardinality on the current root occurrences, with edge weights ΔΦ;
    foreach pair (a, b) ∈ M_t do
        replace a, b in F by a new root occurrence c with P_c = P_a ∪ P_b and child occurrences a, b;
    if Φ(F) < Φ* then
        F* ← F; Φ* ← Φ(F);
return F*;
```

of input sets, hence has size at most N, and one score costs $O(N)$ time in the worst case. The level has $\binom{r_t}{2}$ candidate pairs, so scoring costs $O(r_t^2 N)$ time.

The same level then solves a weighted matching instance on the complete graph over the r_t roots. This graph has $\Theta(r_t^2)$ edges. Weighted matching on a general graph with arbitrary edge weights runs in $O(|E|r + r^2 \log r)$ time [60]. Substituting $|E| = \Theta(r_t^2)$ gives $O(r_t^3)$ matching time at level t. Since $r_t \le \lceil m/2^t \rceil$, the scoring costs over all levels sum to $O(m^2 N)$, and the matching costs sum to $O(\sum_t r_t^3) = O(m^3)$. This sorted-set implementation runs in $O(m^2 N + m^3)$ time.

[60]: Gabow (2018), *Data structures for weighted matching and extensions to b-matching and f-factors*

SUM is a greedy heuristic. The matching at each level minimises that level's cost at fixed cardinality but not the global cost over all binary union forests, so Φ may rise as well as fall over the levels. Even the first level can raise Φ when every candidate pair has positive score and the cardinality $\lfloor r/2 \rfloor$ forces a merge. A level may also pass over an individually profitable merge, because a negative-score edge enters the matching only if it belongs to the cheapest matching of that cardinality, and its endpoints may otherwise be matched to other roots when that lowers the total level cost.

The instance $\mathcal{S} = \{\{1\}, \{2\}, \{3\}\}$ over $U = \{1, 2, 3\}$ in Figure 5.2 exhibits this phenomenon. Every level-1 candidate merges two singletons under their union and pays a positive merge score, so Φ rises and SUM records F_0 as its minimum.

Figure 5.2: Trivial forest F_0 on $\mathcal{S} = \{\{1\}, \{2\}, \{3\}\}$ over $U = \{1, 2, 3\}$, and a level-1 candidate F_1 obtained by merging two singletons under the synthetic union $M_{12} = S_1 \cup S_2$. The costs are $\Phi(F_0) = 3 \lg 3$ and $\Phi(F_1) = 2 \lg 3 + 5$. Every level-1 candidate raises the objective, so SUM records F_0 and returns it.

The instance illustrates a general rule. Algorithm 3 returns the forest with smallest Φ across all levels visited, including F_0 at level zero.

Definition 5.2.5 (Minimum recorded forest) *Let F_t be the forest held by Algorithm 3 after level t, with F_0 the trivial forest. The minimum recorded forest is any forest F^* satisfying*

$$\Phi(F^*) = \min_{0 \le t \le \lceil \lg m \rceil} \Phi(F_t).$$

We write $L_{\mathrm{SUM}}(\mathcal{S}) = \Phi(F^*)$.

Recording F_0 at level zero gives a uniform upper bound against the baseline.

Theorem 5.2.1 (Baseline safety) *For every collection $\mathcal{S}$,*

$$L_{\mathrm{SUM}}(\mathcal{S}) \;\le\; H_{wc}(\mathcal{S}).$$

Proof. F_0 is a recorded forest and $\Phi(F_0) = H_{wc}(\mathcal{S})$, so the minimum across recorded forests is at most $H_{wc}(\mathcal{S})$. □

The atom bound L^* of Proposition 5.1.1 applies to every fixed SUM forest, in particular to the minimum recorded one. Combined with baseline safety, it places L_{SUM} between L^* and H_{wc}.

Proposition 5.2.2 (SUM sandwich) *For every fixed SUM forest F on $\mathcal{S}$,*

$$L^*(\mathcal{S}) \;\le\; \Phi(F).$$

Consequently,

$$L^*(\mathcal{S}) \;\le\; L_{\mathrm{SUM}}(\mathcal{S}) \;\le\; H_{wc}(\mathcal{S}).$$

Proof. With the skeleton of F and the leaf-to-input map fixed, every occurrence label is forced by the leaf labels and the local cardinalities depend only on the atom counts (c_τ), so the SUM record on F (root supports plus ternary splits) is a fixed-composition code of length at most $\Phi(F)$ for the atom type class of $\mathcal{S}$. By Proposition 5.1.1, $L^*(\mathcal{S}) \le \Phi(F)$. Specialising to the minimum recorded forest gives the left inequality, and Theorem 5.2.1 gives the right one. □

Remark 5.2.1 (Strict descent variant) Replacing $\mathscr{M}_t$ in Algorithm 3 by a minimum-weight matching restricted to root-occurrence pairs with $\Delta\Phi < 0$, and skipping the level when no such pair exists, gives a variant in which every non-empty level lowers Φ by the sum of the selected negative scores. The terminal forest $\widehat{F}$ inherits $\Phi(\widehat{F}) \le H_{wc}(\mathcal{S})$ by monotone descent rather than by the best-recorded mechanism. The two algorithms are incomparable, because the maximum-cardinality rule of Algorithm 3 can pay short-term positive-score steps that enable later strongly negative merges, while strict descent rejects every positive-score step and may stop earlier.

5.2.3 Deletion trees

The five operations of Definition 1.2.2 on S_i each reduce to a path query in the SUM forest. The path goes from the root of the component containing the leaf occurrence a_i down to a_i itself, recording the elements that the descent removes from the root. Containment along every parent-child edge makes those deletions pairwise disjoint, so a query on S_i becomes a count or selection over the path.

Proposition 5.2.3 (Containment edges) *For every internal occurrence c with child occurrences a and b, both inclusions $P_a \subseteq P_c$ and $P_b \subseteq P_c$ hold, so every parent-child edge in a SUM forest is a containment.*

Proof. By Definition 5.2.2, $P_c = P_a \cup P_b$. Hence $P_a \subseteq P_c$ and $P_b \subseteq P_c$. □

Let $a_0, a_1, \ldots, a_t$ be a root-to-leaf path in a SUM forest, with $R = P_{a_0}$ and $S = P_{a_t}$. For each step $1 \le i \le t$, set $D_i = P_{a_{i-1}} \setminus P_{a_i}$, the deletion set of the i-th edge, namely the elements it removes from its parent label. Since $P_{a_i} \subseteq P_{a_{i-1}}$ at every step, induction gives

$$S = R \setminus \bigcup_{i=1}^{t} D_i. \tag{5.6}$$

The deletion sets D_i are pairwise disjoint along the path. If $x \in D_i$, then $x \notin P_{a_i}$. Every later label P_{a_j} with $j \ge i$ is contained in P_{a_i}, so x cannot appear in any later deletion set. Equation (5.6) therefore says more than set equality. It says that every element removed from the root is removed exactly once.

Definition 5.2.6 (SUM deletion tree) *Let F be a SUM forest. For each component root r, the SUM deletion tree of that component is obtained by creating one structural node v_a for every forest node occurrence a in the component. Distinct occurrences with the same set label create distinct structural nodes. For every forest edge $a \to b$, replace it by a chain*

$$v_a \to z_1 \to z_2 \to \cdots \to z_h \to v_b,$$

where $h = |P_a \setminus P_b|$ and the nodes $z_1, \ldots, z_h$ are labelled with the distinct elements of $P_a \setminus P_b$ in arbitrary order. If $P_a \setminus P_b = \emptyset$, no deletion node is inserted and the structural edge $v_a \to v_b$ remains.

The deletion labels on the path from v_{a_0} to v_{a_t} are exactly the elements of $P_{a_0} \setminus P_{a_t}$, with no repetition, by (5.6). The query reductions use this path identity directly. A membership query asks whether the label for x appears on the path. A rank query counts labels in a prefix of the root order. An access query selects an element of P_{a_0} whose label is absent from the path.

Insertion paths (Definition 4.2.5) accumulate from $\emptyset$, indel paths (Definition 4.3.3) apply signed edits, SUM paths remove elements from a stored root R.

SUM deletion trees fit the tree-extraction framework of Chapter 4, with a path semantics specific to SUM.

The construction contains ordinary containment encoding as a special case. If $A \subset B$ already holds in $\mathcal{S}$, then the merge of two occurrences

carrying labels A and B gives $M = A \cup B = B$, $k = |A|$, $l = 0$, and $r = |B| - |A|$. Hence

$$w(A, B) = \lg \binom{|B|}{|A|, 0, |B| - |A|} + c(|B|, |A|) = \lg \binom{|B|}{|A|} + O(\lg |B|).$$

The binary merge creates a new internal node with set label B and two children carrying labels A and B. The edge to the child labelled B inserts no deletion node. Contracting that zero-deletion structural edge leaves the containment edge from the parent labelled B to the child labelled A. Its cost is the containment-entropy contribution of A given B in Definition 4.2.2, up to the self-delimiting header.

For incomparable root labels A and B, a containment-only hierarchy can place both sets under one nontrivial parent only when an existing set contains both. Otherwise the available common parent is U. SUM inserts the smallest possible common parent, namely $A \cup B$. If this new parent remains a component root, the forest pays its support cost $\lg \binom{u}{|A \cup B|}$. If a later merge places it below a larger union node, that support charge is replaced by the deletion cost from the larger parent label. The path semantics does not change. Every descendant is still obtained by deleting labels from its component root.

The binary union step is one choice among several possible ways to create deletion paths. Appendix A studies what changes if we admit additional reference sets generated from $\mathcal{S}$, or if one parent may split into more than two children. Such choices can reduce the counting cost because one parent can describe a larger Venn pattern at once. They also remove the weighted-matching structure that makes SUM a polynomial-time construction. The data structure of Section 5.3 keeps the binary deletion trees produced here and adds the root dictionaries and tree-extraction indices needed for the five operations.

5.3 Queries

The five operations of Definition 1.2.2 on S_i reduce to path queries in the minimum recorded forest F^* of Definition 5.2.5, with the path running from the component root to the leaf a_i. For input index i, let a_i be the distinguished leaf, r_i the component root, $R_i = P_{r_i}$, and v_i the structural node created from a_i by Definition 5.2.6. The deletion labels on the path from v_{r_i} to v_i are exactly the elements of $R_i \setminus S_i$. The tree-extraction indices of Section 3.5 expect a labelled ordinal tree whose labels already lie in the query alphabet, so we turn each deletion into a node label and each universe value into its rank inside the component root.

The representation performs these translations through a query directory $\mathcal{D}$, with entry $\mathcal{D}[i] = (R_i, v_i)$ giving the component root label and the structural node of the leaf occurrence for S_i. Each SUM deletion tree is converted into the node-labelled form expected by the HMZ primitives [42, 46], and each component root carries a dictionary between universe values and their ranks inside that root. The deletion alphabet in the tree rooted at R_i has size $|R_i|$, and the five operations of Definition 1.2.2 reduce to one directory lookup, root-dictionary queries, and tree-extraction primitives.

Empty input sets and components with $|R_i| = 0$ are stored as sentinels outside the deletion-tree machinery; member returns false, rank returns zero, and access, predecessor, and successor return their respective failure values.

[42]: He et al. (2014), *A framework for succinct labeled ordinal trees over large alphabets*
[46]: He et al. (2016), *Data structures for path queries*

5.3.1 Forest encoding

Fix a component root occurrence r with set label $R = P_r$. The chain expansion of Definition 5.2.6 produces a structural node v_a for every forest occurrence a in the component below r, and inserts one deletion node z_j for each element removed along the edge from a to a child. Since every set label below r is a subset of R, every deletion label is an element of R.

For every universe value x, $\mathrm{rank}_R(x)$ denotes the number of elements of R at most x, including the case $x \notin R$. We store at each deletion node the local rank $\mathrm{rank}_R(x) \in [1..|R|]$, and at each structural node the dummy label $\delta_R = |R| + 1$. The labelled ordinal tree thus has alphabet $[1..|R| + 1]$, with the deletion alphabet $[1..|R|]$ used by every query. The dummy label δ_R fills the HMZ requirement that every node carry a label, without inflating any query interval.

By (5.6), the deletion labels on each root-to-leaf path are pairwise distinct, so path counting on the deletion alphabet returns each label at most once and path selection on these labels is well posed.

Public queries use universe values, while the tree index stores local ranks. Each root with label R is therefore stored as an RRR dictionary from Theorem 2.1.7, supporting membership, rank_R, and select_R in constant time. The dictionary occupies

$$\lg \binom{u}{|R|} + o(u) + O(\lg \lg u)$$

bits. The leading term is the support cost of R in Definition 5.2.3, while the lower-order terms belong to the query index outside L_{SUM}.

The α-operations index of Theorem 3.6.2 and the path-queries index of Theorem 3.7.4 overlay each labelled deletion tree at technical alphabet size $|R| + 1$. The α-operations index supports $\mathtt{parent}_\alpha(v)$, the lowest ancestor of v with label α, for every $\alpha \in [1..|R|]$ in $O(\lg \lg_\omega |R|)$ time. The path-queries index supports $\mathtt{path\_count}(v, [a..b])$, the number of nodes on the path from the root to v whose label lies in $[a..b]$, in $O(\lg_\omega |R|)$ time, and $\mathtt{path\_select}(v, j)$, the j-th smallest label among the labels on that path, in $O(\lg |R| / \lg \lg |R|)$ time.

Each index stores a labelled ordinal tree of $|T|$ nodes in $O(|T|)$ words. Summed over all roots of F^*, the deletion forest has

$$O\left(m + \sum_{c \in \mathrm{internal}(F^*)} |P_{\mathrm{left}(c)} \triangle P_{\mathrm{right}(c)}|\right)$$

nodes, and the tree-extraction overlay occupies the same number of words.

5.3.2 Complement selection

Access asks for the q-th root label that survives all deletions on the path. The path-selection primitive of Theorem 3.7.4 returns deleted labels rather than surviving ones, so its alphabet descent has to be adapted.

We keep the same descent and replace present-label counts by survivor counts. Membership and rank do not need this adaptation, since they use $\texttt{parent}_\alpha$ and `path_count` directly.

Definition 5.3.1 (Complement path selection) *Let T be a labelled ordinal tree on alphabet $[1..\sigma+1]$. Labels in $[1..\sigma]$ are deletion labels, label $\sigma+1$ is a dummy structural label, and deletion labels are unique on each root-to-leaf path. For every node $v \in T$ and every integer i with $1 \le i \le \sigma - \texttt{path\_count}(v, [1..\sigma])$, the complement path selection $\texttt{path\_compselect}(v, i)$ is the i-th smallest $\alpha \in [1..\sigma]$ absent from the deletion labels of all ancestors of v.*

Lemma 5.3.1 *The path-queries index of Theorem 3.7.4 on a labelled ordinal tree T satisfying Definition 5.3.1 supports $\texttt{path\_compselect}(v, i)$ in $O(\lg\sigma/\lg\lg\sigma)$ time on the word RAM with $\omega = \Omega(\lg n)$.*

Proof. The path-selection descent of Theorem 3.7.4 navigates a multi-ary partition of $[1..\sigma]$ using prefix counts of ancestor labels, with partition depth $O(\lg\sigma/\lg\lg\sigma)$. For a node v, let

$$D_v(\gamma) = \texttt{path\_count}(v, [1..\gamma]), \quad 0 \le \gamma \le \sigma,$$

so that, by deletion-label uniqueness on the root-to-v path, the number of survivors in any child range $[a..b] \subseteq [1..\sigma]$ is

$$(b - a + 1) - \big(D_v(b) - D_v(a-1)\big).$$

The prefix length $b - a + 1$ is fixed by the alphabet partition and the prefix difference is already stored by the selection structure, so survivor prefixes are obtained by one subtraction per child range. Feeding them to the same packed-comparison step that the original `path_select` uses on present-label prefixes leaves the descent depth and per-step cost unchanged, with the dummy label $\sigma + 1$ outside every queried interval. The stated time bound follows. □

5.3.3 Query reductions

Every query reads $\mathcal{D}[i] = (R_i, v_i)$ to recover the component root label and the leaf occurrence for S_i. The deletion labels on the root-to-v_i path are exactly the elements of $R_i \setminus S_i$, so a label $j \in [1..|R_i|]$ appears on the path if and only if the descent from R_i to S_i removes $\text{select}_{R_i}(j)$. When no ancestor of v_i carries label j, we write $\texttt{parent}_j(v_i) = \text{NULL}$, distinct from the dummy label $\delta_{R_i} = |R_i| + 1$ that sits outside the deletion alphabet.

Membership asks whether x belongs to S_i. Since $S_i \subseteq R_i$, if x is not in the component root then x cannot belong to S_i either, and the root dictionary disposes of this case in constant time. When x does belong to R_i, the question is whether the descent from R_i down to v_i deletes x. We translate x to its local rank $j = \text{rank}_{R_i}(x)$, and the path query reduces to asking whether any ancestor of v_i carries label j. The α-operations index answers this in one call. The value $\texttt{parent}_j(v_i) = \text{NULL}$ tells us that no such ancestor exists, in which case x has survived to S_i. Algorithm 4 packages these steps.

Algorithm 4: SUM membership query.

Input: An input index i, a value x, the query directory $\mathfrak{D}$, and the root dictionaries and tree indices.
Output: Whether x belongs to S_i.

```
1 Query member(i, x):
2     (R_i, v_i) ← 𝔇[i];
3     if member_{R_i}(x) = false then return false;
4     j ← rank_{R_i}(x);
5     return parent_j(v_i) = NULL;
```

Rank asks how many elements of S_i are at most x. The elements of R_i at most x are easy to count, since the root dictionary returns $j = \text{rank}_{R_i}(x)$ in constant time, regardless of whether x itself lies in R_i. Among those j candidates, however, only the survivors of the deletions on the path to v_i also belong to S_i, and $\texttt{path\_count}(v_i, [1..j])$ counts exactly how many of them the descent deletes. Subtracting the two quantities yields the rank of x inside S_i. Algorithm 5 performs this subtraction.

Algorithm 5: SUM rank query.

Input: An input index i, a value x, the query directory $\mathfrak{D}$, and the root dictionaries and tree indices.
Output: The number of elements of S_i at most x.

```
1 Query rank(i, x):
2     (R_i, v_i) ← 𝔇[i];
3     j ← rank_{R_i}(x);
4     if j = 0 then return 0;
5     return j − path_count(v_i, [1..j]);
```

Access returns the q-th smallest element of S_i. The complement path selection $\texttt{path\_compselect}(v_i, q)$ of Definition 5.3.1 gives its local rank j in R_i. The root dictionary converts j back to the universe value $\text{select}_{R_i}(j)$. Algorithm 6 chains the two calls.

Algorithm 6: SUM access query.

Input: An input index i, a position $q \in [1..|S_i|]$, the query directory $\mathfrak{D}$, and the root dictionaries and tree indices.
Output: The q-th smallest element of S_i.

```
1 Query access(i, q):
2     (R_i, v_i) ← 𝔇[i];
3     j ← path_compselect(v_i, q);
4     return select_{R_i}(j);
```

Predecessor returns the largest element of S_i at most x, or $-\infty$ if no such element exists. The rank $r = \text{rank}(i, x)$ counts the elements of S_i at most x, so the predecessor is $\text{access}(i, r)$ when $r > 0$ and $-\infty$ when $r = 0$. Algorithm 7 composes the two calls.

Successor returns the smallest element of S_i at least x, or $+\infty$ if none exists. The candidate position is $r = \text{rank}(i, x - 1) + 1$, using the convention $\text{rank}(i, 0) = 0$ for $x = 1$. The size $|S_i| = |R_i| - \texttt{path\_count}(v_i, [1..|R_i|])$ counts the survivors. The successor is $\text{access}(i, r)$ when $r \le |S_i|$, and $+\infty$

Algorithm 7: SUM predecessor query.

Input: An input index i, a value x, the query directory $\mathcal{D}$, and the root dictionaries and tree indices.
Output: The predecessor of x in S_i, or $-\infty$.

```
1 Query predecessor(i, x):
2     r ← rank(i, x);
3     if r = 0 then return −∞;
4     return access(i, r);
```

otherwise. Algorithm 8 runs the rank, the size test, and the access in sequence.

Algorithm 8: SUM successor query.

Input: An input index i, a value x, the query directory $\mathcal{D}$, and the root dictionaries and tree indices.
Output: The successor of x in S_i, or $+\infty$.

```
1 Query successor(i, x):
2     (R_i, v_i) ← D[i];
3     r ← rank(i, x − 1) + 1;
4     q ← |R_i| − path_count(v_i, [1..|R_i|]);
5     if r > q then return +∞;
6     return access(i, r);
```

5.3.4 Representation bound

The SUM representation rests on three structures, an RRR dictionary on each component root from Theorem 2.1.7, the α-operations index of Theorem 3.6.2, and the path-queries index of Theorem 3.7.4 on the labelled deletion forest. The path-queries index also supports the complement path selection of Definition 5.3.1 by Lemma 5.3.1. We bound the space of all three structures together and the time of each operation.

The space has two terms. The root dictionaries pay one support term per component root, providing the conversion between universe values and local ranks at query time. The tree-extraction overlay stores the labelled deletion forest together with the query directory, paying the structural cost in words. Each operation's time matches that of its slowest primitive call.

Theorem 5.3.2 *Let $\mathcal{S}$ be a collection of subsets of $[1..u]$, let F^* be the minimum recorded forest returned by Algorithm 3, and assume the word RAM model with $\omega = \Omega(\lg n)$. The SUM representation occupies the space of the root dictionaries,*

$$\sum_{r \in \mathrm{roots}(F^*)} \left(\lg \binom{u}{|P_r|} + o(u) + O(\lg \lg u) \right) \text{ bits,}$$

plus the space of the tree-extraction overlay,

$$O\left(m + \sum_{c \in \mathrm{internal}(F^*)} |P_{\mathrm{left}(c)} \triangle P_{\mathrm{right}(c)}| \right) \text{ words.}$$

For every input index i with $\mathcal{D}[i] = (R_i, v_i)$, it supports

- member(i, x) *in* $O(\lg \lg_\omega |R_i|)$
- rank(i, x) *in* $O(\lg_\omega |R_i|)$
- access(i, q), predecessor(i, x), successor(i, x) *in* $O(\lg |R_i| / \lg \lg |R_i|)$

Proof. The space bound follows from the RRR dictionaries of Theorem 2.1.7, the node count of the deletion forest, and the $O(|T|)$-word storage of Theorem 3.6.2 and Theorem 3.7.4. Algorithm 4, Algorithm 5, Algorithm 6, Algorithm 7, and Algorithm 8 give the reductions after the directory lookup. Membership is dominated by the α-operation $\texttt{parent}_j$ at $O(\lg \lg_\omega |R_i|)$. Rank is dominated by `path_count` at $O(\lg_\omega |R_i|)$. Access is dominated by `path_compselect` at $O(\lg |R_i| / \lg \lg |R_i|)$ via Lemma 5.3.1. Predecessor and successor each call rank and access once, and the access bound dominates. □

The redundancy term $o(u)$ in each root dictionary depends on the universe size u, not on the local size $|R_r|$. Summed over the roots, the total $\sum_r o(u)$ is not in general $o(L_{\mathrm{SUM}}(\mathcal{S}))$, and can dominate the leading $\lg \binom{u}{|R_r|}$ terms when many roots have $|R_r| \ll u$.

The bounds of Theorem 5.3.2 are sharper than those of Theorem 4.2.9 and Theorem 4.3.7 because the alphabet is local. Where those bounds use $\lg u$ logarithms in the universe size, the SUM bounds use $\lg |R_i|$ in the size of the component root. Whenever $|R_i| < u$, every SUM time bound is strictly smaller than the corresponding insertion or indel bound. At $|R_i| = u$ the membership and rank bounds match Theorem 4.2.9, and the access bound is smaller than the indel access bound by a factor of $\lg \lg u$.

The cost $L_{\mathrm{SUM}}(\mathcal{S})$ of Definition 5.2.3 and the queryable space of Theorem 5.3.2 measure different quantities. L_{SUM} is a counting bound, summing $\lg \binom{|P_c|}{k_c, l_c, r_c}$ per merge plus header and root support terms. The queryable overlay stores each deletion label explicitly in $\lg |R_c|$ bits, giving a bit-level size of

$$O\left(\sum_{c \in \mathrm{internal}(F^*)} |P_{\mathrm{left}(c)} \triangle P_{\mathrm{right}(c)}| \, \lg |R_c| \right)$$

plus structural low-order terms and the root dictionaries. The two forms differ, and the overlay can exceed L_{SUM} when a merge has a small multinomial term but a large symmetric difference. Whether SUM admits a queryable representation of size $L_{\mathrm{SUM}}(\mathcal{S})$ bits within the time bounds of Theorem 5.3.2 is open. A similar gap remains in the symdiff-compressibility setting of Gagie, He, and Navarro [9], who leave matching $\Delta(\mathcal{S}) \lg u + o(\Delta(\mathcal{S}) \lg u)$ bits rather than $O(\Delta(\mathcal{S}))$ words as future work.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

Conclusions 6

The thesis has used one criterion throughout. A relation between sets is useful only when it reduces the counting cost and still leaves a path along which membership, rank, access, predecessor, and successor can be answered. The independent baseline of Definition 1.2.3 gives the cost before such relations are used. The regimes surveyed in Chapter 4 lower that cost by choosing a reference for each set, storing the difference, and realising the resulting measure as a labelled ordinal tree. Containment gives deletion paths, insertion compressibility gives insertion paths, and symmetric-difference compressibility gives signed paths. In all three cases, tree extraction turns a combinatorial measure into a queryable object.

The five operations are those of Definition 1.2.2. They remain in view here because every compressibility measure in the thesis is judged by whether those operations stay queryable.

Chapter 5 asks what remains when useful references are missing from the input collection. The atom bound $L^*(\mathcal{S})$ answers this at the level of information content, since fixing the membership pattern of every universe element determines the collection up to a multinomial choice of atom labels. This bound is tight for a flat encoding, but the flat encoding gives no path along which a query can travel. SUM (Section 5.2) supplies such paths by adding union nodes. When two incomparable roots A and B have no containment relation, the synthetic parent $A \cup B$ contains both. The local Venn count of the pair gives the cost of the merge, and the resulting forest is a deletion hierarchy on which the query structure of Section 5.3 applies.

SUM occupies a middle position between the atom bound, which treats all membership patterns at once and reaches the best count we prove in this thesis, and the known hierarchies of Chapter 4, which use only references already selected by the input or by a graph on the input sets. SUM adds references, yet keeps every edge a containment edge. This containment discipline lets the representation use the same tree-extraction structures as the deletion-based encodings, with local alphabets of size $|R_i|$ at the component roots, but Theorem 5.3.2 also shows the price. The queryable overlay stores explicit deletion labels and root dictionaries, so its bit cost does not automatically match the counting objective $L_{\mathrm{SUM}}(\mathcal{S})$.

Appendix A makes this price explicit by enlarging the space of admissible references. Closing $\mathcal{S}$ under union and intersection gives a finite inclusion lattice indexed by upsets of the realised membership-pattern poset. Allowing a parent to split into more than two children can reach the atom bound in one level, while binary arity keeps the pairwise structure used by SUM. The exact union-only dynamic programme gives a baseline for small m, because when every internal label is forced to be the union of its descendant leaves, the best binary tree can be computed in $O(3^m)$ split evaluations. The full binary problem remains larger, because internal labels may come from the whole lattice closure.

These limitations match the questions left by the papers that this thesis extends. Alanko et al. [2] point out that adding synthetic sets, such as intersections, could reduce the cost of containment-based encodings, while such more general measures may no longer be computable in

The same boundary appears in the literature. Alanko et al. [2] point to synthetic sets; Gagie, He, and Navarro [9] point to bit-level succinctness and richer set operations.

[2]: Alanko et al. (2025), *Compact data structures for collections of sets*

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

polynomial time. Gagie, He, and Navarro [9] ask whether the word-space representation for symmetric-difference compression can be made succinct at $\Delta(\mathcal{S}) \lg u + o(\Delta(\mathcal{S}) \lg u)$ bits without losing the query times, and whether richer operations such as complement, union, and intersection can be incorporated while preserving efficient queries. SUM gives one concrete instance of this programme. It chooses only binary unions, so every auxiliary reference remains a parent in a deletion hierarchy and tree extraction applies without signed labels. This answers one restricted augmentation problem while leaving the optimal choice of auxiliary references unresolved.

The first question left by SUM is therefore bit-level. We know how to count a SUM forest through $L_{\mathrm{SUM}}(\mathcal{S})$, and we know how to query the deletion forest through tree extraction. We do not yet know how to obtain a representation of size $L_{\mathrm{SUM}}(\mathcal{S})$ bits, up to lower-order terms, while keeping the time bounds of Theorem 5.3.2. The same gap appears in the signed symmetric-difference setting, where the combinatorial object is small, the queryable overlay is larger, and collapsing the overlay without slowing the queries is the hard step.

The second question is algorithmic, because SUM chooses binary unions greedily through weighted matching. The appendix gives an exact dynamic programme only for the union-only restriction, with 3^m split evaluations, and separates that restriction from the full lattice model. These facts mark reference selection as an optimisation problem, but they do not prove hardness. The next step is to determine whether broader families of synthetic references admit polynomial-time algorithms, approximation guarantees, or hardness proofs. Until such results are known, the matching step in SUM gives a polynomial construction with a clear query structure.

Large collections also force us to ask how SUM should find the pairs worth scoring. Since Section 5.2 scores the complete graph of current roots at every level, its sorted-set implementation spends $O(m^2 N)$ time on scores. We could maintain resemblance and containment sketches [61], use locality sensitive hashing to form a sparse candidate graph [62], and compute $\Delta\Phi$ only on retained edges before matching. This fits the level structure of SUM, because the MinHash signature of $A \cup B$ is the componentwise minimum of the signatures of A and B. The forest and query structure would stay unchanged, since every retained edge still uses an exact union merge. The hard part becomes measuring the loss when the candidate graph misses pairs with small $\Delta\Phi$, especially containment pairs that resemblance alone can miss.

[61]: Broder (1997), *On the resemblance and containment of documents*

[62]: Gionis et al. (1999), *Similarity Search in High Dimensions via Hashing*

This thesis moves from visible references to constructed references. We began with representations that exploit relations already visible between sets. We then introduced a lower bound based on the atoms of the membership table, showed why the flat optimum is insufficient for queries, and built SUM as a queryable hierarchy that creates the containments it needs. The remaining questions all ask how much further this programme can be pushed, through more synthetic references that reduce the count, more compact overlays that close the bit gap, or better optimisation algorithms that replace the greedy construction on wider families. Across all of these directions, compression for set collections should be designed together with the paths used to query it.

Appendix

A Lattice Models

A.1 Lattice closure **92**
A.2 Binary case **99**

The atom bound $L^*(\mathcal{S})$ of Proposition 5.1.1 answers a counting question. If the realised membership patterns of Definition 5.1.1 and their multiplicities are fixed, the collection can vary only by permuting the elements of U among the atoms of Definition 5.1.2, and the corresponding multinomial gives the number of choices. This counts only the freedom of placing elements among atoms. A queryable representation also chooses parents, paths, labels, and local decoding rules, and those structural choices add bits above L^*.

Set-Union Matching of Section 5.2 takes a restricted form of this problem. It builds a binary hierarchy and admits a single kind of synthetic node, the union $A \cup B$ of two current roots. The restriction makes the construction polynomial and turns every edge from parent to child into a containment edge, but it also rules out alternatives that could lower the bit cost. We embed the problem into a larger optimisation in which those alternatives are admissible, and the gap between its optimum and the binary case measures what the restriction costs.

The first enlargement concerns the family of candidate parents. We let the parent of any set that is not a root come from $\mathcal{S}$ or from the closure of $\mathcal{S}$ under union and intersection, the two monotone operations used by containment arguments. Section A.1 treats this closure as a bounded inclusion lattice. The realised membership patterns of Definition 5.1.1 index it compactly, because every candidate parent is a union of atoms and therefore corresponds to an upset of the pattern poset.

The second enlargement concerns arity. SUM splits each parent into two children, while the model also admits parents that split into several children at once, and higher arity parents can be cheaper than the binary hierarchies that build the same leaves. With all m sets sitting directly under U the cost meets $L^*(\mathcal{S})$. The binary regime where SUM operates remains important because it preserves the graph matching step and the pairwise containment paths used by the query structure.

A.1 Lattice closure

A reference hierarchy assigns each set that is not a root a parent set, and $\mathcal{S}$ alone may fail to supply one. We draw parents from the closure of $\mathcal{S}$ under finite intersection and union, the smallest family containing $\mathcal{S}$ and closed under the two operations used by containment hierarchies.

If S_1 and S_2 are disjoint and cover U, the diagram has the four nodes $\emptyset < S_1, S_2 < U$. If they overlap, $S_1 \cap S_2$ appears between $\emptyset$ and the two generators.

We read the candidates as an inclusion diagram. The universe U is the top node and $\emptyset$ is the bottom node. The input sets $S_1, \ldots, S_m$ are marked generator nodes between them. Whenever two available nodes are present, their union gives a node above both of them, and their intersection gives a node below both of them. The lattice closure is the

family obtained by adding exactly the nodes forced by repeating these two moves.

Definition A.1.1 (Lattice closure) *Let $\mathcal{S} = \{S_1, \dots, S_m\}$ be a family of subsets of the finite universe U. The lattice closure $\mathcal{C}(\mathcal{S})$ is the smallest family $\mathcal{F} \subseteq 2^U$ such that $\mathcal{S} \subseteq \mathcal{F}$ and, whenever $A, B \in \mathcal{F}$, both $A \cap B$ and $A \cup B$ belong to $\mathcal{F}$.*

The order is inclusion. For the bounded closure used below, set $\overline{\mathcal{L}} = \mathcal{C}(\mathcal{S}) \cup \{U, \emptyset\}$. Joins are unions, meets are intersections, and both operations stay inside $\overline{\mathcal{L}}$ by construction.

Ganter and Wille define a lattice by the existence of binary meet and join, and use intersection and union as the meet and join of the power set ordered by inclusion [63]. Since $\overline{\mathcal{L}}$ is closed under those operations and lies in the finite power set 2^U, it inherits the same meet and join. The set identities $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ and $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ hold for all subsets of U, so they hold in $\overline{\mathcal{L}}$. Hence $\overline{\mathcal{L}}$ is a finite distributive lattice.

[63]: Ganter et al. (1999), *Formal Concept Analysis: Mathematical Foundations*

Under the convention $U = \bigcup_i S_i$ of Section 4.1, U already belongs to $\mathcal{C}(\mathcal{S})$. The empty set may or may not belong to $\mathcal{C}(\mathcal{S})$, and adding it gives the bottom element. We write $\mathcal{L} = \overline{\mathcal{L}} \setminus \{\emptyset\}$ for the nonempty reference candidates. The empty set cannot act as a parent for a nonempty input set. Since two nonempty references can have empty intersection, removing $\emptyset$ can destroy meet closure. We therefore use $\overline{\mathcal{L}}$ for lattice statements and $\mathcal{L}$ for reference candidates.

A.1.1 Upset representation

We now name a reference by the atom patterns it contains. By Definition 5.1.2, the nonempty sets A_τ partition U, and each generator S_i either contains all of A_τ or none of it. The same all or nothing property is preserved by intersection and union, so every set in $\overline{\mathcal{L}}$ is a union of whole atoms.

Let

$$R = R(\mathcal{S}) = \{\, \sigma(x) : x \in U \,\} \subseteq 2^{[m]} \setminus \{\emptyset\},$$

ordered by subset inclusion. For every $Q \subseteq R$, define

$$X_Q = \bigcup_{\tau \in Q} A_\tau,$$

with $X_\emptyset = \emptyset$. Conversely, if $X \subseteq U$ is a union of atoms, set $Q_X = \{\tau \in R : A_\tau \subseteq X\}$. The atom partition gives $X = X_{Q_X}$, so subsets of R name references built as unions of atoms without listing their elements in U.

Definition A.1.2 (Upset) *An upset of an ordered set $(M, \leq)$ is a subset $Q \subseteq M$ such that $q \in Q$ and $q \leq q'$ imply $q' \in Q$.*

Each generator S_i has index $Q_i = \{\tau \in R : i \in \tau\}$. This Q_i is an upset of $(R, \subseteq)$, since $i \in \tau$ and $\tau \subseteq \tau'$ imply $i \in \tau'$. Intersections and unions of upsets are still upsets, so every set in $\mathcal{C}(\mathcal{S})$ has an upset index. The bottom $\emptyset$ has index $\emptyset$, and the top U has index R.

A finite upset is determined by its minimal elements. Every element of the upset lies above at least one minimal element, and distinct minimal elements are incomparable. We use the standard name for such incomparable sets.

Definition A.1.3 (Antichain [63]) *An antichain of an ordered set $(M, \leq)$ is a subset $A \subseteq M$ in which any two distinct elements are incomparable.*

The upward arrow records the direction of closure in the ordered set. For one pattern $a \in R$, the notation $\uparrow a$ is shorthand for the set of realised patterns above a, namely $\{\tau \in R : a \subseteq \tau\}$. For a set $A \subseteq R$, we take all such upper sets at once and write $\uparrow A = \{\tau \in R : \tau \supseteq a \text{ for some } a \in A\}$. The empty choice generates no pattern, so $\uparrow \emptyset = \emptyset$.

If Q is an upset of $(R, \subseteq)$, then the elements of $\min(Q)$ are exactly the elements of Q with no smaller element of Q below them. Since R is finite, every element of Q lies above at least one element of $\min(Q)$, and the elements of $\min(Q)$ are pairwise incomparable. Thus $\min(Q)$ is an antichain and $Q = \uparrow \min(Q)$. Conversely, if A is an antichain of R, then $\uparrow A$ is an upset and $\min(\uparrow A) = A$. The maps $Q \mapsto \min(Q)$ and $A \mapsto \uparrow A$ are mutual inverses, so antichains of R index upsets of R bijectively. We write $J(R)$ for the total number of upsets of R, including both $\emptyset$ and R.

Theorem A.1.1 (Closure representation) *The map $Q \mapsto X_Q$ is a lattice isomorphism between the upsets of $(R, \subseteq)$ and $\overline{\mathscr{L}}$, where both lattices use intersection as meet and union as join. It sends the empty upset to $\emptyset$. Restricted to nonempty upsets, it is a poset isomorphism onto $\mathscr{L}$. In particular $|\overline{\mathscr{L}}| = J(R)$ and $|\mathscr{L}| = J(R) - 1$.*

Proof. Since the atoms are pairwise disjoint, for any $Q_1, Q_2 \subseteq R$ we have

$$X_{Q_1} \cap X_{Q_2} = X_{Q_1 \cap Q_2}, \qquad X_{Q_1} \cup X_{Q_2} = X_{Q_1 \cup Q_2}.$$

Both the intersection and the union of upsets are upsets, so $Q \mapsto X_Q$ preserves meet and join on the lattice of upsets.

The image is exactly $\overline{\mathscr{L}}$. It contains $\emptyset = X_\emptyset$, $U = X_R$, and each generator $S_i = X_{Q_i}$, where $Q_i = \{\tau \in R : i \in \tau\}$. The identities above make the image closed under union and intersection, so it contains $\overline{\mathscr{L}}$. For the reverse inclusion, let Q be an upset. If $Q = \emptyset$, then $X_Q = \emptyset$. Otherwise,

$$X_Q = \bigcup_{a \in \min(Q)} X_{\uparrow a} = \bigcup_{a \in \min(Q)} \bigcap_{i \in a} S_i,$$

because $x \in X_{\uparrow a}$ iff $\sigma(x) \supseteq a$ iff $x \in S_i$ for every $i \in a$. The right side is built from $\mathcal{S}$ by intersections and unions, so $X_Q \in \mathscr{C}(\mathcal{S}) \subseteq \overline{\mathscr{L}}$.

The map is injective. If $X_{Q_1} = X_{Q_2}$, then for every $\tau \in R$, $A_\tau \subseteq X_{Q_1}$ iff $\tau \in Q_1$, and $A_\tau \subseteq X_{Q_2}$ iff $\tau \in Q_2$. Thus $Q_1 = Q_2$. The empty upset is the only upset mapped to $\emptyset$, so restricting the isomorphism to nonempty upsets gives a poset isomorphism onto $\mathscr{L}$. The cardinality identities follow. □

Theorem A.1.1 turns the size of the closure into a counting problem on the realised pattern poset. Exact counting would require counting all antichains of R. We use two estimates instead. The first forgets the

realised set R and compares it with the full Boolean lattice. The second keeps one parameter of R, its largest antichain.

Definition A.1.4 (Dedekind number [64]) *For $m \geq 0$, the Dedekind number $D(m)$ is the number of antichains of the Boolean lattice $2^{[m]}$ ordered by inclusion. The empty antichain is counted.*

Kleitman proves that the base two logarithm of $D(m)$ has the order of the middle level of the Boolean lattice,

$$\lg D(m) \sim \binom{m}{\lfloor m/2 \rfloor}.$$

This gives the right scale when every nonempty membership pattern is realised. Our pattern set R can be smaller, so the Dedekind number gives a worst case comparison.

Every antichain of R is an antichain of $2^{[m]} \setminus \{\emptyset\}$. The empty pattern is comparable with every other pattern, so the only antichain of $2^{[m]}$ that contains it is the singleton $\{\emptyset\}$. The nonempty part therefore has $D(m) - 1$ antichains. Hence

$$|\overline{\mathscr{L}}| = J(R) \leq D(m) - 1, \qquad |\mathscr{L}| \leq D(m) - 2.$$

Equality in the first inequality forces every singleton antichain $\{\tau\}$, with $\emptyset \neq \tau \subseteq [m]$, to come from R. Hence equality occurs only when $R = 2^{[m]} \setminus \{\emptyset\}$, the regime in which every nonempty Venn region is realised.

Definition A.1.5 (Width) *The width of a finite ordered set $(M, \leq)$ is the maximum cardinality of an antichain in M.*

Let w be the width of $(R, \subseteq)$. Every antichain of R has size at most w, so the number of antichains of R is at most the number of subsets of R of size at most w. Thus

$$|\overline{\mathscr{L}}| = J(R) \leq \sum_{j=0}^{w} \binom{|R|}{j}, \qquad |\mathscr{L}| \leq \sum_{j=0}^{w} \binom{|R|}{j} - 1.$$

This estimate reacts to the realised order on R. If R has small width, few subsets of R can be antichains, and the lattice closure is correspondingly smaller.

The two bounds count candidate references. They do not choose the edges of a hierarchy. The next choice is how many children a parent may encode at once, and that choice is governed by arity.

A.1.2 Arity chain

The closure gives many possible references. A representation still has to connect them. A root reference is stored as a subset of U. Every lower reference is recovered from a parent by describing which part of the parent belongs to each child. Allowing a parent to split into several children at once gives a joint description of the children inside the parent, and this is the role of arity.

Fix a parent P and children $Q_1, \dots, Q_r$ with $Q_j \subseteq P$. Each element of P has a binary membership pattern across the children. The vector of counts over $\{0,1\}^r$ is the local Venn data of the split. Encoding this vector as one multinomial can be cheaper than encoding the r children independently as r marginal subsets of P.

Definition A.1.6 (Reference hyperarc) *A reference hyperarc is a tuple $e = (P; Q_1, \dots, Q_r)$ with $P, Q_1, \dots, Q_r \in \mathcal{L}$, $r \geq 2$, and $Q_j \subseteq P$ for every j. We call P the parent label, $Q_1, \dots, Q_r$ the child labels, and r the arity of e. Child labels may repeat. We write $P_e = P$, $Q_j^e = Q_j$, and $r_e = r$.*

For each $b \in \{0,1\}^{r_e}$, the local Venn count of b in e is

$$a_b^e \;=\; \Big|\{\, x \in P_e : (\mathbf{1}[x \in Q_j^e])_{j=1}^{r_e} = b \,\}\Big|.$$

The local cost of e is

$$\operatorname{cost}(e) \;=\; \lg \binom{|P_e|}{(a_b^e)_{b \in \{0,1\}^{r_e}}}.$$

A reference hierarchy is now a rooted forest of occurrences, as in Definition 5.2.2. Here an internal occurrence may have arity greater than two, and its label is any reference in $\mathcal{L}$ containing all child labels. If an occurrence labelled P has children labelled $Q_1, \dots, Q_r$, then those labels form a reference hyperarc with parent P.

Definition A.1.7 (Bounded-arity reference hyperforest) *For $2 \leq d \leq m$, a d-ary reference hyperforest for $\mathcal{S}$ on $\mathcal{L}$ is a rooted forest $\mathcal{T}$ of occurrences with labels in $\mathcal{L}$. Each internal occurrence has between 2 and d children, and its label together with the labels of its children forms a reference hyperarc. For every input index i, there is a distinguished occurrence a_i with label S_i. These occurrences are reachable from the component roots and remain distinct even when two input sets are equal.*

The cost of $\mathcal{T}$ is

$$\operatorname{cost}(\mathcal{T}) \;=\; \sum_{\rho \in \operatorname{roots}(\mathcal{T})} \lg \binom{u}{|P_\rho|} \;+\; \sum_{e \in E(\mathcal{T})} \operatorname{cost}(e),$$

where P_ρ is the label of the root occurrence ρ and $E(\mathcal{T})$ is the set of reference hyperarcs induced by the internal occurrences of $\mathcal{T}$.

The definition deliberately permits child labels that coincide with the parent or with one another. This keeps the occurrence forest stable when set labels collapse. The cost still has the lower dimensional form obtained by deleting redundant coordinates. If $Q_j = P$ for some j, every element of P has indicator 1 in coordinate j, so a_b^e vanishes whenever $b_j = 0$, and the multinomial reduces to the multinomial of the remaining children inside P. If $Q_i = Q_j$ for two indices $i \neq j$, the two coordinates carry identical indicators on every element of P, and the multinomial reduces to the multinomial obtained by identifying coordinates i and j.

This permission is needed already by SUM. When a merge of Definition 5.2.2 combines labels $P_a \subseteq P_b$, the union label is P_b, so one child has the same label as the parent. The hyperarc view still charges the same local multinomial, because the coordinate belonging to that child

contributes no choices. Repeated child labels are handled in the same way. Two occurrences may occupy different positions in the forest while carrying the same set label, and the local multinomial identifies their coordinates.

For $2 \le d \le m$, write $\mathfrak{H}_d(\mathcal{S})$ for the family of all reference hyperforests for $\mathcal{S}$ on $\mathcal{L}$ whose internal occurrences have at most d children. The arity bound fixes the admissible local splits, and minimising over this family measures the best hierarchy available at that arity.

Definition A.1.8 (Bounded-arity optimum) *For every $d \in \{2, 3, \dots, m\}$,*

$$\mathrm{OPT}_d(\mathcal{S}) \;=\; \min_{\mathcal{T} \in \mathfrak{H}_d(\mathcal{S})} \mathrm{cost}(\mathcal{T}).$$

The two endpoints of the arity range are already feasible. The trivial hyperforest has m singleton root occurrences labelled $S_1, \dots, S_m$ and no hyperarcs. It is feasible for every $d \ge 2$ and has cost $\sum_i \lg \binom{u}{|S_i|} = H_{wc}(\mathcal{S})$, the per-set baseline of Chapter 4. At the other end, the one-component hyperforest with root label U and a single m-ary hyperarc $U \to (S_1, \dots, S_m)$ is feasible at $d = m$.

The local difference between these endpoints is the difference between marginal and joint encodings. Encoding $S_1, \dots, S_m$ as singleton roots charges m independent binomials against U. Encoding them as siblings of one root charges one m-way Venn multinomial inside U. The same comparison applies at any hyperarc. Once the joint Venn pattern of the children is known, each marginal child subset is known, so a joint multinomial cannot exceed the product of the marginal binomials.

Lemma A.1.2 *Let P be a finite set and $Q_1, \dots, Q_r \subseteq P$. For $b \in \{0,1\}^r$, let*

$$a_b \;=\; \bigl|\{\, x \in P : \mathbf{1}[x \in Q_j] = b_j \text{ for all } j \in [r] \,\}\bigr|.$$

Then

$$\lg \binom{|P|}{(a_b)_{b \in \{0,1\}^r}} \;\le\; \sum_{j=1}^{r} \lg \binom{|P|}{|Q_j|}.$$

A joint-count vector compatible with the marginals is a vector $(\alpha_b)_{b \in \{0,1\}^r}$ of nonnegative integers such that $\sum_b \alpha_b = |P|$ and $\sum_{b : b_j = 1} \alpha_b = |Q_j|$ for every j. The inequality is strict whenever the marginals $|Q_j|$ admit more than one such vector.

Proof. The multinomial $\binom{|P|}{(a_b)}$ counts the labellings $\ell \colon P \to \{0,1\}^r$ whose joint counts are exactly (a_b). Each such ℓ projects to r binary labellings $\ell_j \colon P \to \{0,1\}$, with ℓ_j carrying $|Q_j| = \sum_{b : b_j = 1} a_b$ ones. The map $\ell \mapsto (\ell_1, \dots, \ell_r)$ is injective because the coordinate projections determine ℓ. The codomain is the set of tuples of binary labellings whose j-th coordinate has marginal $|Q_j|$, which has cardinality $\prod_j \binom{|P|}{|Q_j|}$; hence

$$\binom{|P|}{(a_b)} \;\le\; \prod_{j=1}^{r} \binom{|P|}{|Q_j|}.$$

Equality means that the projection reaches every tuple of coordinate labellings with the prescribed marginals. This happens exactly when

all such tuples have joint-count vector (a_b). Equivalently, (a_b) is the unique joint-count vector compatible with the marginals $(|Q_j|)$. If another compatible vector exists, a labelling of P with that vector gives a tuple of coordinate labellings outside the image, so the inequality is strict. □

Lemma A.1.2 explains why a joint sibling description is never more expensive than the corresponding marginal descriptions. At arity m, the one-level hyperforest in $\mathfrak{H}_m(\mathcal{S})$ has one local Venn table over all input sets. Under the coverage convention, this table is the atom-count table of Proposition 5.1.1.

Theorem A.1.3 *Under the convention $U = \bigcup_i S_i$, the one-level m-ary hyperforest with root hyperarc $U \to (S_1, \dots, S_m)$ has cost*

$$\lg \binom{u}{(c_\tau)_{\tau \in R}} \;=\; L^*(\mathcal{S}).$$

Proof. The hyperforest has root U and one hyperarc $U \to (S_1, \dots, S_m)$. The root support cost is $\lg \binom{u}{u} = 0$. By Definition A.1.6, the hyperarc cost is $\lg \binom{u}{(a_b)_{b \in \{0,1\}^m}}$ with $a_b = |\{x \in U : (\mathbf{1}[x \in S_j])_{j=1}^m = b\}|$. Identifying $b \in \{0,1\}^m$ with $\tau = \{j : b_j = 1\} \subseteq [m]$ gives $a_b = c_\tau$, the atom count of Definition 5.1.2. The coverage convention $U = \bigcup_i S_i$ forces $c_\emptyset = 0$, so the multinomial reduces to $\binom{u}{(c_\tau)_{\tau \in R}}$, and $L^*(\mathcal{S}) = \lg \binom{u}{(c_\tau)_{\tau \in R}}$ by Proposition 5.1.1. □

The arity optima can now be compared directly. Increasing d enlarges the feasible family, so the optimum cannot increase. The endpoint $d = m$ reaches the atom count by Theorem A.1.3. The remaining point is that no bounded-arity hyperforest, even with arbitrary references from $\mathcal{L}$, can beat the atom type-class count.

Theorem A.1.4 (Arity chain) *For every $d \in \{2, 3, \dots, m\}$,*

$$L^*(\mathcal{S}) \;=\; \mathrm{OPT}_m(\mathcal{S}) \;\le\; \mathrm{OPT}_d(\mathcal{S}) \;\le\; \mathrm{OPT}_2(\mathcal{S}) \;\le\; H_{wc}(\mathcal{S}).$$

Proof. The trivial hyperforest with singleton roots $S_1, \dots, S_m$ is feasible at $d = 2$ with cost $H_{wc}(\mathcal{S})$, giving the rightmost inequality. The inclusions

$$\mathfrak{H}_2(\mathcal{S}) \subseteq \mathfrak{H}_3(\mathcal{S}) \subseteq \cdots \subseteq \mathfrak{H}_m(\mathcal{S})$$

give the middle inequalities, since every d-ary hyperforest is also feasible when the arity bound is larger. By Theorem A.1.3, $\mathrm{OPT}_m(\mathcal{S}) \le L^*(\mathcal{S})$. It remains to prove $L^*(\mathcal{S}) \le \mathrm{OPT}_d(\mathcal{S})$ for every d.

Fix $\mathcal{T} \in \mathfrak{H}_d(\mathcal{S})$. By Theorem A.1.1, every occurrence label of $\mathcal{T}$ has the form X_B for a nonempty upset $B \subseteq R$. For any collection $\mathcal{S}'$ in the atom type class $\mathcal{C}(R, (c_\tau))$, replace each label X_B by the union X'_B of the atoms of $\mathcal{S}'$ whose patterns lie in B. This preserves all containments because containment is inclusion of upset indices. The same occurrence forest therefore gives a feasible hyperforest for every $\mathcal{S}'$ in the type class. Its root sizes and local Venn counts depend only on the fixed upset labels and on (c_τ).

Encode $\mathcal{S}'$ by recording the actual subset of U at every root occurrence of $\mathcal{T}$ and the Venn labelling of each parent label across the children of its

hyperarc. These records reconstruct all descendants from the roots, and hence reconstruct the distinguished leaves $S'_1, \dots, S'_m$. The encoding is injective, and the number of valid records is at most

$$\prod_{\rho \in \mathrm{roots}(\mathcal{T})} \binom{u}{|P_\rho|} \cdot \prod_{e \in E(\mathcal{T})} \binom{|P_e|}{(a^e_b)_b} \;=\; 2^{\mathrm{cost}(\mathcal{T})}.$$

The type-class count of Proposition 5.1.1 gives $|\mathcal{C}(R,(c_\tau))| = \binom{u}{(c_\tau)_{\tau\in R}} = 2^{L^*(\mathcal{S})}$. Thus $L^*(\mathcal{S}) \le \mathrm{cost}(\mathcal{T})$, and minimising over $\mathcal{T}$ gives $L^*(\mathcal{S}) \le \mathrm{OPT}_d(\mathcal{S})$. □

Theorem A.1.4 identifies binary arity as the most restrictive endpoint of the model. This endpoint is still the one tied to SUM, since its local decisions are pairwise and its query structure uses pairwise containment edges. We now turn from the arity comparison to the binary optimisation problem itself.

A.2 Binary case

We now set $d = 2$, the arity used by SUM. In SUM, each local choice joins two current roots, so the choice can be made by the weighted graph matching step of Algorithm 3. A higher-arity step would have to choose groups of three or more roots at once. The binary restriction also keeps the hierarchy built from pairwise containment steps, the setting used by the query structure of Section 5.3.

A binary hyperarc has the form $(P; A, B)$ with $A, B \subseteq P$. To decode both children from the parent, we partition P according to membership in A and B. Let $k = |A \cap B|$, $l = |A \setminus B|$, $r = |B \setminus A|$, and $a_{(0,0)} = |P \setminus (A \cup B)|$. By Definition A.1.6, the cost of this split is

$$\lg \binom{|P|}{a_{(0,0)}, k, l, r},$$

and $\mathrm{OPT}_2(\mathcal{S})$ minimises the sum of these split costs together with the root support costs $\lg \binom{u}{|P_\rho|}$.

The containment order on $\mathcal{L}$ can be read as a directed acyclic graph. Its vertices are the nonempty references. We draw an arc $P \to Q$ when Q is a strict subset of P, so each arc is a possible one-child containment move. A hierarchy that starts at U and reaches $S_1, \dots, S_m$ then looks like a rooted connection problem on this containment graph.

Suppose each containment arc could be paid for independently. We would put root U at the top, mark $S_1, \dots, S_m$ as terminals, and give each arc $P \to Q$ the cost $\lg \binom{|P|}{|Q|}$. A feasible solution would then be a directed arborescence that starts at U and reaches every terminal. This is exactly a Directed Steiner Tree instance on the containment graph [65].

[65]: Charikar et al. (1999), *Approximation algorithms for directed Steiner problems*

The independent arc model separates the choices below a parent. If an arborescence contains one outgoing arc from P, the model still treats every other outgoing arc as a separate purchase. A binary split behaves differently. Choosing $(P; A, B)$ stores one Venn table for the pair inside P. That table has entries for the four regions determined by both children, so

it has no standalone price for the branch $P \to A$. Both children are part of the same split. Replacing the split by ordinary arcs $P \to h$, $h \to A$, and $h \to B$ would let a Steiner arborescence keep the branch to A and drop the branch to B. That object no longer represents the binary split. The Directed Steiner instance above captures the reachability pattern from U to the input sets through containments. The objective of OPT_2 is different, because each local move buys one two-child split with one joint Venn cost. We therefore keep the hyperforest formulation for the binary model.

Each component root pays its own support cost. If two roots carry labels A and B, the closure also contains their union $M = A \cup B$. We can create a new root labelled M and attach the old roots below it with the binary hyperarc $(M; A, B)$. This move replaces two independent root descriptions with one root description and one joint Venn description of A and B inside M. The lemma proves that this replacement never increases the cost.

Lemma A.2.1 (Root merging) *Let $A, B \subseteq U$, let $M = A \cup B$, and set $k = |A \cap B|$, $l = |A \setminus B|$, and $r = |B \setminus A|$. Then*

$$\lg \binom{u}{|M|} + \lg \binom{|M|}{k, l, r} \;\le\; \lg \binom{u}{|A|} + \lg \binom{u}{|B|}.$$

Proof. The multinomial chain identity gives

$$\binom{u}{|M|}\binom{|M|}{k, l, r} \;=\; \binom{u}{u - |M|, k, l, r}.$$

The tuple $(u - |M|, k, l, r)$ is the Venn count of A and B inside U. Applying Lemma A.1.2 of Section A.1 with parent U and children A, B gives $\binom{u}{u-|M|,k,l,r} \le \binom{u}{|A|}\binom{u}{|B|}$. Taking logarithms proves the claim. □

We can now merge component roots one pair at a time. Pick two roots with labels A and B, insert a new root with label $M = A \cup B$, and make the old roots its two children. The move is valid even when $A \subseteq B$ or $B \subseteq A$, because Definition A.1.6 permits a child label to coincide with its parent. Existing descendant costs do not change, and Lemma A.2.1 shows that the new root charge plus the new split cost is no larger than the two old root charges. Repeating the operation gives a binary hyperforest with one root and no larger cost. Under the coverage convention $U = \bigcup_i S_i$, that root is labelled U, since every element of U appears in some distinguished leaf and all labels lie inside U.

A.2.1 Union-only dynamic programme

Lemma A.2.1 lets us merge all components and study one rooted binary tree. The full OPT_2 model still asks us to choose a reference at every internal node. Suppose the two child subtrees use references A and B. We may choose as parent any reference $P \in \mathcal{L}$ with $A \cup B \subseteq P$, and this choice changes the residue part of the Venn table inside P. By Theorem A.1.1, these possible references are the nonempty upsets of R. Searching the full model therefore means choosing both the binary shape and labels from the lattice closure.

The union-only restriction removes the label choice. Once a node has the input indices $X \subseteq [m]$ below it, its label is forced to be

$$U_X = \bigcup_{i \in X} S_i.$$

The remaining search chooses only how the input indices are split across the binary tree.

Let $\mathfrak{H}_2^{\cup}(\mathcal{S})$ be the family of binary reference hyperforests on $\mathcal{L}$ in which every occurrence v with descendant input set $X_v \subseteq [m]$ has label U_{X_v}. We define

$$\mathrm{OPT}_2^{\cup}(\mathcal{S}) = \min_{\mathcal{T} \in \mathfrak{H}_2^{\cup}(\mathcal{S})} \mathrm{cost}(\mathcal{T}),$$

and the inclusion $\mathfrak{H}_2^{\cup}(\mathcal{S}) \subseteq \mathfrak{H}_2(\mathcal{S})$ gives $\mathrm{OPT}_2(\mathcal{S}) \leq \mathrm{OPT}_2^{\cup}(\mathcal{S})$. The inequality can be strict, since the full model may use an intersection-derived label that the union-only model forbids. By Lemma A.2.1, we may restrict the minimum in $\mathfrak{H}_2^{\cup}(\mathcal{S})$ to one binary tree rooted at $U_{[m]}$.

We can now see the recursive structure. Take a node whose descendant input set is X. If X is a singleton, the node is one of the required leaves and has no split cost. If $|X| \geq 2$, the first decision below that node is a bipartition of X. Write it as Y and $X \setminus Y$. The union-only rule fixes the two child labels as U_Y and $U_{X \setminus Y}$. After paying the Venn cost of this split, the left subtree depends only on Y, and the right subtree depends only on $X \setminus Y$. The same problem has reappeared on two smaller input subsets, so we compute the optimum values for smaller subsets first and reuse them.

To turn this recursive description into a cost recurrence, we separate the cost of storing a root from the cost of splitting an internal node. If a subtree on the input indices $X \subseteq [m]$ becomes a component root, it pays

$$\rho(X) = \lg \binom{u}{|U_X|}.$$

If a split sends the nonempty disjoint sets Y and Z to the two children, the parent label is $U_{Y \cup Z}$ and the child labels are U_Y and U_Z. Let

$$\mu(Y, Z) = \lg \binom{|U_{Y \cup Z}|}{k, l, r},$$

where $k = |U_Y \cap U_Z|$, $l = |U_Y \setminus U_Z|$, and $r = |U_Z \setminus U_Y|$. Since $U_{Y \cup Z} = U_Y \cup U_Z$, every element of the parent lies in at least one child. The residue count $a_{(0,0)}$ of Definition A.1.6 is zero, and $\mu(Y, Z)$ is exactly the cost of the binary hyperarc $U_{Y \cup Z} \to (U_Y, U_Z)$.

The subproblem value should not charge the root support cost at every level. We let $B(X)$ count only the internal split cost of the best binary union tree whose distinguished leaves are exactly the input indices in X. For a singleton, no internal split is needed, so $B(\{i\}) = 0$. For $|X| \geq 2$, a top split chooses a nonempty proper subset $Y \subsetneq X$ and puts $X \setminus Y$ on the other side. That choice contributes the two smaller split costs plus the local cost $\mu(Y, X \setminus Y)$, so

$$B(X) = \min_{\substack{\emptyset \neq Y \subsetneq X \\ \min(X) \in Y}} \left[B(Y) + B(X \setminus Y) + \mu(Y, X \setminus Y) \right].$$

The constraint $\min(X) \in Y$ selects one representative of the symmetric split $(Y, X \setminus Y)$. Only the root of the whole tree pays a support cost, so the minimum union-only cost is $\rho([m]) + B([m])$. Under the coverage convention, $\rho([m]) = \lg \binom{u}{u} = 0$.

The recurrence for $B(X)$ refers only to proper subsets of X. We can therefore evaluate it by increasing subset size. Algorithm 9 first computes every union U_X, since each evaluation of $\mu(Y, Z)$ needs the three sets U_Y, U_Z, and $U_{Y \cup Z}$. It then initializes the singleton values and fills larger subsets. After finishing all subsets of size s, it has computed the correct value $B(X)$ for every nonempty X with $|X| \le s$.

Algorithm 9: Computing $\mathrm{OPT}_2^{\cup}$.

Input: Collection $\mathcal{S} = \{S_1, \ldots, S_m\}$ over universe U.
Output: The value $\mathrm{OPT}_2^{\cup}(\mathcal{S})$.

```
compute U_X = ⋃_{i∈X} S_i for every X ⊆ [m];
foreach i ∈ [m] do
    B({i}) ← 0;
for s = 2, 3, ..., m do
    foreach X ⊆ [m] with |X| = s do
        B(X) ← +∞;
        foreach Y ⊊ X with Y ≠ ∅ and min(X) ∈ Y do
            Z ← X \ Y;
            compute μ(Y, Z) from U_Y, U_Z, and U_X;
            B(X) ← min{B(X), B(Y) + B(Z) + μ(Y, Z)};
return ρ([m]) + B([m]);
```

For a subset X, the loop tries every admissible first split and combines the local split cost with the best values already computed for the two sides. The table therefore represents all binary union trees through their recursive top splits, without listing the trees explicitly. The proposition turns this invariant into the exact value of $\mathrm{OPT}_2^{\cup}$ and counts the split evaluations needed to compute it.

Proposition A.2.2 (Subset dynamic programme) *For every collection $\mathcal{S}$ of m sets over a universe of size u,*

$$\mathrm{OPT}_2^{\cup}(\mathcal{S}) \;=\; \rho([m]) + B([m]).$$

The dynamic programme in Algorithm 9 computes this value in $O(3^m \cdot u/\omega)$ word operations after precomputing the unions $\{U_X : X \subseteq [m]\}$ in $O(2^m \cdot u/\omega)$ word operations on the word RAM.

Proof. First restrict the optimum to one component. If two roots in a union-only hyperforest have disjoint descendant index sets X and Y, their labels are U_X and U_Y. Replacing them by a new root labelled $U_{X \cup Y}$ with children U_X and U_Y stays in $\mathfrak{H}_2^{\cup}(\mathcal{S})$, since $U_{X \cup Y} = U_X \cup U_Y$. The descendant subtrees keep the same costs, and Lemma A.2.1 shows that the new root charge plus the new split cost is no larger than the two old root charges. Repeating this operation leaves one binary tree rooted at $U_{[m]}$ with no larger cost.

We prove by induction on $|X|$ that $B(X)$ is the minimum internal split cost of a binary union tree with leaf set X. The singleton case has cost 0.

Let $|X| \geq 2$. Any valid tree with leaf set X has a top split whose child leaf sets form a nonempty partition Y and $X \setminus Y$. The union-only rule fixes the child labels as U_Y and $U_{X\setminus Y}$, and the top split contributes $\mu(Y, X \setminus Y)$. By the induction hypothesis, the two child subtrees cost at least $B(Y)$ and $B(X \setminus Y)$, so every valid tree costs at least one candidate value in the recurrence.

Conversely, choose any subset Y considered by the recurrence. By the induction hypothesis, there are optimal child trees for Y and $X \setminus Y$. Placing them under the hyperarc $U_X \to (U_Y, U_{X\setminus Y})$ gives a valid binary union tree for X with cost $B(Y)+B(X\setminus Y)+\mu(Y, X\setminus Y)$. The condition $\min(X) \in Y$ selects one side of each unordered bipartition, so the recurrence neither misses a split nor needs its symmetric copy. Thus $B(X)$ is optimal for every X. Adding the root support cost gives $\mathrm{OPT}_2^{\cup}(\mathcal{S}) = \rho([m]) + B([m])$, and Algorithm 9 returns this value.

For the running time, a subset X has at most $2^{|X|-1}$ candidates satisfying $\min(X) \in Y$. Summing over all nonempty subsets gives

$$\sum_{j=1}^{m} \binom{m}{j} 2^{j-1} \;=\; \frac{3^m - 1}{2} \;=\; O(3^m).$$

The unions U_X are precomputed by iterating over the subset lattice and applying $U_X = U_{X\setminus\{i\}} \cup S_i$ for any $i \in X$, in $O(u/\omega)$ word operations per subset and $O(2^m \cdot u/\omega)$ word operations in total. Each evaluation of $\mu(Y, Z)$ from the precomputed unions takes $O(u/\omega)$ word operations, so the dynamic programme itself runs in $O(3^m \cdot u/\omega)$ word operations. □

The result is best read as an exact baseline for small values of m. Once m is fixed, the dependence on the universe is polynomial through the bitset operations on the precomputed unions, while the search over binary shapes costs 3^m. The dynamic programme therefore computes the best binary hierarchy whose internal labels are forced unions, the same label family used by SUM, and gives an exact comparison value inside that restricted model. It does not give a polynomial-time algorithm in the full input size. It also leaves the full OPT_2 separate, since that problem may choose internal labels anywhere in $\mathcal{L}$.

Bibliography

[1] Alessio Conte et al., eds. *From Strings to Graphs, and Back Again: A Festschrift for Roberto Grossi's 60th Birthday*. 2025 (cited on page iii).

[2] Jarno N Alanko et al. 'Compact data structures for collections of sets'. In: *23rd Symposium on Experimental Algorithms*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik. 2025, p. 6 (cited on pages iii, 3, 4, 54–58, 89).

[3] Luca Lombardo. 'Efficient Succinct Data Structures on Directed Acyclic Graphs'. Bachelor's Thesis. University of Pisa, May 2025 (cited on page iii).

[4] Ming Li, Paul Vitányi, et al. *An introduction to Kolmogorov complexity and its applications*. Vol. 3. Springer, 2008 (cited on page 1).

[5] Paolo Boldi and Sebastiano Vigna. 'The webgraph framework I: compression techniques'. In: *Proceedings of the 13th international conference on World Wide Web*. 2004, pp. 595–602 (cited on pages 2, 62).

[6] Giulio Ermanno Pibiri and Rossano Venturini. 'Techniques for Inverted Index Compression'. In: *ACM Computing Surveys* (2020). Draft metadata from local PDF (cited on page 2).

[7] Jarno N Alanko et al. 'Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes'. In: *Bioinformatics* 39.Supplement_1 (2023), pp. i260–i269 (cited on page 2).

[8] Fatemeh Almodaresi et al. 'An efficient, scalable, and exact representation of high-dimensional color information enabled using de Bruijn graph search'. In: *Journal of Computational Biology* 27.4 (2020), pp. 485–499 (cited on pages 2, 62).

[9] Travis Gagie, Meng He, and Gonzalo Navarro. 'Compressed Set Representations based on Set Difference'. In: *arXiv preprint arXiv:2601.23240* (2026) (cited on pages 3, 4, 41, 46, 48, 49, 54, 55, 58–70, 88–90).

[10] Gonzalo Navarro. *Compact data structures: A practical approach*. Cambridge University Press, 2016 (cited on pages 3, 7, 10).

[11] Guy Jacobson. 'Succinct Static Data Structures'. PhD thesis. Pittsburgh, PA: Carnegie Mellon University, 1988 (cited on pages 7, 9, 10, 31).

[12] Guy Jacobson. 'Space-efficient static trees and graphs'. In: *30th annual symposium on foundations of computer science*. IEEE. 1989, pp. 549–554 (cited on pages 7, 26, 27, 31).

[13] V. L. Arlazarov et al. 'On economical construction of the transitive closure of a directed graph'. In: *Doklady Akademii Nauk SSSR* 194 (1970). English translation in Soviet Math. Dokl. 11 (1970), pp. 1209–1210, pp. 487–488 (cited on pages 8, 31).

[14] David Richard Clark. 'Compact Pat Trees'. PhD thesis. Waterloo, Ontario, Canada: University of Waterloo, 1996 (cited on pages 9, 10).

[15] Peter Bro Miltersen. 'Lower Bounds on the Size of Selection and Rank Indexes'. In: *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2005, Vancouver, British Columbia, Canada, January 23–25, 2005*. SIAM. 2005, pp. 11–12 (cited on page 11).

[16] Alexander Golynski. 'Optimal Lower Bounds for Rank and Select Indexes'. In: *Automata, Languages and Programming, 33rd International Colloquium, ICALP 2006, Venice, Italy, July 10–14, 2006, Proceedings, Part I*. Vol. 4051. Lecture Notes in Computer Science. Journal version: Theoretical Computer Science 387(3), 2007, pp. 348–359. Springer, 2006, pp. 370–381 (cited on pages 11, 12).

[17] Paul Beame and Faith E. Fich. 'Optimal Bounds for the Predecessor Problem and Related Problems'. In: *Journal of Computer and System Sciences* 65.1 (2002), pp. 38–72 (cited on page 12).

[18] Mihai Pătraşcu. 'Succincter'. In: *49th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2008, Philadelphia, PA, USA, October 25–28, 2008*. IEEE. 2008, pp. 305–313 (cited on pages 12, 13).

[19] Roberto Grossi et al. 'More Haste, Less Waste: Lowering the Redundancy in Fully Indexable Dictionaries'. In: *26th International Symposium on Theoretical Aspects of Computer Science, STACS 2009, Freiburg, Germany, February 26–28, 2009, Proceedings*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. 2009, pp. 517–528 (cited on page 12).

[20] Venkatesh Raman and S. Srinivasa Rao. 'Static Dictionaries Supporting Rank'. In: *Algorithms and Computation, 10th International Symposium, ISAAC '99, Chennai, India, December 16–18, 1999, Proceedings*. Vol. 1741. Lecture Notes in Computer Science. Springer, 1999, pp. 18–26 (cited on page 13).

[21] Rajeev Raman, Venkatesh Raman, and Srinivasa Rao Satti. 'Succinct indexable dictionaries with applications to encoding k-ary trees, prefix sums and multisets'. In: *ACM Transactions on Algorithms (TALG)* 3.4 (2007), 43–es (cited on pages 13, 14).

[22] Michael L. Fredman, János Komlós, and Endre Szemerédi. 'Storing a Sparse Table with $O(1)$ Worst Case Access Time'. In: *Journal of the ACM* 31.3 (1984), pp. 538–544 (cited on page 13).

[23] Peter Elias. 'Efficient Storage and Retrieval by Content and Address of Static Files'. In: *Journal of the ACM* 21.2 (1974), pp. 246–260 (cited on page 15).

[24] Robert M. Fano. *On the number of bits required to implement an associative memory*. Tech. rep. Memorandum 61. Cambridge, Massachusetts: Computer Structures Group, Project MAC, Massachusetts Institute of Technology, 1971 (cited on page 15).

[25] Roberto Grossi and Jeffrey Scott Vitter. 'Compressed Suffix Arrays and Suffix Trees with Applications to Text Indexing and String Matching'. In: *SIAM Journal on Computing* 35.2 (2005), pp. 378–407 (cited on pages 15, 16).

[26] Abraham Bookstein and Shmuel T. Klein. 'Compression of correlated bit-vectors'. In: *Information Systems* 16.4 (1991), pp. 387–400 (cited on pages 17, 61).

[27] Daisuke Okanohara and Kunihiko Sadakane. 'Practical entropy-compressed rank/select dictionary'. In: *2007 Proceedings of the Ninth Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM. 2007, pp. 60–70 (cited on pages 17, 54).

[28] Sebastiano Vigna. 'Quasi-Succinct Indices'. In: *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining, WSDM 2013, Rome, Italy, February 4–8, 2013*. ACM. 2013, pp. 83–92 (cited on page 17).

[29] Kunihiko Sadakane and Roberto Grossi. 'Squeezing Succinct Data Structures into Entropy Bounds'. In: *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2006, Miami, Florida, USA, January 22–26, 2006*. SIAM. 2006, pp. 1230–1239 (cited on page 18).

[30] Bernard Chazelle. 'A Functional Approach to Data Structures and Its Use in Multidimensional Searching'. In: *SIAM Journal on Computing* 17.3 (1988), pp. 427–462 (cited on page 19).

[31] Roberto Grossi, Ankur Gupta, and Jeffrey Scott Vitter. 'High-order entropy-compressed text indexes'. In: *SODA*. Vol. 3. 2003, pp. 841–850 (cited on pages 19, 22, 23, 47).

[32] Gonzalo Navarro. 'Wavelet Trees for All'. In: *Journal of Discrete Algorithms* 25 (2014), pp. 2–20 (cited on page 22).

[33] Paolo Ferragina et al. 'Compressed Representations of Sequences and Full-Text Indexes'. In: *ACM Transactions on Algorithms* 3.2 (2007), p. 20 (cited on pages 24, 50).

[34] J Ian Munro and Venkatesh Raman. 'Succinct representation of balanced parentheses and static trees'. In: *SIAM Journal on Computing* 31.3 (2001), pp. 762–776 (cited on pages 25, 26, 30).

[35] Gonzalo Navarro and Kunihiko Sadakane. 'Fully Functional Static and Dynamic Succinct Trees'. In: *ACM Transactions on Algorithms* 10.3 (2014), 16:1–16:39 (cited on pages 26, 27, 34, 37, 38).

[36] David Benoit et al. 'Representing Trees of Higher Degree'. In: *Algorithmica* 43.4 (2005), pp. 275–292 (cited on pages 27, 32, 33).

[37] J. Ian Munro. 'Tables'. In: *Foundations of Software Technology and Theoretical Computer Science, 16th Conference, FSTTCS 1996, Hyderabad, India, December 18–20, 1996, Proceedings*. Vol. 1180. Lecture Notes in Computer Science. Springer, 1996, pp. 37–42 (cited on page 31).

[38] Michael A. Bender and Martín Farach-Colton. 'The LCA Problem Revisited'. In: *LATIN 2000: Theoretical Informatics, 4th Latin American Symposium, Punta del Este, Uruguay, April 10–14, 2000, Proceedings*. Vol. 1776. Lecture Notes in Computer Science. Springer, 2000, pp. 88–94 (cited on pages 31, 36, 64).

[39] Meng He, J. Ian Munro, and S. Srinivasa Rao. 'Succinct Ordinal Trees Based on Tree Covering'. In: *Automata, Languages and Programming, 34th International Colloquium, ICALP 2007, Wroclaw, Poland, July 9-13, 2007, Proceedings*. Vol. 4596. Lecture Notes in Computer Science. Journal version: ACM Transactions on Algorithms 8(4), 2012, Article 42. Springer, 2007, pp. 509–520 (cited on pages 34, 36, 37).

[40] Richard F. Geary, Rajeev Raman, and Venkatesh Raman. 'Succinct Ordinal Trees with Level-Ancestor Queries'. In: *ACM Transactions on Algorithms* 2.4 (2006), pp. 510–534 (cited on pages 34, 36, 37, 40, 52).

[41] Arash Farzan and J. Ian Munro. 'A Uniform Approach Towards Succinct Representation of Trees'. In: *Algorithm Theory – SWAT 2008, 11th Scandinavian Workshop on Algorithm Theory, Gothenburg, Sweden, July 2–4, 2008, Proceedings*. Lecture Notes in Computer Science. Journal version in Algorithmica 2014. Springer, 2008, pp. 173–184 (cited on pages 37, 52).

[42] Meng He, J Ian Munro, and Gelin Zhou. 'A framework for succinct labeled ordinal trees over large alphabets'. In: *Algorithmica* 70.4 (2014), pp. 696–717 (cited on pages 39–46, 83).

[43] Kuo-Chung Tai. 'The tree-to-tree correction problem'. In: *Journal of the ACM* 26.3 (1979), pp. 422–433. DOI: 10.1145/322139.322143 (cited on page 41).

[44] Philip Bille. 'A survey on tree edit distance and related problems'. In: *Theoretical Computer Science* 337.1–3 (2005), pp. 217–239. DOI: 10.1016/j.tcs.2004.12.030 (cited on page 41).

[45] Meng He, J. Ian Munro, and Gelin Zhou. 'Path queries in weighted trees'. In: *Algorithms and Computation (ISAAC 2011)*. Ed. by Takao Asano et al. Vol. 7074. Lecture Notes in Computer Science. Springer, 2011, pp. 140–149. DOI: 10.1007/978-3-642-25591-5_16 (cited on page 41).

[46] Meng He, J Ian Munro, and Gelin Zhou. 'Data structures for path queries'. In: *ACM Transactions on Algorithms (TALG)* 12.4 (2016), pp. 1–32 (cited on pages 41, 42, 46–52, 69, 83).

[47] Djamal Belazzougui and Gonzalo Navarro. 'Optimal lower and upper bounds for representing sequences'. In: *ACM Transactions on Algorithms* 11.4 (2015), 31:1–31:21. DOI: 10.1145/2629339 (cited on page 45).

[48] Jérémy Barbay et al. 'Succinct indexes for strings, binary relations and multi-labeled trees'. In: *ACM Transactions on Algorithms* 7.4 (2011), 52:1–52:27. DOI: 10.1145/2000807.2000820 (cited on page 45).

[49] Alexander Golynski, J. Ian Munro, and S. Srinivasa Rao. 'Rank/select operations on large alphabets: a tool for text indexing'. In: *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2006)*. SIAM. 2006, pp. 368–373 (cited on page 45).

[50] David R. Clark and J. Ian Munro. 'Efficient suffix trees on secondary storage (extended abstract)'. In: *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 1996)*. Distinct from Clark's 1996 PhD thesis "Compact Pat Trees" already in this bib. SIAM. 1996, pp. 383–391 (cited on page 45).

[51] Prosenjit Bose et al. 'Succinct orthogonal range search structures on a grid with applications to text indexing'. In: *Algorithms and Data Structures Symposium (WADS 2009)*. Vol. 5664. Lecture Notes in Computer Science. Springer, 2009, pp. 98–109. DOI: 10.1007/978-3-642-03367-4_9 (cited on page 50).

[52] Timothy M. Chan, Kasper Green Larsen, and Mihai Pătraşcu. 'Orthogonal range searching on the RAM, revisited'. In: *Proceedings of the 27th Annual Symposium on Computational Geometry (SoCG 2011)*. ACM, 2011, pp. 1–10. DOI: 10.1145/1998196.1998198 (cited on page 51).

[53] Adam L. Buchsbaum et al. 'Engineering the compression of massive tables: an experimental approach'. In: *Proceedings of the Eleventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2000)*. ACM/SIAM, 2000, pp. 175–184 (cited on page 61).

[54] Andreas Björklund and Andrzej Lingas. 'Fast Boolean Matrix Multiplication for Highly Clustered Data'. In: *Algorithms and Data Structures, 7th International Workshop (WADS 2001), Providence, RI, USA, August 8–10, 2001, Proceedings*. Vol. 2125. Lecture Notes in Computer Science. Springer, 2001, pp. 258–263 (cited on page 61).

[55] José N. F. Alves et al. 'Accelerating Graph Neural Networks Using a Novel Computation-Friendly Matrix Compression Format'. In: *IEEE International Parallel and Distributed Processing Symposium (IPDPS 2025)*. IEEE, 2025, pp. 1091–1103 (cited on page 62).

[56] Alberto Apostolico. 'The myriad virtues of subword trees'. In: *Combinatorial Algorithms on Words*. Vol. 12. NATO ASI Series F. Springer-Verlag, 1985, pp. 85–96 (cited on page 64).

[57] Martín Farach-Colton, Paolo Ferragina, and S. Muthukrishnan. 'On the sorting-complexity of suffix tree construction'. In: *Journal of the ACM* 47.6 (2000), pp. 987–1011 (cited on page 64).

[58] Steven Givant. 'Atoms'. In: *Introduction to Boolean Algebras*. New York, NY: Springer New York, 2009, pp. 117–126 (cited on page 72).

[59] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd ed. Wiley-Interscience, 2006 (cited on page 74).

[60] Harold N. Gabow. 'Data structures for weighted matching and extensions to b-matching and f-factors'. In: *ACM Transactions on Algorithms* 14.3 (2018), 39:1–39:80. DOI: 10.1145/3183369 (cited on page 80).

[61] Andrei Z. Broder. 'On the resemblance and containment of documents'. In: *Compression and Complexity of SEQUENCES 1997*. IEEE, 1997, pp. 21–29. DOI: 10.1109/SEQUEN.1997.666900 (cited on page 90).

[62] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 'Similarity Search in High Dimensions via Hashing'. In: *Proceedings of the 25th International Conference on Very Large Data Bases*. Morgan Kaufmann, 1999, pp. 518–529 (cited on page 90).

[63] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Berlin: Springer, 1999 (cited on pages 93, 94).

[64] Daniel Kleitman. 'On Dedekind's problem: The number of monotone Boolean functions'. In: *Proceedings of the American Mathematical Society* 21.3 (1969), pp. 677–682 (cited on page 95).

[65] Moses Charikar et al. 'Approximation algorithms for directed Steiner problems'. In: *Journal of Algorithms* 33.1 (1999). Preliminary version in SODA 1998, pp. 192–200, pp. 73–91 (cited on page 99).